学校代码：10284
分 类 号：TP399
密　　级：公开
U D C：004.22
学　　号：502023330036



南京大学

硕 士 学 位 论 文

论 文 题 目　面向复杂负载的范围过滤器优化及其弹性缓存应用技术研究

作 者 姓 名　刘非凡

专 业 名 称　计算机科学与技术

研 究 方 向　大数据与云计算

导 师 姓 名　顾荣　研究员

2026 年 5 月 25 日

答辩委员会主席 程龚 教授

评　　阅　　人 程龚 教授

高家全 教授

论文答辩日期　2026 年 5 月 14 日

研究生签名：刘非凡

导师签名：[illegible]

Research on Range Filter Optimization for Complex Workloads and Its Application in Elastic Caching Techniques

by

Feifan Liu

Supervised by

Research Professor Rong Gu

A dissertation submitted to

the graduate school of Nanjing University

in partial fulfilment of the requirements for the degree of

Master

in

Computer Science and Technology



School of Computer Science

Nanjing University

May 25, 2026

南京大学学位论文原创性声明

本人郑重声明，所提交的学位论文是本人在导师指导下独立进行科学研究工作所取得的成果。除本论文中已经注明引用的内容外，本论文不包含其他个人或集体已经发表或撰写过的研究成果，也不包含为获得南京大学或其他教育机构的学位证书而使用过的材料。对本文的研究做出重要贡献的个人和集体，均已在论文的致谢部分明确标明。

本人郑重声明愿承担本声明的法律责任。

签名：刘非凡

日期：2026年5月25日

南京大学研究生毕业论文中文摘要首页用纸

毕业论文题目：面向复杂负载的范围过滤器优化及其弹性缓存应用技术研究

计算机科学与技术 专业 2023 级硕士生姓名： 刘非凡

指导教师（姓名、职称）： 顾荣 研究员

摘 要

近年来，大数据分析场景持续拓展，大模型服务应用快速普及，推动数据规模呈现爆发式增长，数据访问模式也日趋复杂。海量数据的持续累积使现有数据系统在查询时延、吞吐能力和资源利用效率等方面面临日益突出的瓶颈，系统性能与运行成本之间的矛盾不断加剧。具体而言，数据查询与批量扫描操作容易引发大量无效 I/O 与冗余计算，数据热点分布不均与负载倾斜又会进一步放大局部访问压力，增加系统整体运行负担。范围过滤器作为一类面向范围查询的近似数据结构，能够以极低的空间开销对数据分布特征进行紧凑刻画，并在范围查询执行前快速筛除无命中可能的数据块，从而减少无效 I/O、缩小扫描范围、降低冗余计算开销。然而，在复杂多变的实际负载场景下，现有的范围过滤器的优化设计仍面临以下挑战：其一，传统范围过滤器在应对数据倾斜与对抗性查询时易出现重复误判，导致过滤精度与端到端查询性能持续退化；其二，紧凑编码的范围过滤器通常缺乏高效的在线增删能力，难以适应动态数据负载；其三，范围过滤器的性能优化成果如何在实际工程系统中转化为稳定、可观测的收益，仍缺乏成熟的应用验证路径。

现有相关工作主要分别针对静态范围过滤精度或系统缓存调度进行优化，且缺乏一条从范围过滤器结构优化到系统级应用验证的统一技术路径。针对上述问题，本文围绕面向复杂负载的范围过滤器优化及其弹性缓存应用展开系统研究。本文的主要研究工作和贡献点包括：

（1）针对倾斜查询负载下范围过滤器适应性与鲁棒性难以兼顾的问题，本文提出了面向倾斜负载的自适应范围过滤器——Hourglass。该方法通过半排序指纹组织、紧凑混合后缀编码以及滑动窗口自适应更新机制，在保持空间效率的同时有效抑制重复误判，并对数据结构的构建访存过程进行了系统性优化。

（2）针对动态更新场景下紧凑范围过滤器难以在线维护的问题，本文提出了面向更新负载的后缀位级化增删技术。该技术在不改变既有物理布局的前提下，设计了基于位级合并的后缀插入策略、基于二分补位的后缀删除策略以及前缀槽位级联清除机制，并进一步引入关联度感知的空间动态化分配方法，以兼顾更新效率、空间利用率与过滤精度。

（3）为将上述自适应能力与动态更新能力转化为真实系统中的可观测收益，本文研究了基于自适应动态范围过滤器的多租户弹性缓存方法，并面向 Alluxio 系统设计并实现了弹性缓存调度集成方案。该方法将范围过滤器的近似计数能力用于估计块内有效记录比例，基于阿姆达尔定律构建请求性能收益与块密度协同的数据价值模型，结合 Holt-Winters 自适应价值估计、采样优化与值感知驱逐机制，实现面向差异化性能–成本偏好的在线配额推荐与执行，进而形成了从范围过滤器结构优化、动态更新维护到系统级验证的完整技术路径。

实验结果表明，在对抗性工作负载下，Hourglass 的误判率较前沿方法降低 9.8–35.4 倍，端到端查询延迟降低 1.4–37.7 倍，构建吞吐提升近 4 倍；所提出的后缀位级化增删技术在典型动态删除场景下较重建式参考方案具有显著更低的更新开销，并在多轮插入与删除后保持稳定、较低的误判率；面向 Alluxio 的多租户弹性缓存集成方案在稀疏过滤负载下与 Alluxio-LRU 查询性能总体持平的同时，缓存成本比例降低 59.5%，单位成本收益提升至其 2.38 倍；在热点迁移且成本优先阶段，系统以平均 238 页缓存保持 1.004–1.500 的滚动加速比，相比 Alluxio-LRU 的 4,000 页缓存缩减约 94.0%，平均滚动加速比仅下降约 6.1%。

关键词：范围过滤器；自适应过滤器；动态更新；弹性缓存；多租户调度

南京大学研究生毕业论文英文摘要首页用纸

THESIS: Research on Range Filter Optimization for Complex Workloads and Its Application in Elastic Caching Techniques

SPECIALIZATION: Computer Science and Technology

POSTGRADUATE: Feifan Liu

MENTOR: Research Professor Rong Gu

ABSTRACT

In recent years, big data analysis scenarios have been continuously expanded, and large model services have gained rapid popularity, triggering explosive growth in data volume and increasingly sophisticated data access patterns. The continuous accumulation of massive data has brought prominent bottlenecks to existing data systems in terms of query latency, throughput and resource utilization efficiency, and intensified the contradiction between system performance and operational costs. Specifically, data queries and batch scanning operations incur substantial invalid I/O operations and redundant computations. Uneven distribution of data hotspots and load skew further exacerbate local access pressure and raise the overall system operating overhead.

As a type of approximate data structure dedicated to range queries, range filters can concisely depict data distribution characteristics with extremely low space overhead. They can rapidly filter out data blocks with no potential hits prior to range query execution, so as to reduce invalid I/O, narrow down scanning ranges and cut redundant computational costs. Nevertheless, the optimized design of existing range filters still faces severe challenges under complex and dynamic practical workloads. Firstly, traditional range filters are prone to repeated misjudgments when coping with data skew and adversarial queries, which leads to continuous deterioration of filtering accuracy and end-to-end query performance. Secondly, compactly encoded range filters generally lack efficient online insertion and deletion capabilities, making them incompetent for dynamic data workloads. Thirdly, there is still no mature application verification paradigm to convert performance optimization achievements of range filters into stable

and measurable practical benefits in real engineering systems.

Current relevant researches mainly conduct separate optimizations on static range filtering accuracy, dynamic update efficiency and system cache scheduling, while lacking a unified technical roadmap from structural optimization of range filters to system-level application validation. Targeting the above problems, this thesis carries out systematic research on range filter optimization for complex workloads and its elastic cache applications. The major research contents and contributions are presented as follows:

(1) To resolve the dilemma of balancing adaptability and robustness of range filters under skewed query workloads, this thesis proposes Hourglass, an adaptive range filter tailored for skewed workloads. Equipped with semi-sorted fingerprint organization, compact hybrid suffix encoding and sliding window adaptive update mechanism, this method effectively suppresses repeated misjudgments while preserving high space efficiency, and achieves systematic optimization on memory access during data structure construction.

(2) Aiming at the difficulty of online maintenance of compact range filters in dynamic update scenarios, this thesis puts forward a suffix bit-level insertion and deletion technique oriented to update-intensive workloads. Without modifying the original physical layout, this technique designs a bit-level merging based suffix insertion strategy, a binary padding based suffix deletion strategy and a cascaded clearing mechanism for prefix slots. Furthermore, a correlation-aware dynamic space allocation scheme is introduced to strike a fine balance among update efficiency, space utilization and filtering accuracy.

(3) To translate the aforementioned adaptive regulation and dynamic update capabilities into measurable benefits in practical systems, this thesis investigates a multi-tenant elastic caching method based on adaptive dynamic range filters, and implements the corresponding integrated solution for the Alluxio system. This method adopts the approximate counting property of range filters to estimate the proportion of valid records inside data blocks. It constructs a data value model combining request performance gain and block density based on Amdahl' s law. Combined with Holt-Winters adaptive value estimation, sampling optimization and value-aware eviction strategy, it realizes

online cache quota recommendation and execution adapting to diverse performance-cost preferences, thus forming a complete technical workflow covering range filter structure optimization, dynamic update maintenance and system-level verification.

Experimental results demonstrate that under adversarial workloads, Hourglass reduces the false positive rate by 9.8–35.4 times and end-to-end query latency by 1.4–37.7 times compared with state-of-the-art methods, and nearly quadruples the construction throughput. The proposed suffix bit-level insertion-deletion technique achieves much lower update overhead than reconstruction-based baseline methods in typical dynamic deletion scenarios, and maintains stable and low false positive rates after frequent insertions and deletions. When deployed on Alluxio, the proposed multi-tenant elastic cache solution achieves comparable overall query performance with Alluxio-LRU under sparse filtering workloads, reduces the cache cost ratio by 59.5%, and boosts the benefit per unit cost to 2.38 times that of Alluxio-LRU. In hotspot migration and cost-prioritized scenarios, the system maintains a rolling speedup ratio of 1.004–1.500 with an average cache size of only 238 pages. Compared with the 4,000-page cache configuration of Alluxio-LRU, the cache usage is reduced by approximately 94.0%, while the average rolling speedup ratio only drops by around 6.1%.

KEYWORDS: range filters; adaptive filters; dynamic updates; elastic caching; multi-tenant scheduling

目　录

第一章 绪论 1

1.1 研究背景 1

1.2 现有相关工作及不足 3

1.2.1 范围过滤器 3

1.2.2 自适应及全局过滤器 5

1.2.3 弹性缓存管理 6

1.3 研究难点分析 8

1.4 本文主要研究工作 9

1.5 本文的组织结构 10

第二章 背景知识 13

2.1 范围过滤器基本原理 13

2.1.1 布谷鸟过滤器 13

2.1.2 半排序布谷鸟过滤器 14

2.1.3 常用范围过滤器 14

2.2 复杂查询负载特征 18

2.2.1 范围查询关联度 18

2.2.2 对抗性查询 18

2.2.3 Q/U 比率与查询重复性 19

2.3 弹性缓存基本原理 19

2.3.1 数据系统分层架构 19

2.3.2 弹性缓存与工作集大小估计 20

2.3.3 多租户弹性缓存问题 21

第三章 面向倾斜负载的范围过滤器自适应指纹更新技术 23

3.1 问题背景与分析 23
3.2 基于半排序指纹的范围查询过滤器总体设计 26
3.2.1 设计目标 26
3.2.2 数据结构 28
3.2.3 构建与查询过程 30
3.3 基于滑动窗口的范围过滤器自适应更新机制 33
3.3.1 自适应更新机制 33
3.3.2 自适应更新理论保证 34
3.4 基于处理器架构的范围过滤器构建访存优化 36
3.4.1 向量化空槽检测 36
3.4.2 预取式延迟隐藏 37
3.4.3 大页化 TLB 优化 38
3.4.4 两阶段顺序写入 39
3.5 实验与性能评估 40
3.5.1 实验环境与配置 40
3.5.2 误判率评估 41
3.5.3 运行性能评估 44
3.5.4 鲁棒性分析 48
3.5.5 结果分析与讨论 49
3.6 本章小结 50

第四章 面向更新负载的范围过滤器后缀位级化增删技术 51
4.1 问题背景与分析 51
4.2 基于位级合并的过滤器后缀插入策略 52
4.2.1 后缀编码格式体系 52
4.2.2 后缀插入流程 54
4.2.3 二进制格式到位合并格式的转换 54
4.2.4 计数布隆过滤器与安全写入 55
4.3 基于二分补位的过滤器后缀删除策略 55
4.3.1 后缀删除流程 55

4.3.2 二进制格式的位移补位删除 56
4.3.3 位合并格式与位图格式的清位策略 56
4.3.4 前缀槽位的级联清除 57
4.4 关联度感知的过滤器空间动态化分配 57
4.4.1 空间估算模型 57
4.4.2 关联度感知的空间分配模型 58
4.4.3 误判率与空间复杂度分析 59
4.5 实验与性能评估 61
4.5.1 实验环境与配置 62
4.5.2 动态插入与删除吞吐性能评估 62
4.5.3 多轮插入下的范围误判率趋势评估 64
4.5.4 多轮删除下的范围误判率趋势评估 65
4.5.5 不同关联度下的误判率 66
4.5.6 结果分析与讨论 67
4.6 本章小结 68

第五章 基于自适应动态范围过滤器的多租户弹性缓存配额调度 . 69
5.1 问题背景与分析 69
5.2 性能收益与块密度协同的多租户数据价值建模 73
5.2.1 请求性能收益建模 73
5.2.2 块密度价值修正 74
5.2.3 价值趋势自适应估计 75
5.3 基于自适应动态范围过滤器的智能化缓存空间分配策略 75
5.3.1 价值趋势索引存储 75
5.3.2 范围过滤器密度估计 77
5.3.3 多租户配额优化模型 78
5.3.4 在线配额调度流程 79
5.4 系统设计与实现 80
5.4.1 系统架构 80
5.4.2 组件实现 82

5.4.3 系统运行示例 86
5.4.4 系统的不足 88
5.5 实验与性能评估 89
5.5.1 实验环境与配置 89
5.5.2 不同负载下的查询加速比对比 91
5.5.3 性能-成本偏好权衡评估 93
5.5.4 采样策略对比评估 96
5.5.5 参数敏感性分析 97
5.5.6 Alluxio 集成有效性评估 99
5.5.7 结果分析与讨论 101
5.6 本章小结 101

第六章 总结与展望 103
6.1 本文工作总结 103
6.2 下一步工作 104

参考文献 107

致 谢 115

附录 A 成果列表 117
A.1 研究生期间发表的论文 117
A.2 研究生期间申请的专利 117
A.3 研究生期间参与的科研项目 117
A.4 研究生期间获得的奖项 117

插图目录

1-1 存储层级的访问延迟与单位成本对比[1-2] 1
1-2 本文工作整体框架 10

2-1 布谷鸟过滤器结构示意图[6] 14
2-2 范围过滤器的设计空间 15
2-3 查询重复性与倾斜性可视化 19

3-1 不同查询负载下的误判率对比 24
3-2 自适应范围过滤器的设计思路 27
3-3 自适应范围过滤器的组成与查询流程 27
3-4 查询 x 时 ASCF 中自适应性的示例 28
3-5 紧凑比特位图与混合后缀块 28
3-6 插入 y 时两类碰撞的示例 31
3-7 对 x 进行半排序自适应的示例 34
3-8 对抗性工作负载下的误判率对比 42
3-9 各类工作负载下的误判率对比 43
3-10 真实世界数据集上的误判率对比 44
3-11 构建时间对比 46
3-12 查询时间对比 47
3-13 端到端查询延迟对比 48
3-14 不同偏斜查询分布下的误判率对比 48
3-15 重复性与偏斜性动态变化下的误判率对比 49

4-1 暴力插入流程示例（左）以及暴力操作定量性能对比（右） 51
4-2 后缀插入示例 53
4-3 后缀删除示例 56

4-4 不同方法在相关动态工作负载下的插入与删除延迟对比 63
4-5 不同方法在多轮插入过程中的范围误判率变化 64
4-6 不同方法在多轮删除过程中的范围误判率变化 66
4-7 关联度从 0 到 1 变化时的误判率对比 67

5-1 价值趋势布谷鸟索引 76
5-2 在线弹性缓存配额调度流程 80
5-3 基于 Alluxio 的多租户弹性缓存系统架构示意图 81
5-4 请求价值观测与块密度估计模块运行示意图 82
5-5 价值趋势布谷鸟索引模块运行示意图 83
5-6 块采样与性能-成本偏好决策模块运行示意图 84
5-7 价值感知驱逐执行模块运行示意图 85
5-8 不同 I/O 占比下 VTCI 与各近似索引方法的查询平均加速比对比 . . 92
5-9 不同块密度分布下 VTCI 与 VTCI− 及各近似索引方法的对比 . . . 93
5-10 不同用户偏好参数 γ 下的请求加速比与缓存成本（均值与标准差） 94
5-11 不同采样算法的效果对比 96
5-12 Holt-Winters 参数与采样比例的敏感性分析 98
5-13 集成 Alluxio 原型上的整体效果对比 99
5-14 集成原型中缓存决策的时序变化 100

表格目录

2-1 本文工作与其他范围过滤器的比较 16

3-1 第三章常用符号说明 26
3-2 构建时间的函数级剖析 44
3-3 查询时间的函数级剖析 45
3-4 构建时间的指令级剖析 45
3-5 查询时间的指令级剖析 46

4-1 第四章常用符号说明 52
4-2 实验硬件与软件环境 62

5-1 两类典型请求的各阶段耗时比例与缓存收益示例 70
5-2 最大化性能策略的缓存分配结果 71
5-3 性能成本权衡的缓存分配结果 72
5-4 单租户性能-成本权衡曲线 72
5-5 第五章主要符号说明 73
5-6 三类范围查询请求的定义 90
5-7 第 4.5 节实验核心配置 91

第一章　绪论

1.1　研究背景

在数字化转型持续深化的当下，大数据分析的应用场景不断拓展，大模型服务的需求日益旺盛、应用范围快速普及，直接驱动数据规模实现爆发式增长，数据访问的模式也随之呈现出多元化、复杂化的发展态势。IDC 研究显示，全球数据生成量在 2025 年达到 213,557 EB，并于 2029 年增长至 527,469 EB[1]；同时，全球人工智能与自动化解决方案支出预计将在 2027 年超过 5,000 亿美元[2]。

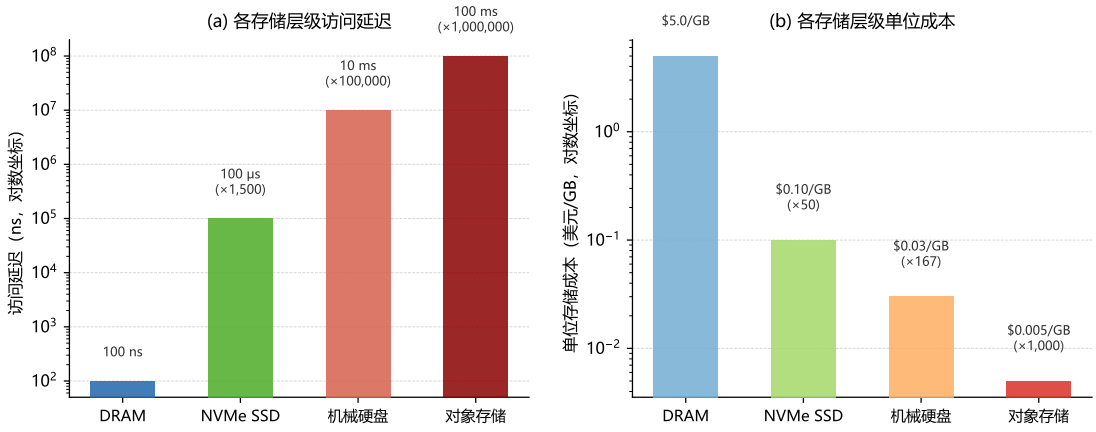


图 1-1　存储层级的访问延迟与单位成本对比[1-2]

然而，海量数据无法直接驻留于高速内存。如图 1-1 所示，现代存储层级在访问延迟与单位成本之间存在巨大鸿沟：DRAM 的访问延迟约为 100 ns，而 NVMe SSD、机械硬盘与对象存储的延迟依次增至约 100 µs、10 ms 和 100 ms，相差高达三至六个数量级[3]；与此同时，DRAM 的单位存储成本比 SSD 高出约 50 倍，比机械硬盘高出逾百倍[4]。这种不对称性决定了绝大多数数据只能存放于慢速的磁盘或对象存储层，系统在处理查询时必须频繁发起代价高昂的 I/O 操作。与此同时，研究表明 DRAM 带宽的提升速度（约每两年 1.6 倍）远低于计算能力的增长速度（约每两年 3.0 倍）[5]，使得内存带宽成为现代数据系统的核心瓶颈。换言之，海量数据带来的核心矛盾已从“能否存下数据”转向“能否以可接

受的时延和成本高效访问数据”。

在这种“数据增长快于有效承载能力提升”的格局下，现有数据系统在查询时延、吞吐能力和资源利用效率等方面面临日益突出的瓶颈。具体到执行路径层面，越来越多的数据访问请求呈现出区间扫描、批量过滤、块级读写与热点迁移并存的特征。对于分析型数据库、键值存储、日志检索和分层存储这类系统，往往需要先访问整个数据块并完成部分计算，才能判定其中是否包含查询所需数据，从而带来大量无效 I/O 与冗余计算开销；而数据热点分布不均、工作负载倾斜以及访问相关性波动又会进一步放大局部访问压力，导致尾延迟升高、吞吐下降以及资源浪费加剧。为了在有限内存开销下减少无效访问，近似数据结构（Approximate Data Structures）已经成为现代存储系统的重要优化手段[6-8]。

然而，传统近似数据结构主要围绕点查询进行设计，难以直接适配区间查询、前缀匹配和数据密度估计等需求。范围过滤器（Range Filter）作为一类面向区间查询场景的近似数据结构，能够以较低空间开销对键空间分布进行紧凑刻画，并在查询执行前快速筛除不可能命中的数据块，从而减少无效 I/O、缩小扫描范围并降低冗余计算开销；进一步地，范围过滤器提供的近似范围计数能力还可用于估计数据块内有效记录比例，为系统级数据价值评估提供支持[9-11]。因此，范围过滤器不仅是区间查询剪枝的重要基础结构，也是连接数据访问优化与系统级资源调度的潜在桥梁。

尽管现有范围过滤器在静态场景下已经取得较大进展，但在复杂实际负载下仍面临两类关键挑战：其一，传统范围过滤器大多围绕静态键分布和均匀查询模式构建，当面对数据倾斜、查询相关性变化以及对抗性范围查询时，容易产生重复误判，导致过滤精度与端到端查询性能持续退化；其二，为追求高空间效率，许多方法采用紧凑编码或高度压缩的前缀-后缀组织方式，使得插入、删除和结构重组代价较高，难以适应频繁写入、删除和键集波动并存的动态数据场景[11-12]。因此，如果不能同时提升范围过滤器对复杂查询负载的自适应能力以及对动态数据场景的在线维护能力，其在真实系统中的收益难以体现。

在上述基础上，缓存系统为缓解存储层访问延迟提供了另一条重要路径。随着数据规模扩大和多租户工作负载并发增强，固定规模缓存已难以匹配持续变化的访问需求，弹性缓存（Elastic Caching）因能够按负载动态调整缓存配额而成为重要的系统优化方向[13-14]。然而，现有缓存调度方法大多以缓存命中率或

工作集大小作为主要优化依据，难以准确反映请求的端到端性能收益，也缺乏对数据块密度与多租户性能-成本偏好的细粒度建模[15-19]。在这一场景下，范围过滤器能够将近似范围计数、工作集观测以及块价值建模联系起来，为弹性缓存中的配额调度和收益评估提供更细粒度的数据支撑。因此，将弹性缓存作为上述两类范围过滤器优化方法的系统级验证场景，既具有工程合理性，也有助于验证过滤器结构创新为真实数据系统带来的性能与成本收益。

基于上述背景，本文围绕面向复杂负载的范围过滤器优化及其弹性缓存应用展开系统研究。首先，针对倾斜与对抗性查询场景，研究范围过滤器的自适应优化机制，以抑制重复误判并提升复杂负载下的过滤精度；其次，针对动态数据场景，研究范围过滤器的高效在线增删与空间动态分配方法，以兼顾更新效率、空间利用率与过滤性能；最后，面向多租户弹性缓存场景，探索基于自适应动态范围过滤器的数据价值建模与缓存配额调度方法，验证前述结构优化在实际系统中的落地价值。

1.2 现有相关工作及不足

为了提升范围查询的过滤性能、增强过滤器对动态负载的适应能力并实现弹性缓存的高效管理，相关研究工作主要集中在范围过滤器[9-12,20]、自适应及全局过滤器[21-23] 与弹性缓存管理[24-29] 三个方面。本节将简要介绍其中的关键方法并分析其不足。

1.2.1 范围过滤器

范围过滤器（Range Filter）是一类用于判断给定查询范围内是否存在键的近似数据结构，其目标是以有限的空间开销实现尽可能低的误判率（False Positive Rate，FPR），从而减少底层存储系统（如日志结构化合并树[30]）中不必要的磁盘访问。该类方法通常对键集进行编码压缩，在查询时通过对编码结构的高效检索来判断范围内是否存在键。根据底层编码方式的不同，现有范围过滤器主要分为基于字典树、基于位图、基于哈希与混合四类方法。

（1）基于字典树的范围过滤器。基于字典树（Trie）的范围过滤器通过压缩键的前缀信息构建精简字典树结构，利用前缀匹配实现范围查询的剪枝，从而避免

不必要的磁盘访问。SuRF[9]提出了一种快速精简字典树（Fast Succinct Trie，FST）以编码键的前缀信息，支持范围查询的快速过滤。该方法首先对键集进行字典树构建，然后采用层序一元度序（Level-Order Unary Degree Sequence, LOUDS）-Dense 与 LOUDS-Sparse 两种互补的编码方式对字典树进行紧凑压缩，分别以内存效率和空间效率为优化目标，最后在查询阶段沿编码字典树进行前缀匹配以判断范围内是否存在键。然而，受限于 FST 编码结构的固有特性，SuRF 在空间利用率方面存在明显不足，相同内存预算下难以实现较低的误判率。

（2）基于位图的范围过滤器。基于位图的范围过滤器利用学习型分段线性模型将键映射至压缩位图，通过位操作支持范围查询的高效过滤，典型代表包括 SNARF[11]、Oasis[31] 等。SNARF[11]通过顺序选取固定数量的键构建分段线性模型，并将键映射至对应的位图区间以支持范围过滤；然而，当键分布不规律时，线性模型的预测误差较大，导致位图中存在大量空白区间，空间利用率显著下降，过滤效果随之恶化。Oasis[31]在此基础上引入了剪枝机制以提升空间效率，但对于高关联度查询——即查询端点与键集距离较近的场景——其过滤效果十分有限，几乎无法有效拒绝误判查询。Grafite[32]中的 Bucketing 组件将键空间划分为 n 个等大小的桶，每个桶用单个比特位表示，构建与查询速度均具明显优势，但当查询范围集中于误判桶区间时，过滤性能会显著退化。

（3）基于哈希的范围过滤器。基于哈希的范围过滤器通过将键或其前缀映射至哈希过滤器结构，以紧凑方式存储键的近似信息，从而支持范围成员查询，通常基于布隆过滤器[7]或布谷鸟过滤器[6]进行扩展。Rosetta[10]采用多级布隆过滤器存储不同长度的键前缀，通过逐级检索判断查询范围内是否存在键；然而，多级结构的维护需要显著的空间与时间开销，整体效率较低。BloomRF[33]利用分段单调哈希函数替代传统均匀哈希，以减少内存访问次数并加速查询；然而，单调性约束限制了哈希函数的灵活性，导致空间效率存在明显劣势。REncoder[34]通过集成范围布隆过滤器减少内存访问次数，但受限于其编码结构，误判率上界较高，难以满足高精度过滤的需求。Grafite[32]针对近似范围空集问题[35]，利用成对独立哈希函数与 Elias-Fano 编码构建紧凑的过滤器结构，在强关联查询分布下取得了良好性能；然而，在低关联度查询场景下，Grafite 的过滤性能相较于面向非相关负载优化的方法存在明显差距。Memento[12]是一种专为 B 树设计的动态范围过滤器，支持键集的动态插入与删除操作，继承了 Grafite 的核心编码机制

并针对 B 树场景进行了适配；然而，由于底层结构与 Grafite 高度一致，Memento 同样继承了 Grafite 在非相关负载场景下的性能局限，且在键集频繁更新时，前缀长度与后缀长度的空间配比问题缺乏系统性的优化指导。

（4）基于混合方法的范围过滤器。混合方法将字典树与哈希过滤器的优势相结合，设计联合配置模型，以在不同场景下取得更优的综合性能。Proteus[20]将基于字典树的 SuRF 与基于哈希的 Rosetta 有机结合，并引入条件前缀误判率预测（Conditional Prefix False Positive Rate，CPFPR）模型以指导过滤器的配置选择；然而，由于 Proteus 依赖两种整体性能平庸的基础过滤器，其综合性能仅能达到平均水平，在强竞争场景下缺乏竞争优势。

综上所述，现有范围过滤器在空间效率与过滤性能之间的权衡上取得了不同程度的进展。然而，这些方法存在两类系统性不足。其一，从查询负载适应性的角度看，上述方法均为静态结构，无法针对重复出现的误判查询动态调整指纹编码，在倾斜与对抗性查询负载下会产生持续性的误判率退化，难以满足第三章所研究的自适应范围过滤器设计需求。其二，从键集动态性的角度看，尽管有研究尝试支持键的动态插入与删除，但现有方法在键集频繁更新场景下，后缀结构的合并效率与前缀-后缀空间的动态分配问题尚未得到系统性解决，这正是第四章所着力研究的核心问题。

1.2.2 自适应及全局过滤器

自适应过滤器（Adaptive Filter）针对倾斜与对抗性查询场景，通过在线或离线机制动态调整过滤器结构，以应对重复误判问题。该类方法的核心思路是利用查询历史或实时反馈，在不增加结构空间的前提下改变指纹或桶的分配策略，从而抑制已知高频误判查询的重复触发。当前自适应过滤器研究主要分为基于先验知识的离线自适应方法与基于在线反馈的动态自适应方法两个分支。

（1）离线自适应方法。该类方法[36-40] 利用查询工作负载的先验知识，在构建阶段对数据结构进行最优化配置，或在系统停机时离线调整过滤器参数，以减少已知高频查询的误判次数。然而，在实际系统中预先获取查询工作负载的精确先验知识往往不切实际，严重限制了此类方法的实用性。

（2）在线自适应方法。该类方法[21-23,41-42] 在查询过程中实时监测误判情况，并动态调整过滤器的内部结构以抑制重复误判，无需先验知识，能够在线应对

工作负载的变化。然而，现有在线自适应方法均针对点查询（Point Query）场景设计，其自适应机制无法直接迁移至范围查询过滤器，因而无法解决范围过滤器在倾斜与对抗性负载下的重复误判问题。

此外，**全局范围过滤器**也是相关研究方向之一。Remix[43]维护数据的全局有序视图以加速正性范围查询，GRF[44]在整棵 LSM-Tree 上构建全局过滤器以减少过滤器探测次数、缓解 CPU 瓶颈。上述方法优化的是全局正性查询路径，与本文针对本地误判率优化的目标正交，并不冲突。

综上所述，自适应过滤器研究为解决重复误判问题提供了重要思路，但现有方法要么依赖不可获取的先验知识，要么仅针对点查询设计，缺乏对范围查询场景下自适应机制的系统性研究，难以有效应对范围过滤器在对抗性负载下的持续性能退化问题，这正是第三章所着力解决的核心挑战。

1.2.3 弹性缓存管理

弹性缓存管理面向云存储与分布式系统中缓存容量的动态调配问题，旨在通过对工作负载特征的在线感知，自适应地分配存储资源，在资源消耗与系统性能之间取得最优平衡。该类方法的核心思路是构建工作负载特征模型，在线估计各租户或服务的资源需求，并据此动态调整缓存与计算资源的分配策略。现有工作主要围绕弹性缓存配额估计、弹性缓存资源调度、弹性缓存性价比优化三个维度展开。

（1）**弹性缓存配额估计**。此类方法通过构建未命中率曲线（Miss Ratio Curve，MRC），刻画缓存大小与缓存未命中率之间的函数关系，为缓存容量分配提供量化依据。eMRC[24]提出面向多层级缓存架构的高效多维 MRC 近似方法，通过凸包近似与缓存分区算法消除 MRC 中的性能断崖现象，并设计 ORCA 多层缓存编排框架，满足具有不同服务水平目标的租户缓存需求；然而，该方法以缓存命中率作为数据价值评估的核心指标，未考虑请求内部计算与 I/O 环节的比例关系，在计算密集型任务中盲目追求命中率难以实现显著的端到端加速效果。Cuki[25]提出基于布谷鸟过滤器的轻量级近似数据结构，用于在线估计滑动窗口内的工作集大小（Working Set Size，WSS）与访问重复率（Item Repetition Ratio，IRR），并据此构建自适应缓存容量调优机制，在 Presto 与 Alluxio 分布式缓存系统中取得显著性能提升；然而，Cuki 同样以命中率为核心优化目标，其工作集

统计以数据项为基本单位，忽略了实际生产环境中以页/块为粒度进行缓存存取时数据块内有效查询数据的密度差异，导致缓存价值评估存在系统性偏差。

（2）**弹性缓存资源调度**。此类方法针对函数即服务（Function as a Service，FaaS）平台，利用弹性资源调配与机器学习模型降低存储访问延迟，减少资源空闲浪费。Pocket[26]是面向无服务器分析场景的分布式弹性临时存储系统，通过控制面、元数据面与数据面的职责分离，实现跨 DRAM、NVM Flash、HDD 存储层级的自动化资源弹性伸缩，以低成本满足任意规模数据集的高性能 I/O 需求；然而，Pocket 面向无服务器中间数据共享场景，其资源弹性策略以最大化存储性能为目标，不具备对多租户差异化成本-性能偏好的柔性建模能力。OFC[27]通过挖掘 FaaS 平台中函数沙箱因内存过度配置与保活策略产生的空闲内存，构建机会式分布式内存缓存系统，利用机器学习模型预测各函数调用的实际内存占用，将剩余容量用于透明加速数据读取与加载阶段，在单阶段与流水线函数中分别实现最高 82% 与 60% 的执行时间压缩；然而，OFC 的缓存调度策略以函数执行时间最优化为目标，缺乏对多租户场景下“以适度成本换取目标性能”等定制化性价比需求的支持。

（3）**弹性缓存性价比优化**。此类方法将服务水平目标（Service Level Objective，SLO）约束与资源成本显式纳入调度决策框架，探索缓存与计算层弹性伸缩中的性能-成本权衡问题。RAIS[28]是亚马逊 Redshift 无服务器版本中的新一代 AI 弹性伸缩系统，通过资源预测模型与实时扩容控制器协同工作，自动为各查询按需分配计算资源，并引入可配置的“价格-性能目标”滑块捕获用户的成本-性能权衡偏好；然而，RAIS 的弹性建模对象为计算节点的 CPU/内存规模，调度粒度为单次查询的整体资源配额，未能深入数据缓存层对过滤器密集型负载进行精细化缓存空间管理。OptScaler[29]提出面向云端长期驻留应用的协同自动伸缩框架，通过以模型预测控制（Model Predictive Control，MPC）为纽带协同主动预测模块与被动反应模块，在 Alipay 支付平台的实际部署中实现 SLO 违约率降低 36% 以上；然而，OptScaler 关注计算节点的水平扩缩容问题，优化目标为 CPU 利用率与 SLO 满足率，与多租户缓存配额分配中的性价比建模问题存在本质差异。

综上所述，现有弹性缓存管理与云端资源调度方法在提升缓存性能与资源利用效率方面已取得重要进展。然而，这些方法存在三类系统性不足。其一，从请求性能收益建模的角度看，现有方法以缓存命中率为唯一优化目标，忽略请

求内部计算与 I/O 比例对端到端性能增益的影响，导致在计算密集型请求中盲目扩充缓存却收效甚微。其二，从缓存单元粒度建模的角度看，现有工作集统计以数据项为基准，未考虑以 Block/Page 为缓存单位时块内有效数据的密度差异，致使缓存价值评估存在系统性偏差。其三，从多租户差异化性价比建模的角度看，现有系统无法对” 以适度成本获取目标性能” 等差异化需求进行柔性建模，难以在多租户云原生环境中实现资源配额的帕累托最优分配，这正是第五章所着力解决的核心挑战。

1.3 研究难点分析

针对现有范围过滤器、自适应过滤器与弹性缓存管理方法的不足，本文围绕面向复杂负载的范围过滤器优化及其弹性缓存应用关键技术展开研究，旨在提升范围查询过滤的精度、动态更新能力以及缓存资源调度效率。实现上述目标面临如下研究难点。

（1）**倾斜查询负载下范围过滤器的适应性与鲁棒性难以兼顾**。范围查询负载往往同时具有关联性、倾斜性和重复性，静态范围过滤器在这类场景下容易出现误判集中放大，导致过滤效果显著退化。与此同时，现有自适应过滤器大多围绕点查询设计，其更新机制难以直接迁移到范围查询场景。因此，如何在保持范围查询低延迟和低空间开销的前提下，为范围过滤器引入面向复杂负载的自适应能力，并兼顾不同关联度场景下的过滤性能，是本文首先需要解决的关键问题。

（2）**动态数据场景下范围过滤器的在线更新能力与紧凑存储难以统一**。为了获得较高的空间效率，范围过滤器通常采用高度压缩的前缀-后缀组织方式，这类结构在静态场景中优势明显，但在插入、删除频繁发生时往往面临维护困难。一旦更新操作引发大规模结构重排，不仅会带来较高的时间开销，还可能破坏原有的结构一致性与过滤精度。因此，如何在不显著增加空间成本的前提下，支持范围过滤器的低开销在线增删与动态维护，使其同时适应查询负载变化和键集变化，是本文面临的第二个研究难点。

（3）**多租户弹性缓存场景下数据价值评估与资源调度难以精细协同**。范围过滤器应用于弹性缓存后，缓存决策不仅取决于访问频率，还受到请求类型、I/O 收益、块内有效数据密度以及租户性能-成本偏好等多种因素的共同影响。现有

方法多以命中率或工作集大小作为主要依据，难以准确反映缓存对端到端性能的真实贡献，也难以支持多租户环境中的差异化配额分配。因此，如何构建兼顾性能收益、块密度和动态趋势的数据价值模型，并在有限资源预算下实现面向多租户偏好的高效缓存调度，是本文需要进一步解决的重要挑战。

1.4 本文主要研究工作

本文围绕面向复杂负载的范围过滤器优化及其弹性缓存应用关键技术展开研究，旨在解决范围过滤器在复杂查询负载下适应性不足、在动态数据场景下在线更新困难，以及在多租户弹性缓存场景下价值评估与资源调度不够精细等问题。如图1-2 所示，本文前两个工作为不同场景下基础算法与数据结构的设计，第三个工作将前两个工作的数据结构集成到系统中，协同解决数据访问难题，其主要研究工作和贡献点包括：

（1）**针对倾斜查询负载下范围过滤器适应性与鲁棒性难以兼顾的问题，本文提出了面向倾斜负载的范围过滤器自适应指纹更新技术**。该技术设计了自适应范围过滤器——Hourglass[45]（本模块工作已在数据库会议 SIGMOD 2026 出版），通过基于半排序指纹的前缀组织方式和紧凑的后缀存储结构，在保持范围查询高效性的同时引入自适应能力。进一步地，本文提出了基于滑动窗口的半排序自适应更新机制，以抑制倾斜与对抗性查询场景下重复误判的持续累积；同时，提出了基于处理器架构的构建访存优化方法，以降低过滤器构建过程中的随机访存开销。通过上述研究，可以有效提升范围过滤器在复杂查询负载下的过滤精度与运行效率。

（2）**针对动态数据场景下范围过滤器在线更新能力与紧凑存储难以统一的问题，本文提出了面向更新负载的范围过滤器后缀位级化增删技术**。该技术围绕紧凑后缀布局在动态更新场景下面临的维护困难，提出了基于位级合并的后缀插入策略和基于二分补位的后缀删除策略，在不破坏既有压缩布局的前提下支持后缀信息的原位增删。进一步地，本文提出了关联度感知的过滤器空间动态化分配方法，以兼顾动态更新能力、空间利用率和过滤性能。通过上述研究，可以有效提升范围过滤器在频繁插入删除场景下的在线维护能力。

（3）**针对多租户弹性缓存场景下数据价值评估与资源调度难以精细协同的**

问题，本文提出了基于自适应动态范围过滤器的多租户弹性缓存配额调度方法。该方法首先构建了访问模式与性能增益协同的弹性缓存价值模型，以更准确地刻画缓存对端到端性能的真实贡献；接着，利用范围过滤器的近似范围计数能力对块密度进行修正，并在此基础上提出了基于范围过滤器的智能化缓存空间分配策略；进一步地，本文将上述方法集成到实际系统中，实现了面向多租户差异化性能-成本偏好的缓存配额调度。通过上述研究，可以有效提升多租户弹性缓存系统的资源利用效率和调度精细化水平。

基于上述研究，本文从数据结构优化到缓存调度应用构建了一条完整的技术路径，并通过多种实验对所提出的方法进行了系统评估。

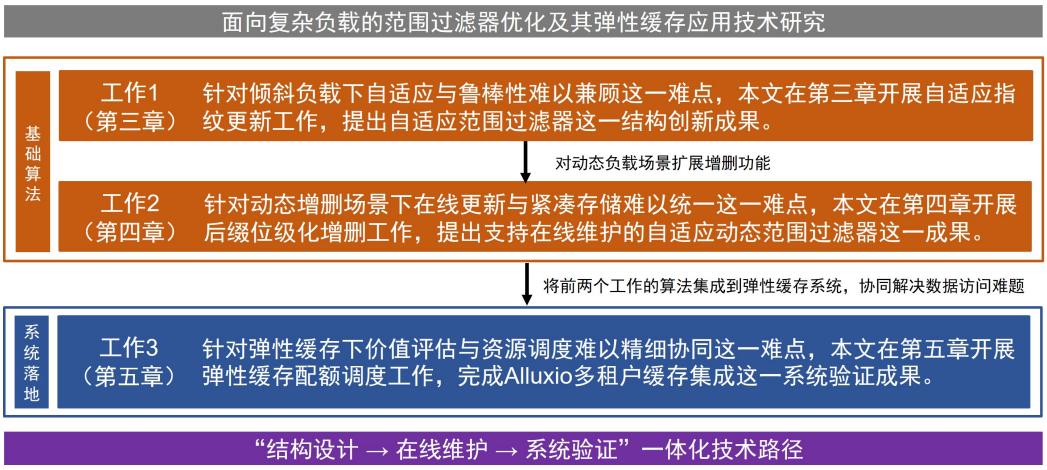


图 1-2 本文工作整体框架

1.5 本文的组织结构

本文由六个章节组成，各章节主要内容如下：

第一章，绪论。该章首先介绍本文的研究背景与研究意义；接着，对现有相关工作及其不足进行分析；然后，对面向复杂负载的范围过滤器优化及其弹性缓存应用所面临的研究难点加以概括；随后，介绍本文的主要研究工作；最后，说明本文的组织结构。

第二章，背景知识。该章介绍本文研究所涉及的基础知识，包括布谷鸟过滤器、半排序布谷鸟过滤器、常用范围过滤器、范围查询关联度、对抗性查询、Q/U 比率与查询重复性以及弹性缓存基本原理等内容，为后续章节的设计与分析提供基础。

第三章，面向倾斜负载的范围过滤器自适应指纹更新技术。该章首先介绍问题背景并分析现有方法在倾斜与对抗性查询场景下的局限性；接着，提出 Hourglass 自适应范围过滤器总体设计，以及基于滑动窗口的半排序自适应更新机制；然后，提出基于处理器架构的范围过滤器构建访存优化方法；最后，对该章提出的方法进行实验评估与分析。

第四章，面向更新负载的范围过滤器后缀位级化增删技术。该章首先介绍动态数据管理场景下范围过滤器在线更新所面临的问题；接着，提出基于位级合并的过滤器后缀插入策略和基于二分补位的过滤器后缀删除策略；然后，提出关联度感知的过滤器空间动态化分配方法；最后，对该章提出的方法进行实验评估与分析。

第五章，基于自适应动态范围过滤器的多租户弹性缓存配额调度。该章首先介绍多租户弹性缓存场景下的问题背景与分析；接着，提出访问模式与性能增益协同的弹性缓存价值模型，以及基于范围过滤器的智能化缓存空间分配策略；然后，介绍系统设计与实现；最后，对该章提出的方法进行实验评估与分析。

第六章，总结与展望。该章首先对全文研究工作进行系统总结，归纳本文在范围过滤器自适应更新、动态增删维护以及多租户弹性缓存调度方面的主要研究内容与实验结论；然后，结合当前方法的适用范围与系统实现情况，对后续值得深入研究的方向进行展望。

第二章　背景知识

本章介绍本文研究所涉及的基础知识，重点围绕范围过滤器的数据结构基础、复杂查询负载的刻画方式以及弹性缓存基本原理展开，为后续章节的方法设计与实验分析提供背景支撑。本章内容组织结构如下：第 2.1 节介绍范围过滤器基础，包括布谷鸟过滤器、半排序布谷鸟过滤器与现有范围过滤器；第 2.2 节介绍复杂查询负载特征，包括范围查询关联度、对抗性查询以及 Q/U 比率与查询重复性；第 2.3 节介绍弹性缓存基本原理，为后续的多租户缓存调度研究提供应用背景。

2.1　范围过滤器基本原理

本节介绍范围过滤器所依赖的核心数据结构基础。首先介绍布谷鸟过滤器的基本原理与操作语义；在此基础上，进一步介绍半排序布谷鸟过滤器的改进方案，作为本文自适应范围过滤器的底层存储结构；最后综述现有典型范围过滤器的设计思路与性能特点，为后续的方法分析与比较提供参照。

2.1.1　布谷鸟过滤器

布谷鸟过滤器（Cuckoo Filter）[6]是一类基于指纹存储的近似成员查询结构，兼顾较低误判率、较高空间利用率和较好的删除支持能力。它由 n 个桶组成，每个桶最多存储 b 个元素的指纹。对于任意元素 x，系统首先计算其指纹，再由哈希函数确定两个候选桶位置：第一个候选桶由元素哈希值直接映射得到，第二个候选桶则由第一个桶位置与指纹哈希值异或计算得到。

当需要插入元素时，若两个候选桶中至少有一个存在空槽，则直接将该元素指纹写入对应槽位；否则，系统执行踢出（Kickout）操作，随机替换某个候选桶中的一个已有指纹，并将被替换指纹重新插入到它的另一个候选桶中。查询时，只需检查两个候选桶内是否存在与目标元素一致的指纹；若两处均不存在，则

可判定该元素必不在集合中。由于布谷鸟过滤器仅存储指纹而非完整键值，因此能够以较低空间开销提供高效查询能力。

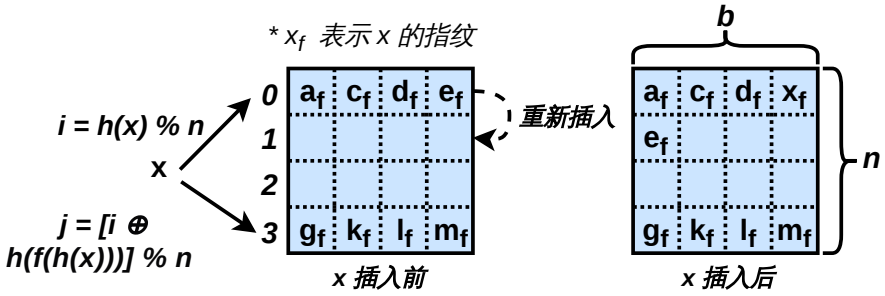


图 2-1 布谷鸟过滤器结构示意图[6]

2.1.2 半排序布谷鸟过滤器

半排序布谷鸟过滤器（Semi-sorted Cuckoo Filter）[6]是在标准布谷鸟过滤器基础上的紧凑化改进，其核心思想是在不改变查询正确性的前提下，对桶内指纹进行半排序编码，从而进一步节省存储空间。以桶大小 $b = 4$、指纹长度 $f = 4$ 的标准布谷鸟过滤器为例，一个桶原本需要 16 位来存储 4 个指纹；若对桶内 4 个指纹按顺序排列，则不同排列状态的数量显著减少，仅需 12 位即可唯一编码整个桶。

对于长度超过 4 位的指纹，半排序优化通常只作用于其最低有效 4 位，其余高位仍按标准方式保存。这样既能够保留压缩收益，又不会破坏过滤器在查询和更新过程中的可操作性。需要指出的是，半排序改变的是桶内指纹的编码与排列方式，而不是布谷鸟过滤器的基本查询逻辑。由于查询时只关心目标指纹是否存在于两个候选桶中，而不依赖桶内指纹的具体顺序，因此该优化不会影响过滤器的正确性。

2.1.3 常用范围过滤器

本节首先介绍具有代表性的现有范围过滤器；随后概括这些方法在复杂查询负载下的主要局限。为便于从整体上理解该方向的设计空间，图 2-2 对现有范围过滤器进行了分类，表 2-1 则从误判鲁棒性、自适应能力、查询复杂度与空间复杂度等维度，对代表性方法以及本文提出的自适应范围过滤器进行对比，以

为后续章节的设计分析提供统一参照。

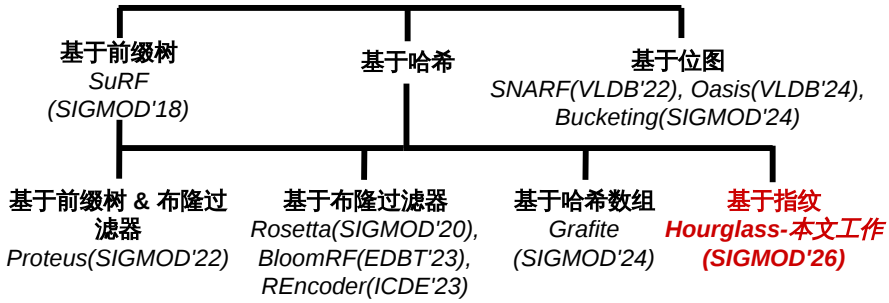


图 2-2 范围过滤器的设计空间

SuRF. SuRF[9] 采用快速精简字典树（Fast Succinct Trie, FST）存储键，并通过混合编码策略在高层使用 LOUDS-Dense、在低层使用 LOUDS-Sparse，以兼顾查询效率与空间效率。为进一步压缩空间，SuRF 仅保留能够区分所有键的最短前缀长度 k，并裁剪更深层的后缀信息。围绕误判率与空间开销之间的权衡，SuRF 设计了三种变体：SuRF-Base 保留完整字典树而不裁剪后缀；SuRF-Hash 使用紧凑哈希比特替代被裁剪的后缀；SuRF-Real 则将前缀长度从 k 扩展到可调参数 $k+\Delta$。对于范围查询 $[l,r]$，SuRF 通过定位大于等于 l 的最小键，并判断其是否小于等于 r 来完成过滤。

Rosetta. Rosetta[10] 采用分层布隆过滤器结构，共设置 $d=\lceil \log u \rceil$ 层，其中 u 表示键空间大小，第 i 层存储长度为 i 的键前缀。Rosetta 进一步通过可调的层级深度与各层布隆过滤器尺寸配置提升空间利用率。对于范围查询，Rosetta 自顶向下逐层处理：在第 i 层，它将查询区间划分为若干共享长度为 i 公共前缀的子区间，并在对应布隆过滤器中逐一探测这些前缀。若某个前缀返回负例，则相应子区间可直接剪枝；若返回正例，则继续向下一层递归验证，直到所有子区间均被剪枝或到达最底层。

Proteus. Proteus[20] 可以看作是 Rosetta 的两层变体，并在结构上做了两点关键改进。其一，在两层结构中，Proteus 保留第二层的布隆过滤器，同时使用来自 SuRF 的 FST 结构替代第一层。其二，Proteus 引入条件前缀误判率预测（Conditional Prefix False Positive Rate, CPFPR）模型，以联合调优 FST 与布隆过滤器的配置参数。对于范围查询，Proteus 采用两阶段验证流程：首先查询 FST，只有当 FST 返回真时，才进一步访问布隆过滤器完成最终确认。

BloomRF. 不同于 Rosetta 使用 d 个彼此独立的布隆过滤器分别存储不同长

度前缀，BloomRF[33] 将这一分层结构压缩为单个布隆过滤器。其核心做法是构造 d 个哈希函数 $\{h_i\}$，并利用这些哈希函数隐式编码不同长度的前缀信息，其中 $h_i(x)$ 融合了键 x 的长度为 i 的前缀。为提升缓存局部性，BloomRF 进一步采用分段单调哈希函数，使相邻前缀尽量映射到连续的位位置上。对于范围查询，BloomRF 仍然沿用基于前缀验证的思路，但将 Rosetta 的纵向逐层探测改为沿不同哈希函数的横向迭代。

表 2-1 本文工作与其他范围过滤器的比较

过滤器	非关联	关联	自适应	数据结构	查询复杂度	空间复杂度
SuRF	-	- -	✗	字典树	$O(\log u)$	$(10+m)n+10z+o(n)$
Rosetta	-	-	✗	布隆过滤器	$O(L\log\frac{1}{\epsilon})$	$1.44n\log\frac{L}{\epsilon}$
Proteus	-	-	✗	字典树 + 布隆过滤器	$O(L)$	$10n+10z+1.44\log\frac{1}{\epsilon}$
BloomRF	-	- -	✗	布隆过滤器	$O(\log\frac{u}{n})$	$1.44n\log\frac{L}{\epsilon}$
REncoder	-	- -	✗	布隆过滤器 + 位图	$O(\frac{kL}{i})$	$n(k+\log\frac{i}{\epsilon})$
SNARF	++	- -	✗	位图	$O(\log n)$	$n(\log\frac{1}{\epsilon}+2.4)$
Oasis	++	- -	✗	位图	$O(\log n)$	$n(\log\frac{i}{\epsilon}+2.4)$
Grafite	-	++	✗	有序哈希向量	$O(\log\frac{L}{\epsilon})$	$n(\log\frac{L}{\epsilon}+2)+o(n)$
Bucketing	++	- -	✗	位图	$O(\log\frac{u}{ts})$	$t\log\frac{u}{ts}+2t+o(t)$
Hourglass（本文工作）	+	++	✓	**半排序布谷鸟过滤器 + CBB* + HSB* + 布谷鸟哈希表（磁盘）†**	$\mathbf{O(\max(\log L,\frac{L}{i}))}$	$\mathbf{n(\log\frac{L}{\epsilon}+4)}$

表中 “++” 表示过滤效果最好，“+” 表示过滤效果较好，“-” 表示过滤效果较差，“- -” 表示几乎不具备过滤能力。其中，u 表示键空间大小，n 表示键集合规模，L 表示最大范围大小，ϵ 表示期望误判率，z 表示字典树内部节点数，k 表示哈希函数个数，i 表示 SIMD 寄存器宽度，s 表示桶大小，t 表示位图中值为 1 的比特数。*CBB（Compact Bitwise Bitmap，混合后缀块）与 HSB（Hybrid Suffix Block，紧凑比特位图）是本文工作引入的新结构；† 磁盘上的布谷鸟哈希表是实现自适应机制所需的辅助结构，这一点与文献[21]中的设定一致。

REncoder. 与 BloomRF 类似，REncoder[34] 同样尝试将 Rosetta 的层级结构压缩到单个布隆过滤器中，并利用空间局部性减少内存访问。具体而言，它将键空间组织成一棵深度为 $d=\lceil\log u\rceil$ 的二叉树，其中 u 为键空间大小，每个节点对应一个比特位。随后，该二叉树被划分为前缀部分（深度 1 到 k）和后缀部分（深度 $k+1$ 到 d）。从根节点到深度为 k 的某个节点 n_k 的路径定义了一个前缀 p，而以 n_k 为根的子树则通过 BST 编码为一个 2^{d-k+1} 位的位图。为了提升空间效率，REncoder 在实际实现中会对这些子树进行截断。插入时，前缀 p 通过 k 个哈希函数映射到若干位置，再将位图按位或到相应子数组中；查询时，范围先被拆分为若干共享长度为 k 前缀的子区间，再通过哈希定位、位图读取和按位与重建子树，以判断该子树是否与查询区间相交。

SNARF. 对于键集 $\{k_1,k_2,\cdots,k_n\}$，SNARF[11] 首先利用经验累积分布函数的单调估计（Monotonic CDF, MCDF）将所有键单调映射到区间 $[0,1]$。随后，分配长度为 $K\times n$ 的位数组来存储键，其中 K 为可调参数，每个键 k_i 被映射到位置 $\lfloor \mathrm{MCDF}(k_i)\times K\times n\rfloor$。为压缩空间，SNARF 使用 Elias-Fano 对位数组进行

编码。借助 MCDF 的单调性，范围查询 $[l, r]$ 可以映射为位数组上的连续子区间 $\left[\lfloor \text{MCDF}(l) \times K \times n \rfloor, \lfloor \text{MCDF}(r) \times K \times n \rfloor\right]$，只要区间内存在置位比特，即返回真。

Oasis. 在 SNARF 的设计空间中，当数据分布不均衡时，位数组中往往会出现长串连续的零，造成空间浪费。Oasis[31] 通过显式排除大空区间，将键空间划分为互不相交的若干区间，以此缓解这一问题。进一步地，基于 “在稀疏区间上，前缀编码通常比学习型编码更具空间效率” 这一观察，Oasis+ 采用混合设计：对稠密区间使用 Oasis 编码，对稀疏区间使用 Proteus 编码。查询时，Oasis+ 首先将范围拆解为若干连续区间，再根据不同区间的编码类型调用 Oasis 或 Proteus 的查询过程。

Grafite. Grafite[32] 受到近似范围空集问题中保局哈希思想的启发，通过专用哈希函数将键映射到值域 $[0, r]$。对于满足 $b - a < L$ 的范围 $[a, b]$，其中 $r = \frac{nL}{\epsilon}$，映射结果满足两种模式之一：要么形成连续区间 $[h(a), h(b)]$，要么形成环绕区间 $[h(a), r] \cup [0, h(b)]$。因此，只要存在键 $k \in [a, b]$，其哈希值必落入上述区间之一。为了实现紧凑存储与快速查询，Grafite 采用两阶段流程：先对哈希向量进行排序，再用 Elias-Fano 进行压缩。由于 Elias-Fano 本身支持高效检索，Grafite 因而在相关负载场景下兼具良好的空间效率与查询性能。

Bucketing. Bucketing[32] 主要面向非关联负载实现近似最优性能。它将整个键空间划分为 k 个大小相同的桶，并用位数组标记哪些桶非空，随后使用 Elias-Fano 对该位数组进行压缩。对于范围查询，Bucketing 只需扫描与查询区间相交的桶，检查是否存在被置位的桶即可完成过滤。

综合来看，现有范围过滤器虽然在不同设计方向上分别取得了进展，但仍存在两类共性局限。其一，大多数方法难以针对倾斜范围查询中的重复误判引入真正的自适应能力。这是因为，许多结构要么使用单个比特位同时表示多个键，要么依赖静态压缩布局，使得在不破坏原有编码的前提下难以对误判查询进行局部修正。其二，已有方法的误判性能通常偏向于某一类负载：多数结构为了节省空间，会裁剪键的低位信息，从而在相关查询场景下难以区分相邻键；而少数专门面向高关联负载优化的方法，又往往在非关联场景下无法保持同样理想的空间利用率与过滤效果。这些背景共同构成了后续章节设计自适应动态范围过滤器的出发点。

2.2 复杂查询负载特征

本节介绍本文所研究场景中复杂查询负载的关键特征。首先介绍范围查询关联度的概念及其对过滤精度的影响；随后介绍对抗性查询的成因与危害；最后介绍 Q/U 比率与查询重复性，用于刻画负载的重复访问强度。上述三类特征共同构成本文第 三 章与第 四 章方法设计的负载假设基础。

2.2.1 范围查询关联度

关联度 D 由文献[32]提出，用于刻画范围查询与其最近键之间的接近程度，是后续分析范围过滤器误判行为的重要指标。设查询与最近键之间的距离为 S，则二者满足关系

$$S = 2^{30(1-D)}.$$

其中，$D \in [0,1]$；D 越大，表示查询区间越靠近已有键，查询与键之间的相关性越强；D 越小，则表示查询与最近键之间距离更远。采用上述定义后，可以将不同负载下 “查询是否靠近真实键集” 这一现象统一到同一度量框架中。对于相邻键之间的范围查询，当均匀存储约 2 亿个键时，最大键-查询距离约为 2^{35}，因此上述定义中取 30 作为参数，以便在常见范围查询场景下获得稳定的整数刻画。

例如，对键集 $\{1,4,8\}$ 执行范围查询 $[5,6]$ 时，距离最近的键为 4，因此键-查询距离为 $S = 5-4 = 1$，对应的关联度为 $D = 1$。在实际实验中，关联度常被用来区分 correlated 与 uncorrelated 等不同查询场景，并进一步分析不同过滤器在强相关和弱相关负载下的过滤能力差异。

2.2.2 对抗性查询

文献[21]指出，对于单次查询误判概率为 ϵ 的近似成员查询（Approximate Membership Query, AMQ）结构，攻击者可以通过不断重复那些已经触发误判的查询，使整体有效误判率持续升高。**对抗性查询**（Adversarial Queries）指的正是这类被刻意重复执行、并会显著放大系统误判代价的查询序列。在范围过滤器场景中，一旦某个查询区间对应误判结果，攻击者或热点负载便可能持续访问这一查询区间，从而使静态过滤器性能迅速退化。

2.2.3 Q/U 比率与查询重复性

本文采用 Q/U 比率来衡量查询集合中的平均重复程度，其中 $|Q|$ 表示查询总数，$|U|$ 表示不重复查询数量。Q/U 越大，说明同一查询被反复访问的现象越明显，系统面临的重复误判风险也越高。需要注意的是，查询负载不仅具有重复性，还具有倾斜性：重复出现的查询未必均匀分布，而可能集中在少量热点区间上。

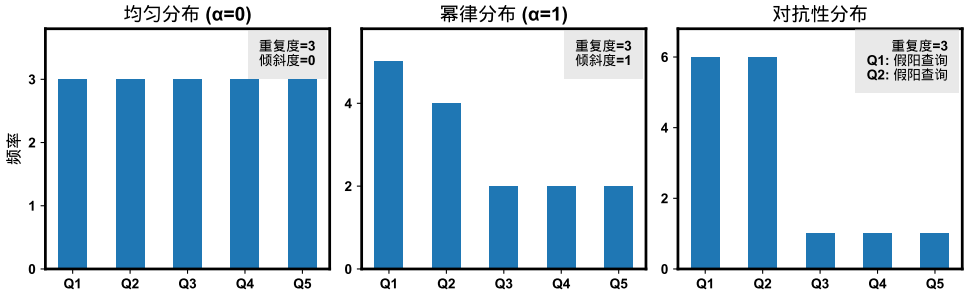


图 2-3 查询重复性与倾斜性可视化

如图 2-3所示，查询集通常由重复性和偏斜性共同决定。重复性描述同一查询被反复访问的程度，而偏斜性描述这些重复访问在不同查询之间的分布方式：在幂律分布中，参数 α 越大，查询越集中于少量热点；在对抗性分布中，少数误判查询会表现出极高重复性，而其他查询仅出现一次。因此，重复性和倾斜性共同决定了实际负载下过滤器误判代价的放大程度，也是后续章节设计自适应机制的重要动因。

2.3 弹性缓存基本原理

本节介绍弹性缓存的基本概念与相关背景。首先从数据系统的分层架构出发，阐明缓存层在现代数据系统中的定位与作用；随后介绍弹性缓存的核心概念与工作集大小估计的主要方法；最后介绍多租户弹性缓存问题及其特有的挑战，为第五章的研究工作提供应用背景。

2.3.1 数据系统分层架构

现代数据系统通常可以分为以下四层：应用与查询层、资源与编排层、数据与缓存层以及持久化存储层。其中，应用与查询层负责承载 SQL 查询、计算任

务与服务接口；资源与编排层负责统一管理计算实例及其伸缩；数据与缓存层负责在访问路径上提供热点数据加速；持久化存储层则提供对象存储、分布式文件系统或块存储等长期数据保存能力。在这一分层结构下，缓存层位于查询执行与底层存储之间，既影响上层访问时延，也影响系统整体的资源消耗。

工业界已经在多类现代数据系统中广泛部署缓存层。例如，Snowflake 在计算与存储解耦的数据仓库体系中利用多级缓存减少远端对象存储访问；StarRocks 在面向分析查询的执行路径中结合本地缓存与对象存储提升查询效率；Alluxio[46] 提供面向多种上层计算框架的独立分布式缓存与数据访问服务；ADB[47] 则在分析型数据仓库内部将缓存能力与查询执行、存储管理协同设计。这些工业实践表明，缓存层已经成为现代数据平台中的基础组成部分。

2.3.2 弹性缓存与工作集大小估计

弹性缓存是指当负载强度、实例规模与租户并发持续变化时，计算或估计当前状态下较优的资源分配值，并据此对缓存资源（如内存、SSD）进行在线动态调节的技术[13,26-27]。其核心在于使缓存容量能够随系统状态变化而自适应调整，而不是长期依赖固定配额或静态阈值。

工作集大小（Working Set Size, WSS）往往被用作弹性缓存策略的重要参考值，即在给定时间窗口内，为维持当前访问性能所需保留的活跃数据规模。若 WSS 被高估，系统会保留过多缓存资源并增加成本；若 WSS 被低估，则容易导致缓存抖动、频繁驱逐与性能退化。

WSS 估计技术大致可以概括为四类。第一类是基于阈值的方法：当系统缓存占用超过上阈值时触发扩容，低于下阈值时触发缩容，以此实现容量的在线调节，其特点是实现直接、控制开销低。第二类是基于访问统计直方图（Histogram）或近似重用统计的方法，通过维护访问频次或重用分布来估计活跃数据规模，但在强动态负载下精度容易受到影响[19]。第三类是基于未命中率曲线（Miss Ratio Curve, MRC）的方法，通过刻画“缓存大小-未命中率”关系，为容量决策提供更强的可解释性；相关工作进一步关注在线近似与低开销构建，如 eMRC 与后续改进方法[17,24]。第四类则是面向在线系统的大规模轻量估计，例如 Cuki 使用紧凑近似结构同时估计 WSS 与重复性，在生产系统中实现了较高的实用性[25]。

尽管上述方法显著推动了弹性缓存发展，但它们主要回答的仍是” 当前需要

多大缓存”这一问题；进一步如何面向多租户共享环境进行资源分配，仍需结合更复杂的目标与约束来考虑。

2.3.3 多租户弹性缓存问题

多租户共享是现代数据系统的另一个核心约束。在该场景下，缓存资源竞争、租户目标差异与负载异步波动会同时出现：同一时刻不同租户可能分别追求最低时延、最低成本或稳定吞吐，这使“全局命中率最大化”不再等价于“租户满意度最优”[18,24,48]。例如，分析型租户通常希望保留更大的热点数据区，而短作业或计算密集型租户更关注单位成本收益；若长期采用统一配额规则，容易出现强租户挤占缓存、弱租户抖动加剧、整体资源利用率下降等现象。

因此，多租户弹性缓存的关键难点不只是“估计多大缓存”，还包括“为谁分配缓存、按什么目标分配、如何在动态变化中保持公平与稳定”。这类问题通常需要同时处理两类约束：其一是性能约束，即不同租户的时延或吞吐目标；其二是成本约束，即按需计费下的预算边界。只有在多目标约束下进行协同建模，弹性缓存才能满足多租户场景中的异构化需求。

第三章　面向倾斜负载的范围过滤器自适应指纹更新技术

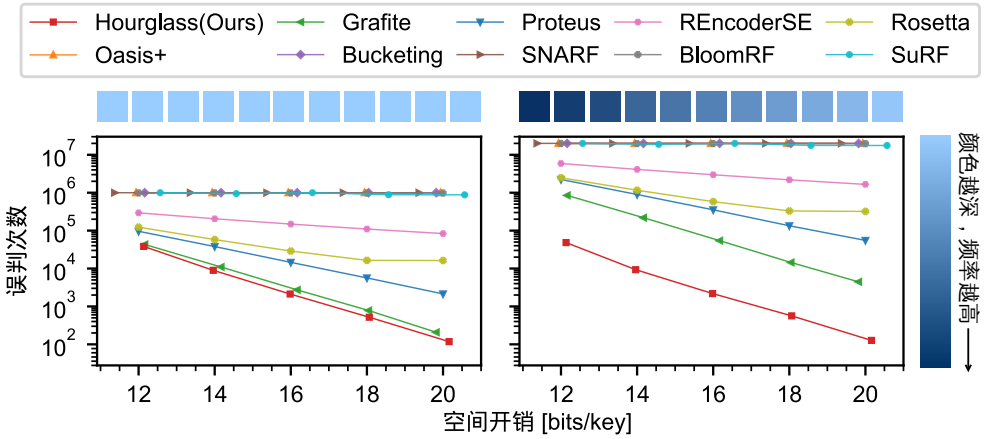
3.1　问题背景与分析

本章针对倾斜与对抗性查询场景下范围过滤器的重复误判问题展开研究，提出自适应范围过滤器——Hourglass，并从结构设计、自适应更新和构建优化三个方面提升其综合性能。本章内容组织结构如下：第 3.1 节介绍问题背景与现有方法的局限性，提炼核心子问题；第 3.2 节介绍 Hourglass 的总体设计框架与各组件的数据结构；第 3.3 节介绍基于滑动窗口的半排序自适应更新机制及其理论保证；第 3.4 节介绍基于处理器架构的范围过滤器构建访存优化；第 3.5 节对 Hourglass 与当前最先进的范围过滤器进行全面的实验与性能评估。

范围查询（Range Query）在数据分析与管理系统中具有举足轻重的地位，广泛应用于网络监控[49-51]、实时分析[52-54]、大数据处理[55-57]、时序数据库[58-59]、日志处理[60]、缓存系统[61-62] 以及物联网[63]等诸多领域。给定键集合 S，范围查询 $R[a,b]$ 返回 $S \cap [a,b]$ 中的所有元素。例如，管理员可能需要检索学号在 1762320 至 1762329 之间的学生信息；用户可能请求七月至八月期间的账单列表；导航应用可能需要特定地理区域内的交通状况以规划路线。尽管近年来已提出多种点查询过滤器[7,64-68] 以加速点查询，但它们对范围查询效果有限。为此，研究人员提出了多种范围过滤器[9-11,20,31-34] 以提升范围查询的性能。

现有范围过滤器可分为四类：基于字典树（Trie-based）的方法[9]、基于位图（Bitmap-based）的方法[11,31]、基于哈希（Hash-based）的方法[10,32-34] 以及混合方法（Hybrid-based）[20]。在基于字典树的范围过滤器中，SuRF[9] 利用快速精简字典树压缩键，但存在较高的误判率（False Positive Rate, FPR）。基于位图的范围过滤器，如 SNARF[11] 和 Oasis[31]，采用学习模型将键映射到压缩位图，但在高键-查询关联度（Correlation Degree）场景下表现不佳——关联度表示查询与距其最近端点的键之间的距离。在基于哈希的范围过滤器中，Rosetta[10]、BloomRF[33] 和 REncoder[34] 在布隆过滤器基础上进行了创新，但整体性能有限。Grafite[32] 首次

通过成对独立哈希函数解决高关联度问题，但在低关联度场景下仍有改进空间。在混合方法中，Proteus[20] 结合了 SuRF 与 Rosetta 的优势，但整体性能平均。



均匀（左）和幂律（右）查询分布下的误判情况，每个方格表示一条唯一查询

图 3-1 不同查询负载下的误判率对比

如图 3-1 所示，现有范围过滤器在倾斜查询分布下的误判数量比均匀分布下高出一个数量级以上。这是因为，现有范围过滤器通常假设查询均匀分布，然而现实场景中倾斜查询分布十分普遍[69-70]——热点数据的查询频率远超冷门数据。例如，城市中心区域的交通状况查询频率远高于郊区；在某些预约系统中，周末晚间的查询量也远多于工作日。当查询频率极高时，同一查询可能反复触发误判。考虑两条均匀分布的查询 Q_1（误判）和 Q_2（真负例），此时误判率为 50%；然而，在极度倾斜的查询分布中，若 Q_1 执行 99 次而 Q_2 仅执行一次，误判率则上升至 99%。在对抗性场景（Adversarial Scenario）中，攻击者可反复执行误判查询，导致系统延迟大幅增加。尽管已有自适应过滤器[21-22,41-42]被提出以应对倾斜和对抗性的点查询，但它们无法消除范围查询中重复误判的问题。

综上所述，现有范围过滤器在倾斜和对抗性工作负载下存在以下核心局限：首先，在倾斜查询分布下，高频误判查询的有效误判率显著高于静态误判率，与均匀分布相比可相差一个数量级以上；其次，对抗性攻击者可通过反复触发误判查询使系统性能急剧劣化；进一步地，现有自适应机制专为点查询设计，无法直接迁移至范围查询场景。然而，为现有范围过滤器引入自适应能力[1]（Adaptivity）并非易事，主要面临以下障碍：对于基于字典树的范围过滤器，截断后缀会丢失识别误判原因所需的关键信息；对于基于位图和布隆过滤器的范围过滤器，单

1 本文中的”自适应性”是指防御重复误判查询的能力，而非适应不断变化的关联度。

比特修正往往会影响其他正确存储的键，因为每个比特通常对应多个键；混合方法继承了各自组成部分的局限性；Grafite 对哈希码进行了精简压缩，没有留下实现自适应性的空间。

上述问题可以转化为一个兼具自适应能力与高效查询性能的范围过滤器设计问题，进一步可以划分为以下两个子问题：

子问题 1：如何设计高效的指纹式范围过滤器结构，使其在保持常数查询复杂度的同时，支持轻量级自适应更新以防御重复误判?

子问题 2：如何在保持过滤精度与查询效率的同时，降低 Hourglass 构建过程中的随机访存开销?

对于子问题 1，本章在第 3.2 节提出 Hourglass 的整体设计框架，基于自适应半排序布谷鸟过滤器存储前缀指纹，并通过混合后缀块紧凑存储后缀数组；在第 3.3 节详述基于滑动窗口的半排序自适应更新机制，通过更新指纹并保持半排序顺序来防止重复误判。对于子问题 2，本章在第 3.4 节提出基于处理器架构的访存优化方案，通过向量化空槽检测、访存预取、大页内存和两阶段构建等方法，降低过滤器构建过程中的随机访存代价并提升整体吞吐。

本章主要贡献如下：

（1）提出自适应范围过滤器——Hourglass，通过轻量级混合编码与半排序自适应机制，有效防御重复误判查询。

（2）提出基于滑动窗口的半排序自适应更新机制，并给出相应的理论保证，以防御重复误判查询。

（3）提出基于处理器架构的范围过滤器构建访存优化方案，通过向量化、预取、大页与两阶段构建降低构建开销。

（4）在多种数据集和工作负载上进行全面评估，验证 Hourglass 在对抗性工作负载下误判率降低 9.8-35.4 倍，在多种场景下均表现出良好的鲁棒性，且自适应开销可忽略不计。

为便于后文阅读，本章高频符号统一见表 3-1；低频符号在正文相应位置就地解释。

表 3-1 第三章常用符号说明

符号	含义	符号	含义
L	键总位长	l	后缀位长（$M = 2^l$）
p	键前缀	s	键后缀
f	指纹长度	n_{fe}	窗口索引位数
D	查询关联度	ϵ	范围误判率
ρ	后缀块密度	β	位图负载因子

3.2 基于半排序指纹的范围查询过滤器总体设计

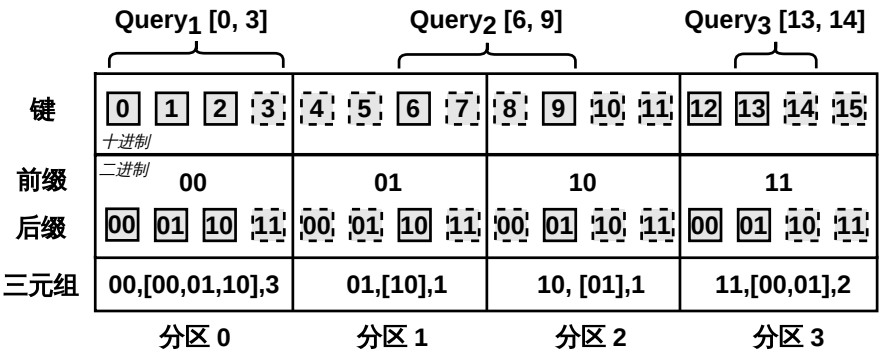
本节首先阐述 Hourglass 的设计目标与核心思路；此后依次介绍自适应半排序布谷鸟过滤器、混合后缀块、紧凑比特位图三个组件的数据结构，并详述构建流程与查询流程。

3.2.1 设计目标

在倾斜和对抗性范围查询场景中，同一查询可能多次触发误判，导致误判率显著上升，而现有范围过滤器无法有效应对这一问题。为此，本节提出 Hourglass 自适应范围过滤器，具备以下三项特性：（1）**自适应兼容性**——过滤器能有效处理范围查询，并自适应重复误判（见第 3.2.2 节）；（2）**低查询复杂度**——过滤器尽量减少计算量，即使面对大范围查询也能保持高效（见第 3.2.3 节）；（3）**高效内存利用**——过滤器足够紧凑以适应内存，同时保持较低的误判率（见第 3.2.2 节）。

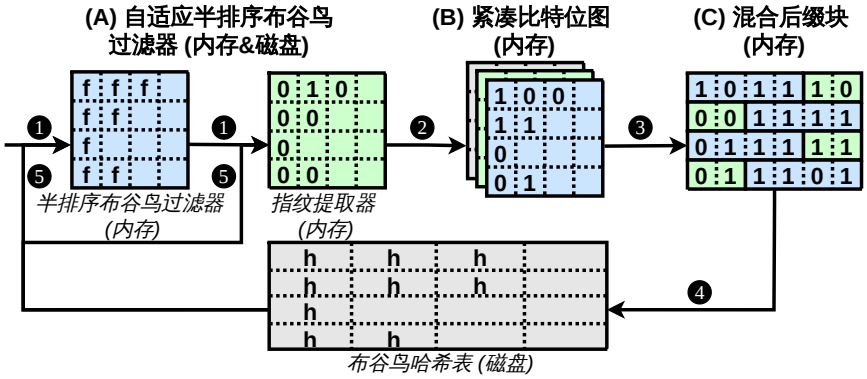
为防御重复误判，Hourglass 基于指纹式过滤器（Fingerprint-based Filter）的设计思想。这类过滤器从键的哈希码的特定部分提取指纹，从而利用剩余比特位来更新指纹。然而，已有的指纹式过滤器，如布谷鸟过滤器（Cuckoo Filter）[6]和商过滤器（Quotient Filter）[71]，均无法支持范围查询。

为构建指纹式范围过滤器，本节的核心思路是：最大范围大小为 $M = 2^l$ 的查询最多与两个相邻的大小为 M 的区间相交。如图 3-2 所示，全域 $[0, 2^L - 1]$ 中的每个键被拆分为前缀 p（长度 $L - l$）和后缀 s（长度 l），其中 L 为键的总长度，l 为后缀长度，2^l 为最大范围大小。分区（Partition）定义为共享相同前缀的键的集合，由公共前缀、后缀数组及数组长度三元组表示。公共前缀以指纹形式



虚线方格表示空键。[0, 15] 中的键被划分为 2 位前缀和 2 位后缀。由共同前缀、后缀数组和数组长度组成的三元组表示共享相同前缀的键的分区。范围大小 ⩽ 4 的查询最多与相邻的两个分区相交

图 3-2 自适应范围过滤器的设计思路

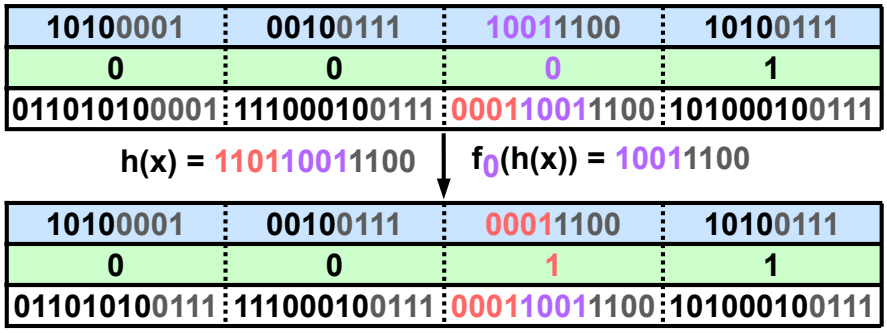


前缀、后缀数组的元数据以及后缀数组分别存储在 (A)、(B) 和 (C) 中。范围查询流程依次包括：查询指纹（❶）、获取后缀数组元数据（❷）、访问后缀数组（❸）、验证哈希码（❹）以及更新指纹（❺）

图 3-3 自适应范围过滤器的组成与查询流程

存储，后缀数组紧凑存储，数组长度经编码后建立前缀与后缀数组之间的一一对应关系。

基于上述观察，本节提出 Hourglass（如图 3-3 所示），一种专为防御范围查询中重复误判而设计的自适应范围过滤器。Hourglass 由以下三个组件构成：（1）**自适应半排序布谷鸟过滤器（Adaptive Semi-sorted Cuckoo Filter, ASCF）**：一种新型数据结构，用于存储前缀并支持自适应性。它使用半排序布谷鸟过滤器存储前缀指纹，使用指纹提取器指定指纹的提取方式，并在磁盘上使用布谷鸟哈希表存储键的哈希码。与标准布谷鸟过滤器类似，ASCF 保证常数查询复杂度。（2）**混合后缀块（Hybrid Suffix Blocks, HSB）**：负责紧凑存储后缀数组。它根据稀疏程度，采用位图和二进制表示对后缀数组进行混合编码，并通过二分搜索和 `popcount` 操作查询后缀块。（3）**紧凑比特位图（Compact Bitwise Bitmaps,**



蓝色、绿色和白色桶分别表示半排序布谷鸟过滤器、指纹提取器和布谷鸟哈希表

图 3-4 查询 x 时 ASCF 中自适应性的示例

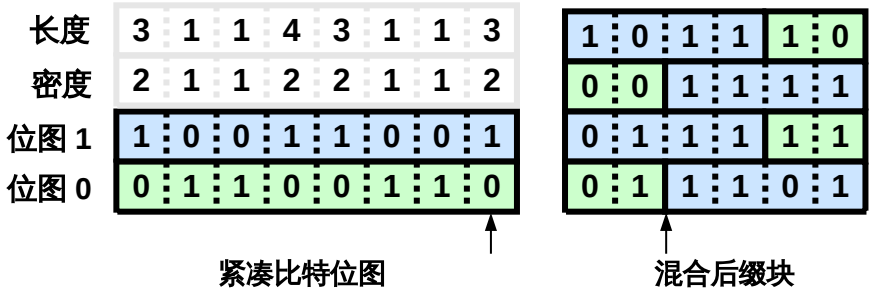


图 3-5 紧凑比特位图与混合后缀块

CBB)：将 ASCF 中的指纹与 HSB 中的后缀块建立一一对应关系。它高效编码后缀块的偏移量和长度，并支持常数时间查询。

3.2.2 数据结构

自适应半排序布谷鸟过滤器

自适应半排序布谷鸟过滤器（Adaptive Semi-sorted Cuckoo Filter, ASCF）用于存储前缀并支持自适应性，由三个主要组件构成：半排序布谷鸟过滤器、指纹提取器和布谷鸟哈希表。半排序布谷鸟过滤器存储由滑动窗口提取的指纹；指纹提取器是一个位图，存储与指纹对应的窗口索引；布谷鸟哈希表保存与指纹对应的哈希码，存储于磁盘上，用于判断是否需要进行半排序自适应更新。指纹提取器和布谷鸟哈希表的容量与半排序布谷鸟过滤器保持一致。

以图 3-4 中的示例说明各组件的协作方式。在 ASCF 中查询 x 时，首先获取其哈希码 `110110011100`；随后在每个桶的第三个槽中，检索指纹 `10011100` 和窗口索引 0。在窗口索引为 0 时，x 的指纹为 `10011100`，与槽中存储的指纹匹配，因此 ASCF 对 x 的查询返回真。接着检索第三个槽中的哈希码 `000110011100`，

发现与 x 的哈希码不同，表明这是一个误判。为防止未来查询中再次出现该误判，滑动窗口向前移动 4 位（窗口索引加一），提取新指纹 `00011100`。虽然远程访问布谷鸟哈希表会增加正查询的延迟，但第 3.5.3 节的实验表明，该开销与自适应性带来的收益相比可忽略不计。

混合后缀块

混合后缀块（Hybrid Suffix Blocks, HSB）用于高效存储后缀数组。为优化空间，后缀数组根据其稀疏程度，通过混合位图和二进制表示进行编码，如图 3-5 所示。后缀数组在编码前先去重并按升序排序。

在位图表示中，分区中的每个后缀将位图中对应位置设为 1，位图大小与分区大小 2^l 对齐（l 为后缀长度）。利用 CPU 单指令多数据（Single Instruction Multiple Data, SIMD）指令集中的 `popcountll`，可快速计算位图中任意位置前的 1-bit 数量。

然而，当分区稀疏时，直接存储二进制序列更节省空间。在二进制表示中，二进制序列大小为 $n \times l$，其中 n 为分区内后缀数量，l 为后缀长度。为达到最优空间利用率，当 $n < \frac{2^l}{l}$ 时采用二进制表示，否则采用位图表示。例如，在图 3-2 中，分区零的二进制表示为 `000110`，位图表示为 `1110`。由于 $n = 3$，$l = 2$，选择更短的位图表示作为存储格式。

为统一两种表示方式，引入**块密度**（Block Density）的概念来描述其大小。对于二进制型后缀块，块大小为 $n \times l$；对于位图型后缀块，块大小固定为 $l \times \lceil \frac{2^l}{l} \rceil$。块密度 ρ 定义为块大小与后缀长度之比：二进制型后缀块的密度 $\rho = n < \lceil \frac{2^l}{l} \rceil$，位图型后缀块的密度 $\rho = \lceil \frac{2^l}{l} \rceil$。

虽然将后缀块作为负载存储在布谷鸟过滤器中看似直观，但后缀块的异构性会因对齐而导致较大的空间开销。为此，后缀块被连续存储以构成 HSB 结构，并使用第 3.2.2 节中详述的紧凑比特位图结构对后缀块进行定位。

紧凑比特位图

紧凑比特位图（Compact Bitwise Bitmaps, CBB）用于建立前缀（指纹）与后缀数组（后缀块）之间的一一对应关系。要从 HSB 结构中检索某个后缀块，需

要两条元数据：后缀块偏移量和后缀块大小。然而，直接存储所有后缀块的偏移量和大小空间效率低下。为此，紧凑比特位图仅编码后缀块的密度，同时仍能高效解码其偏移量和大小。

编码：为优化空间，密度以比特为粒度进行编码。如图 3-5 所示，紧凑比特位图由多个位图组成，每个位图编码块密度的一个比特。位图数量 c 设为 $\log_2(\rho_{\max}+1)$，其中 $\rho_{\max}$ 为最大密度。例如，若 $\rho_{\max}=3$，则 $c=2$；对于密度 $\rho=2$ (10_2) 的后缀块，最低有效位 (0) 存储在位图零中，最高有效位 (1) 存储在位图一的对应位置。位图大小与半排序布谷鸟过滤器的槽数一致，确保位图条目与 ASCF 中的指纹一一对应。

解码：为高效检索后缀块，其元数据按如下步骤解码：（a）**获取密度：**给定位图中的位置（$\geqslant 0$），逐位连接各位图在该位置的值，得到后缀块密度 ρ。例如，在图 3-5 中，位置 7 处 $\rho=0\times 2^0+1\times 2^1=2$。（b）**计算累积密度：**利用秩（Rank）操作，按公式 (3-1) 计算该位置前所有后缀块的累积密度 ρ_c：

$$\rho_c=\sum_{i=0}^{n_{bt}}\mathrm{rank}_1(bt_i,j)\times 2^i, \tag{3-1}$$

其中 n_{bt} 为位图数量，j 为位置，$\mathrm{rank}_1(bt_i,j)$ 返回第 i 个位图中位置 j 前的 1-bit 数量。在图 3-5 中，$\rho_c=4\times 2^0+3\times 2^1=10$。(c) **确定块大小和偏移量：**后缀块大小计算为 $\rho\times l$，后缀块偏移量计算为 $\rho_c\times l$，其中 l 为后缀长度。例如，当 $l=2$ 时，块偏移量为 20，块大小为 4。(d) **检索后缀块：**根据偏移量和大小，检索后缀块 1101，如图 3-5 所示。(e) **解码后缀数组：**根据后缀块密度 $\rho=2=\frac{2^l}{l}$，依据第 3.2.2 节原则可知该后缀块类型为位图，因此将后缀数组解码为 `[00, 01, 11]`。

3.2.3 构建与查询过程

Hourglass 的构建流程如算法 1 所示，包含四个步骤：采样与建模（第 1–2 行）、键分区（第 3 行）、分区插入（第 4–7 行）和位图压缩（第 8 行）。本节重点介绍后三个步骤，第一步在第 4.4 节中详述。

算法 1 Hourglass 构建算法

输入： 键集合 S，每键比特数 b，最大范围大小 M

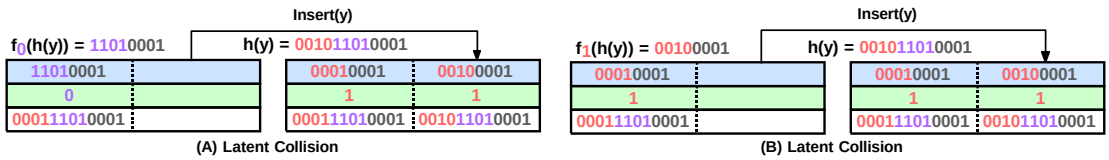
```
ts ← GATHER(S, l_s)
for (p, n, arr) in ts do
    INSERTPREFIX(p)
    INSERTCBB(n)
    INSERTSUFFIX(arr)
end for
COMPRESS()
```

构建过程

键分区： 为将共享公共前缀的键归组，首先如第 3.2.1 节所述，将每个键拆分为前缀 p 和后缀 s。具有相同前缀 p 的键被放入同一分区。因此，每个分区由三元组表示：前缀哈希码 $h(p)$、分区内不重复后缀数量 n 和后缀数组 $arr[s_1, s_2, \cdots, s_n]$。

分区插入： 本节首先引入预适应（Pre-adaptivity）策略，以防止将来产生误判；然后介绍将三元组插入 Hourglass 的详细过程。

（1）**预适应策略：** 预适应策略用于处理插入阶段 ASCF 中的碰撞，重点是如何更新碰撞元素的指纹以防止未来误判。如图 3-6 所示，插入过程中可能发生两类碰撞。



蓝色、绿色和白色桶分别表示半排序布谷鸟过滤器、指纹提取器和布谷鸟哈希表

图 3-6 插入 y 时两类碰撞的示例

第一类是直接碰撞，即两个碰撞元素在当前窗口索引下共享相同指纹。如图 3-6 (A) 所示，指纹长度 $f = 8$，x 的哈希码 $h(x) = 000111010001$，当前索引 $idx(x) = 0$，y 的哈希码 $h(y) = 001011010001$。在索引 0 处发生直接碰撞，x 和 y 的指纹均为 11010001。为解决此问题，ASCF 同时更新 x 和 y 的指纹，确保后续对 x 和 y 的查询不再产生误判。

第二类是潜在碰撞，即两个元素在当前窗口索引下指纹不同，但在较小的窗口索引下可能发生碰撞。如图 3-6 (B) 所示，插入 y 时，在当前索引 1 处不发生碰撞，但在索引 0 处会发生碰撞。在这种情况下，ASCF 使用滑动窗口在索引 1 而非默认索引 0 处插入 y 的指纹；若桶中有多个元素，ASCF 在最高可用索引处

插入指纹，主动适应以防止未来误判。

（2）**插入三元组：**按分区方式插入键，使用三元组 $(h(p), n, arr)$，分别对应 ASCF、CBB 和 HSB。向 ASCF 插入前缀时，先通过哈希码 $h(p)$ 定位桶；然后由预适应策略确定指纹提取的窗口索引；最后将哈希码存放在布谷鸟哈希表的对应位置。

向 CBB 插入元数据时，首先计算插入位置：给定半排序布谷鸟过滤器中的指纹位置 (i, j) 和桶大小 b，对应的 CBB 索引为 $k = i \times b + j$；然后将后缀数量 n 转换为密度值 ρ，存储在位图的位置 k 处。后缀数组在 HSB 中连续存储，但由于指纹的随机放置（通过哈希）导致直接插入困难。为此，先记录指纹与其后缀数组的映射关系，然后按序遍历布谷鸟过滤器，检索数组，将其转换为后缀块，并构建 HSB。

位图压缩： 该步骤的目标是在保持高效查找操作的同时优化位图的空间使用。由于指纹提取器的位图始终稀疏，使用 Roaring 位图[72]对其进行压缩；压缩后，指纹提取器实现最小空间占用，每键不足 0.1 位。

受已有工作[11]的启发，本节尝试使用 Elias-Fano 编码[73]压缩 CBB 中的位图。然而，Elias-Fano 编码在处理稠密位图时所需空间会急剧增加。此外，精简数据结构中常用的秩-选择索引（Rank-Select Index）[71,74] 会产生原始位图 1.25× 的固定空间开销，但能快速统计位图中的 1-bit 数量。为在空间效率和查询性能之间取得平衡，Hourglass 采用混合方式，结合 Elias-Fano 和秩-选择技术对位图进行编码。

CBB 中的位图动态编码方式如下：通过采样不同位图负载因子并计算编码后的空间占用，分析负载因子对空间效率的影响。实验表明，当位图负载因子超过约 30% 时，Elias-Fano 编码的空间效率低于秩-选择索引。因此，当负载因子低于 30% 或高于 70% 时采用 Elias-Fano 编码；否则采用秩-选择索引。

查询过程

Hourglass 的查询流程如算法 2 所示。首先，将范围查询的左右边界拆分为两个前缀-后缀对 (p_l, s_l) 和 (p_r, s_r)（第 1 行）。

若 $p_l = p_r$（第 2–5 行），范围查询仅与一个分区相交。此时用前缀 p_l 和后缀范围 $[s_l, s_r]$ 表示该范围查询。若前缀 p_l 在 ASCF 中为正，则查询 CBB 获取对应

算法 2 Hourglass 查询算法

输入： 左边界 L，右边界 R

```
1:  (p_l, p_r, s_l, s_r) ← Partition(L, R)
2:  if p_l = p_r then
3:      if FindPrefix(p_l) and SearchSuffix(s_l, s_r) then
4:          Adapt()
5:          return true
6:      end if
7:  else
8:      if FindPrefix(p_l) and s_l ⩽ Max(p_l) then
9:          Adapt()
10:         return true
11:     end if
12:     if FindPrefix(p_r) and s_r ⩾ Min(p_r) then
13:         Adapt()
14:         return true
15:     end if
16: end if
17: return false
```

后缀块的偏移量和大小，并将其转换为后缀数组。若后缀范围与后缀数组相交（第 3 行），则在返回真之前（第 5 行）访问布谷鸟哈希表进行自适应检查（第 4 行）。为识别误判，Hourglass 将 p_l 的哈希码与布谷鸟哈希表中对应位置存储的哈希码进行比较；若两个哈希码不同，则表明这是误判，此时在 ASCF 中使用半排序自适应更新对应的指纹，以防止未来重复误判。

若 $p_l \neq p_r$（第 6–12 行），范围查询与两个相邻分区相交。对于左分区，以与单分区情况相同的方式检索后缀数组；将 s_l 与后缀数组中的最大值 s_{max} 进行比较（第 7 行）：若 $s_l \leqslant s_{max}$，则执行自适应检查并返回真。类似地，对于右分区，将 s_r 与其后缀数组中的最小后缀 s_{min} 比较以确定结果（第 10–12 行）。

3.3 基于滑动窗口的范围过滤器自适应更新机制

本节首先介绍自适应更新机制的基本原理；随后对自适应机制的正确性、自适应能力与查询复杂度进行理论分析。

3.3.1 自适应更新机制

为实现范围过滤器的自适应能力，本节基于指纹式过滤器提出半排序自适应策略。

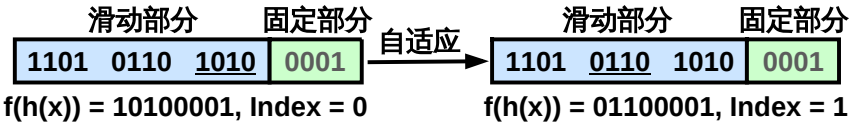


图 3-7 对 x 进行半排序自适应的示例

如图 3-7 所示，哈希码被划分为**固定部分**（Fixed Part）和**滑动部分**（Sliding Part）。固定部分包含最低有效 4 位（LSB），滑动部分包含其余各位。设指纹长度为 f（$f \geqslant 4$），则：指纹的最低有效 4 位来自固定部分；其余 $f-4$ 位从滑动部分提取；使用长度为 $f-4$ 的滑动窗口从滑动部分提取这 $f-4$ 位。

这一设计与半排序布谷鸟过滤器的编码方式相一致。由于半排序编码依赖桶内指纹相对顺序的稳定性，本文在实现中采用对最低有效位进行半排序编码的方式，以兼顾空间效率与在线调整能力。选择最低有效位而非最高有效位的原因在于，基于滑动窗口的自适应更新机制需要在保持桶内相对顺序不变的前提下调整指纹，而最低有效位固定更有利于维持半排序编码的稳定性。

默认情况下，滑动窗口包含滑动部分最低有效的 $f-4$ 位，窗口位置（窗口索引）设为 0。当发生误判时，滑动窗口向前移动 $f-4$ 位，索引加一。由于最低有效 4 位在更新过程中保持不变，因此半排序顺序得以保留。

通过上述策略，Hourglass 能够自适应范围查询中的重复误判。具体地，对于范围查询，Hourglass 首先识别查询所属的对应分区，然后查询半排序布谷鸟过滤器和指纹提取器以验证这些分区的存在性（❶）；对于每个存在的分区，查询 CBB 以获取对应后缀块的偏移量和长度（❷），并使用这些值从 HSB 中检索后缀块（❸）；若后缀数组与查询相交，则查询磁盘上的布谷鸟哈希表以验证是否为误判（❹）；若确认为误判，则根据自适应更新策略更新指纹和窗口索引（❺）。

3.3.2 自适应更新理论保证

本节从正确性、自适应上界、计算复杂度与访存代价四个方面分析自适应更新机制，说明该机制在保持结构稳定性的同时能够以低开销抑制重复误判。

正确性与自适应上界

半排序自适应策略通过滑动窗口更新误判查询对应的前缀指纹。由于固定部分（LSB 的 4 位）在更新过程中始终不变，桶内指纹的半排序相对顺序不会被破坏，因此 ASCF 的查询语义保持一致。

设哈希码滑动部分长度为 k 位，指纹长度为 f，则滑动窗口最多可前移 $\lfloor \frac{k}{f-4} \rfloor$ 次，即同一键最多可适应 $\lfloor \frac{k}{f-4} \rfloor$ 次重复误判。在本文实验设置中，对抗性负载下指纹提取器窗口索引取 $n_{fe} = 1$ 位，每键支持 1 次自适应更新，已可有效抑制重复误判。

查询计算复杂度分析

ASCF 与 CBB 中的操作均基于哈希函数与秩-选择运算，时间复杂度为 $O(1)$。对于 HSB 的后缀检索，分以下两种情形分析。

单分区情形。当范围查询仅涉及一个分区时，需在二进制后缀块中执行二分搜索，计算复杂度为 $O(L)$，其中 L 为完整后缀长度。对于位图型后缀块，利用 CPU SIMD 指令集中的 `popcountll` 操作，可在单条指令内并行统计多个比特位，将位图查询复杂度降至 $O\left(\frac{2^L}{i}\right)$，其中 i 为 SIMD 指令的操作位宽（实际中 $i = 64$ 或 $i = 128$）。

双分区情形。当范围查询跨越两个相邻分区时，仅需对每个分区的后缀数组各执行一次边界比较，计算复杂度为 $O(1)$，与范围大小及后缀长度均无关。

综上，Hourglass 的范围查询保持 $O(1)$ 的常数时间上界。其核心原因在于最大范围大小 $M = 2^l$ 的约束使单次查询最多涉及两个相邻分区，避免了随范围增长的递归或迭代遍历。

查询访存延迟分析

一次完整的范围查询所需内存访问次数分析如下：对 ASCF 的访问至多 2 次，对 CBB 的访问 n_{cbb} 次，对 HSB 的访问 2 次，合计至多 $4 + n_{cbb}$ 次内存访问。

自适应性的引入为每次查询增加了 n_{fe} 次额外访问，其中 n_{fe} 为指纹提取器中窗口索引所占的比特数。实验表明，在对抗性工作负载下，取 $n_{fe} = 1$ 即可实现充分的自适应能力，因此自适应性带来的额外访存代价极低。

虽然每次误判触发一次对磁盘布谷鸟哈希表的远程访问，但与底层存储系统的实际磁盘 I/O 延迟相比，该开销可忽略不计。第 3.5.3 节的端到端实验进一步验证了这一结论。

3.4 基于处理器架构的范围过滤器构建访存优化

如图3-11所示, Hourglass 的构建性能存在明显瓶颈。第3.5.3小节中性能分析表明，主要开销集中于两处：（1）Cuckoo Filter 的随机插入与键踢出（Kickout）——每次桶访问命中末级缓存的概率极低，多数情况下退化为延迟约 100 ns 的主存随机访问；（2）相关查询场景下的后缀写入——原始方案以随机跳跃的方式向混合后缀块写入后缀数据，破坏了空间局部性。本节针对上述瓶颈，分别从向量化指令、访存预取、大页内存及两阶段构建四个维度提出优化方案，各小节依次展开论述。

3.4.1 向量化空槽检测

Cuckoo Filter 的每个桶包含 4 个 64 位槽位，插入时须先检测是否存在空槽（即槽值为 0）。原始实现以标量循环逐槽比较：

```
// 循环检测
for (size_t j = 0; j < bucket_size_; j++) {
    if (value_table_[i][j] == 0) {
        value_table_[i][j] = current_value_;
        return true;
    }
}
```

4 个槽位共占 $4 \times 8 = 32$ 字节，恰好对应一个 256 位高级向量扩展 2（Advanced Vector Extensions 2, AVX2）寄存器的宽度。利用 AVX2 指令，可将上述串行循环替换为一次向量化操作：一条 `_mm256_loadu_si256` 将整个桶加载为向量寄存器，`_mm256_cmpeq_epi64` 同时对 4 个槽位执行零值比较，`_mm256_movemask_epi8` 将比较结果压缩为 32 位掩码，最后以 `__builtin_ctz` 定位第一个空槽：

```
uint64_t* bucket_ptr = value_table_ + (curindex *
    bucket_size_);
__m256i bucket_data  = _mm256_loadu_si256((__m256i*)
    bucket_ptr);
__m256i zero         = _mm256_setzero_si256();
__m256i cmp          = _mm256_cmpeq_epi64(bucket_data, zero);
int     mask         = _mm256_movemask_epi8(cmp);

if (mask != 0) {
    // __builtin_ctz 返回最低置位的字节偏移，>> 3 转换为槽位下标
    size_t slot = __builtin_ctz(mask) >> 3;
    value_table_[curindex * bucket_size_ + slot] =
    current_value_;
}
```

由于 _mm256_movemask_epi8 以字节为粒度提取每个字节的最高位，而每个 64 位槽占据 8 个字节，因此掩码中最低置位的字节偏移量右移 3 位方可得到槽位下标。该方案将 4 次串行条件跳转压缩为常数时间的向量操作，在需要检查所有槽位才能确认满桶的高负载场景下优势尤为显著。

3.4.2 预取式延迟隐藏

Cuckoo Filter 的键踢出机制使单次插入可能需要连续访问多个随机桶，相邻两次桶访问之间存在百纳秒量级的主存延迟。__builtin_prefetch 利用 CPU 计算单元与内存子系统之间的流水线并行性，在当前桶操作执行期间，提前向内存控制器发出下一候选桶的预读请求，从而将访存延迟与计算时间相互覆盖。

具体地，每当踢出完成并计算出新候选桶下标后，立即发起预取：

```
// 发起对下一候选桶的写预取，时间局部性最高（驻留 L1 缓存）
__builtin_prefetch(&value_table_[curindex * bucket_size_], 1,
    3);
```

```
for (uint32_t count = 0; count < kMaxCuckooCount; count++) {
    // ... 空槽检测、标签写入、踢出逻辑 ...

    curindex = AltIndex(curindex, TagHash(current_value_));

    // 在进入下一迭代前，对再下一个候选桶发起预取
    __builtin_prefetch(&value_table_[curindex * bucket_size_
   ], 1, 3);
}
```

__builtin_prefetch 的第二个参数为 1（写预取），第三个参数为 3（高时间局部性）。由于踢出链的计算涉及哈希运算、标签读写等若干条指令，为预取请求提供了充足的执行窗口，使得数据在实际访问前已载入缓存，有效消除了主存等待。在相关查询负载构建路径中，同一策略也被应用于 prefix_index_table 的预取，以隐藏前缀索引表的访存延迟。

3.4.3 大页化 TLB 优化

操作系统以 4 KB 为默认页粒度管理虚拟地址到物理地址的映射。当布谷鸟哈希表的体积达到数十 MB 时，其所跨越的页数远超转译后备缓冲区（Translation Lookaside Buffer, TLB）的容量（1024 项 TLB 在 4 KB 页下仅能覆盖 4 MB），导致每次随机桶访问几乎都引发 TLB 缺失，退化为需要遍历内存页表的多级访问。将页大小提升至 2 MB（大页，Huge Page）后，所需的页表条目数减少 512 倍，相同大小的布谷鸟哈希表仅需数十个条目即可完全驻留于 TLB，彻底消除地址转换的缺失开销。原始实现以 new 分配布谷鸟哈希表：

```
// 原始方案：分散分配，碎片化，无法利用大页
value_table_ = new uint64_t*[num_buckets_]();
uint64_t* data = new uint64_t[num_buckets_ * bucket_size_]();
for (size_t i = 0; i < num_buckets_; i++)
    value_table_[i] = data + i * bucket_size_;
```

优化方案改用 mmap 并携带 MAP_HUGETLB 标志申请大页内存，并在大页不可用时自动回退至普通匿名映射：

```
void* ptr = mmap(NULL, total_bytes, PROT_READ | PROT_WRITE,
                 MAP_PRIVATE | MAP_ANONYMOUS | MAP_HUGETLB,
    -1, 0);

if (ptr == MAP_FAILED) {   // 系统未配置大页时自动回退
    ptr = mmap(NULL, total_bytes, PROT_READ | PROT_WRITE,
               MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
}

value_table_ = static_cast<uint64_t*>(ptr);
```

与 new 相比，基于 mmap 的分配还保证了内存区域的物理连续性，消除了堆分配器的碎片化问题，有助于预取指令的流式预读效果进一步发挥。

3.4.4 两阶段顺序写入

在相关查询构建场景下，混合后缀块按桶顺序线性存储各槽位的后缀数据，其物理布局与 Cuckoo Filter 的桶地址分布一致，而非与原始键集的有序性相关。若在插入过程中即时写入后缀，由于插入目标位置由哈希值决定，后缀写入呈随机跳跃分布，访存效率低下。

为解决上述问题，本节将构建流程重组为两个解耦阶段，以前缀索引（prefix_index）为桥梁传递阶段间信息。

第一阶段——带索引的前缀随机插入：对每个唯一前缀 p，记录其在有序键集中首次出现的编号 prefix_index，并将其与前缀哈希值一同写入辅助数组 prefix_index_table 的对应桶槽。prefix_index 仅占 4 字节，在踢出链中随前缀哈希值同步迁移，确保两者始终保持位置一致。此阶段写入的数据量远小于完整后缀集合，降低了踢出链中的写带宽消耗。

第二阶段——基于前缀索引的后缀顺序写入：第一阶段完成后，顺序遍历所有桶槽，对每个非空槽位 (i, j)，从 prefix_index_table[i][j] 读取 prefix_index，以此为起点在有序键集中顺序扫描，收集该前缀的所有唯一后缀，并将其按顺序追加写入混合后缀块。由于遍历顺序与混合后缀块的物理布局完全对齐，后缀写入呈严格的顺序追加模式，硬件流式预取器可提前将待

写区域载入缓存，访存带宽利用率接近理论峰值。上述两阶段构建流程如算法 3 所示。

算法 3 相关查询负载下的两阶段构建

输入： 有序键集 $[begin, end)$

// 第一阶段：前缀随机插入，同步记录 prefix_index
for 每个唯一前缀 p，其在键集中首次出现的编号为 idx **do**
    INSERT(p, idx, *prefix_index_table*)
end for
// 第二阶段：顺序遍历桶槽，顺序写入后缀
$pos \leftarrow 0$
for $i = 0$ **to** $N_{\text{buckets}} - 1$； $j = 0$ **to** 3 **do**
    $idx \leftarrow prefix_index_table[i][j]$； **若** $idx = 0$ **则跳过**
    以 idx 为起点顺序扫描键集，收集后缀集合 S
    将 S 追加写入 $HSB[pos:]$，$pos \mathrel{+}= |S| \cdot n_l$
end for

两阶段方案的核心优势在于访存模式的根本性转变：第一阶段保留了 Cuckoo Filter 原有的随机插入特性，但每次踢出仅迁移 4 字节的 `prefix_index`，而非完整的后缀数组；第二阶段将后缀写入从随机访存转变为严格的顺序追加，消除了混合后缀块写入阶段的缓存局部性破坏，与大页内存、SIMD 及预取指令共同构成完整的访存优化链路。

3.5 实验与性能评估

本节对 Hourglass 与当前最先进的范围过滤器进行实证评估，重点回答以下五个问题：（1）Hourglass 的自适应机制效果如何？（2）Hourglass 在不同关联度下的表现如何？（3）Hourglass 在真实世界数据集上的表现如何？（4）Hourglass 的构建与查询效率如何？（5）Hourglass 在端到端场景下的性能如何？

3.5.1 实验环境与配置

数据集：实验中使用 64 位无符号整数键，包含合成数据集和真实世界数据集，与范围过滤器领域现有评估工作保持一致[9-11,20,31-34,75]：（1）**均匀分布（Uniform）：**从 $[0, 2^{64})$ 中均匀生成的键；（2）**正态分布（Normal）：**服从正态分布的键，均值 $\mu = 2^{63}$，标准差 $\sigma = 2^{64} \times 0.1$；（3）**图书数据集（Books）：**从亚马逊图书数据中采样的 2 亿个整数；（4）**地图数据集（Osm）：**从 OpenStreetMap

中采样的 2 亿个地理整数。如 Grafite[32] 所建议，另外两个真实数据集 fb 和 wiki 可使用 Elias-Fano 编码无损压缩，每键仅需 12 位和 6 位，且无误判，因此仅展示 books 和 osm 数据集上的结果。

工作负载：本节生成 100 万条形如 $[x, x+2^L)$ 的范围空查询。选取 $L = 0, 5, 10$ 分别对应点查询、小范围查询和大范围查询。使用非关联、关联和对抗性工作负载分别模拟随机分布、靠近存储键以及极度倾斜的查询分布：（1）**对抗性（Adversarial）：**从 $[0, 2^{64})$ 中选取唯一键集合 U，x 按幂律分布（$\alpha = 0.5$）从 U 中抽取；误判率计算为误判数除以唯一查询数；（2）**非关联（Uncorrelated）：** x 从 $[0, 2^{64})$ 中随机选取；（3）**关联（Correlated）：**随机从键集中选取 k，x 从 $(k, k + 2^{30(1-D)}]$ 中随机选取，其中 D 为关联度；若无特殊说明，关联工作负载的 $D = 1$。

对比基准：本节与当前最先进的范围过滤器进行对比。SuRF[76]、Rosetta、SNARF[77]、Proteus[78] 和 REncoder[79] 的配置遵循 Grafite[80] 中的设置；对于 REncoder，仅展示其高级版本 REncoderSE 的结果。此外，本节还评估了 Grafite 中提出的简洁范围过滤器 Bucketing[80]；BloomRF[81] 使用代码中的默认范围查询配置；Oasis[82] 使用其高级版本 Oasis+ 的默认配置。

运行环境：所有实验均在 AWS t2.2xlarge 实例上进行，配备 Intel(R) Xeon(R) CPU E5-2686 v4（2.30GHz）和 32GB 内存。

其他配置： Hourglass 中的半排序布谷鸟过滤器桶大小设为 4，负载因子设为 0.95，遵循文献[6]的设置；对于对抗性工作负载，指纹提取器的窗口索引使用 $n_{fe} = 1$ 位。

3.5.2 误判率评估

自适应机制的有效性

在 16 BPK 的条件下，本节评估 Q/U 比从 1 到 20 变化时的对抗性工作负载误判率。为适应不断增大的 Q/U 比，保持 100 万条唯一查询，同时扩大查询集规模；误判率计算为误判数除以唯一查询数。

如图 3-8 所示，Hourglass 在大多数情形下均优于其他范围过滤器，且无论 Q/U 比如何变化，始终保持较低的误判率。实验结果表明，其他过滤器的误判率

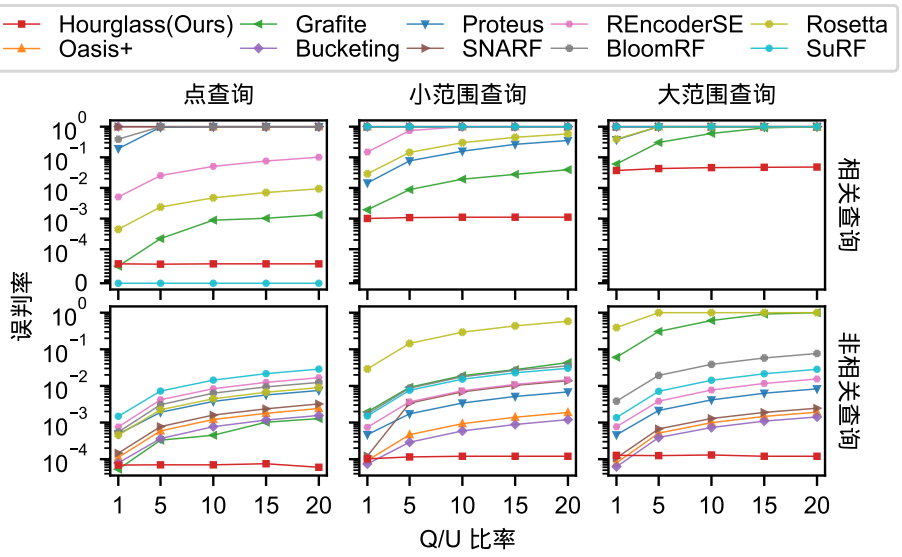


图 3-8 对抗性工作负载下的误判率对比

随 Q/U 比的增大而上升，这是因为重复误判不断累积所致。Hourglass 的误判率比次优过滤器低 9.8–35.4 倍。SuRF 在关联点查询中无误判，这是因为高键-查询关联度降低了与 SuRF 哈希函数发生碰撞的概率，如 Grafite[32] 所述；然而，SuRF 在其他场景下表现较差。此外，当误判查询数量超过唯一查询数量时，许多范围过滤器的误判率接近 1。

不同工作负载与真实数据集

多种分布下的误判率： 为全面评估 Hourglass，本节在均匀分布和正态分布数据集以及关联和非关联工作负载上开展实验，固定 20 BPK。

如图 3-9 所示，Hourglass 在均匀分布上表现优越，在正态分布上也具有竞争力，这得益于其轻量级混合编码。Hourglass 和 Grafite 在关联工作负载上领先，但 Hourglass 在非关联工作负载上显著优于 Grafite；在均匀分布的非关联工作负载上，Hourglass、SNARF、Bucketing 和 Oasis+ 均比 Grafite 优约一个数量级。在正态分布上，Oasis+、SNARF 和 Bucketing 超过 Hourglass，这是因为更密集的键簇提高了位图压缩效率，但它们在关联工作负载下表现较差。Hourglass 是唯一在关联和非关联工作负载上均表现良好的鲁棒过滤器。

真实世界数据集： 如图 3-10 所示，本节在多种范围大小和 BPK 下的真实世界数据集上评估 Hourglass。得益于高效稳定的编码方式，Hourglass 保持持续低且稳定的误判率。

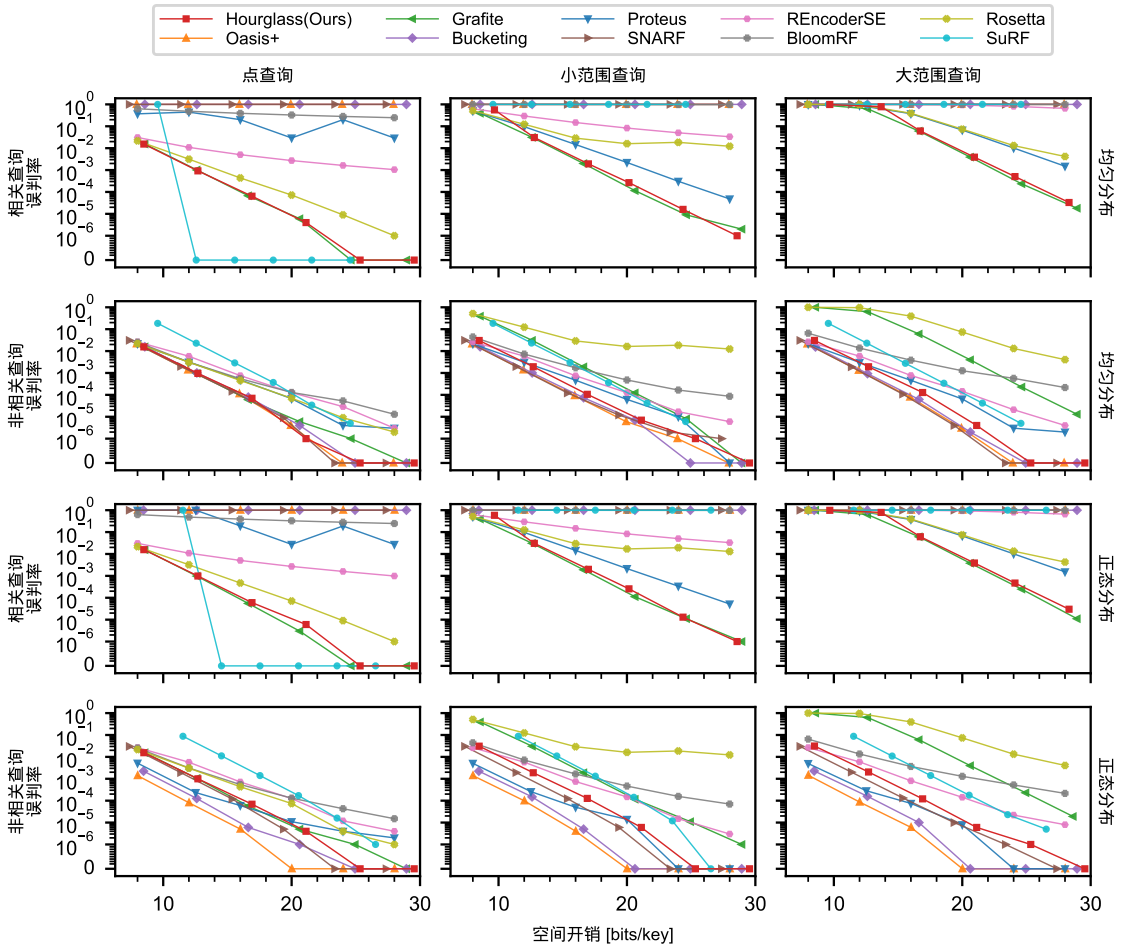


图 3-9 各类工作负载下的误判率对比

在点查询中，Hourglass 和 Grafite 优于所有其他范围过滤器，因为其哈希函数不受键分布影响。在 8 BPK 下，BloomRF、REncoderSE、Proteus 和 Rosetta 与 Hourglass 误判率相近，但随 BPK 增大，性能趋于平稳。Bucketing、SNARF 和 Oasis+ 在真实数据上表现较差，因为分布偏离了均匀或正态假设。

在范围查询中，Hourglass 在 Books 和 Osm 数据集上分别比次优的 Oasis+ 误判率低约 32 倍和 13 倍。Grafite 在 8 BPK 下的误判率比点查询分别增加了约 26 倍和 67 倍。尽管 Proteus 在 8 BPK 下于 Osm 数据集上略优于 Hourglass，但 Hourglass 在 12 BPK 以上即反超。

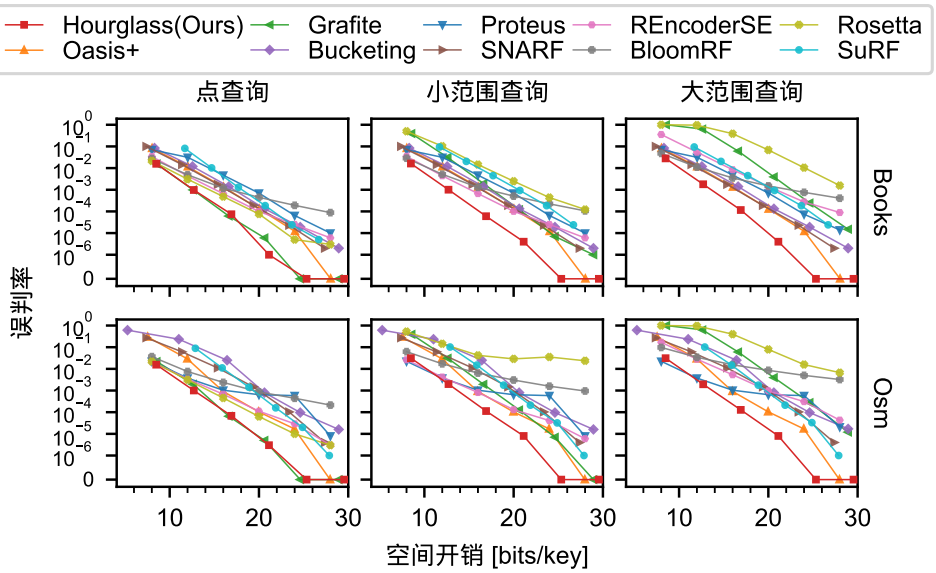


图 3-10 真实世界数据集上的误判率对比

3.5.3 运行性能评估

量化与定性分析

为分析 Hourglass 的构建与查询瓶颈，本节在函数级和指令级进行了详细剖析（表 3-2、表 3-3、表 3-4、表 3-5）。实验固定 BPK 为 16，使用大范围大小，在关联和非关联工作负载下进行；在指令级，测量了指令数量、L1 缓存加载量和缺失率，以评估计算开销和内存效率。

构建瓶颈：（a）量化分析：如第 3.2.3 节所述，构建流水线包含六个阶段：数据预处理、哈希计算、前缀插入、后缀偏移量插入、后缀插入和压缩。在函数级，构建时间由前缀插入主导（表 3-2）；在指令级，Hourglass 的 L1 缓存缺失率比竞争过滤器高 2.57–6.77 倍（表 3-4）。（b）定性分析：性能不足主要源于三个因素：（1）半排序布谷鸟过滤器和布谷鸟哈希表中的随机写入破坏了空间局部性；（2）布谷鸟过滤器中的重新插入增加了随机写入次数；（3）可变长度指纹阻碍了位掩码优化。相比之下，Bucketing 在位数组中顺序插入键，Grafite 使用并行排序和顺序哈希插入，SuRF 顺序构建字典树并通过 BST 进行编码。

查询瓶颈：（a）量化分析：如第 3.2.3 节所述，查询流水线包含五个阶段：

表 3-2 构建时间的函数级剖析

工作负载	数据预处理	哈希计算	前缀插入	偏移量插入	后缀插入	压缩
关联	12.38%	1.13%	64.75%	0.43%	13.62%	7.69%
非关联	3.76%	1.50%	90.88%	–	–	3.86%

表 3-3 查询时间的函数级剖析

工作负载	哈希计算	前缀查找	偏移量查找	后缀查找	指纹更新
关联	6.04%	69.38%	3.02%	21.45%	0.11%
非关联	6.89%	90.48%	–	–	2.63%

哈希计算、前缀查找、后缀偏移量查找、后缀查找和指纹更新。函数级剖析表明，前缀查找主导查询时间，而指纹更新开销可忽略不计（表 3-3）；在指令级，Hourglass 的 L1 缓存缺失率与 Grafite 和 SuRF 相当，但 Bucketing 所需指令数和缓存访问量远少于 Hourglass（表 3-5）。(b) 定性分析：Bucketing 的每次查询仅需两次前驱操作；而 Hourglass 的层次化半排序结构需要更多内存访问，且存在随机访问模式。

设计权衡： Hourglass 的设计哲学与 Rosetta[10] 所阐述的基本权衡一致——在过滤精度显著提升时，接受较高的计算开销是合理的。这是因为范围过滤器中的误判会导致不必要的 SSD 或 HDD 访问，即使最快的 SSD 也需要数十微秒来获取数据；相比之下，Hourglass 每键构建仅需数百纳秒，每次查询执行也仅需数百纳秒。

构建时间

本节在不同数据分布和键-查询关联度下评估构建时间。对于 Rosetta、Proteus、REncoderSE，在柱状图上方额外添加条纹以标示其建模时间。

如图 3-11 所示，Hourglass 的柱状图分为两部分：下方实心区域表示优化后的构建时间，上方条纹区域表示优化前后的差值。经过对构建流水线中前缀插入阶段的缓存友好优化，Hourglass 在非关联工作负载下的构建时间从约每键 460 ns 降低至约每键 120 ns，在关联工作负载下从约每键 690 ns 降低至约每键 176

表 3-4 构建时间的指令级剖析

过滤器	工作负载	指令数（10^9）	L1-dcache-loads（10^9）	L1 缺失率
Grafite	关联	3.69	0.62	7.58%
	非关联	3.69	0.62	7.55%
Bucketing	关联	2.03	0.34	5.42%
	非关联	2.03	0.34	5.42%
SuRF	关联	10.88	3.06	2.38%
	非关联	10.88	3.06	2.46%
Hourglass（本文工作）	关联	18.04	3.79	9.68%
	非关联	4.89	1.14	16.98%

表 3-5 查询时间的指令级剖析

过滤器	工作负载	指令数（10^9）	L1-dcache-loads（10^9）	L1 缺失率
Grafite	关联	0.30	0.04	16.49%
	非关联	0.28	0.04	15.97%
Bucketing	关联	0.16	0.02	3.98%
	非关联	0.16	0.02	2.90%
SuRF	关联	1.14	0.32	1.85%
	非关联	0.85	0.27	1.23%
Hourglass（本文工作）	关联	0.83	0.16	5.48%
	非关联	0.41	0.05	17.22%

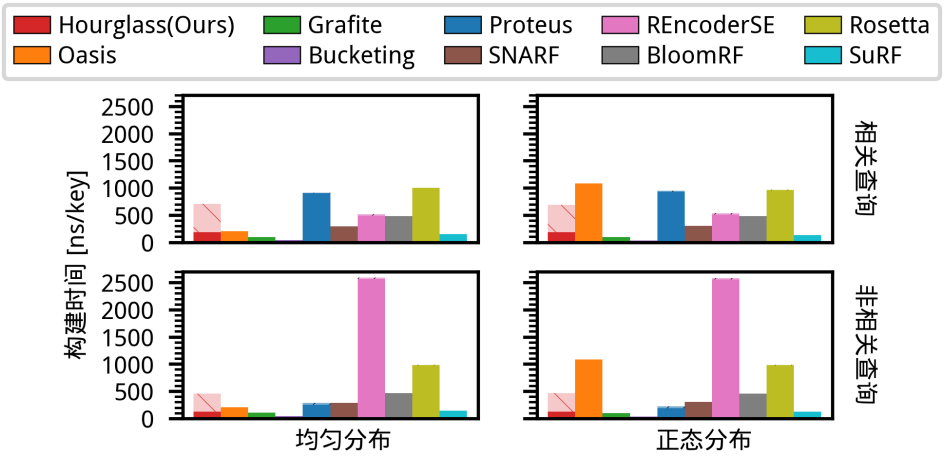


图 3-11 构建时间对比

ns，分别提升了约 3.8 倍和 3.9 倍。优化后，Hourglass 的构建速度在各场景下均优于或接近 Grafite、Bucketing 等快速过滤器。Oasis+ 在正态分布下每键需 1085 ns，比均匀分布慢约 5.4 倍，这是由于其两层嵌套循环的分段算法所致。Proteus 在关联工作负载下每键需 931 ns（慢约 4.9 倍），使用了类似 Rosetta 的前缀布隆过滤器。REncoderSE 在非关联工作负载下每键需 2556 ns（慢约 5.0 倍），因为需要分轮重建以优化层选择。Grafite、Bucketing、SNARF 和 SuRF 构建速度快，每键均不超过 300 ns，得益于其简洁的设计。

查询时间

如图 3-12 所示，本节在多种工作负载和空间占用下评估查询延迟。为确保公平比较，排除了误判引发的磁盘 I/O 开销，仅关注纯查询时间。得益于较低的计算复杂度，Hourglass 在多种场景下均稳定跻身最快范围过滤器的第二梯队前列。Bucketing 始终最快，但在对抗性和关联工作负载下表现较差。

在点查询中，Hourglass 每键延迟为 100–200 ns，优于大多数过滤器；Bucketing

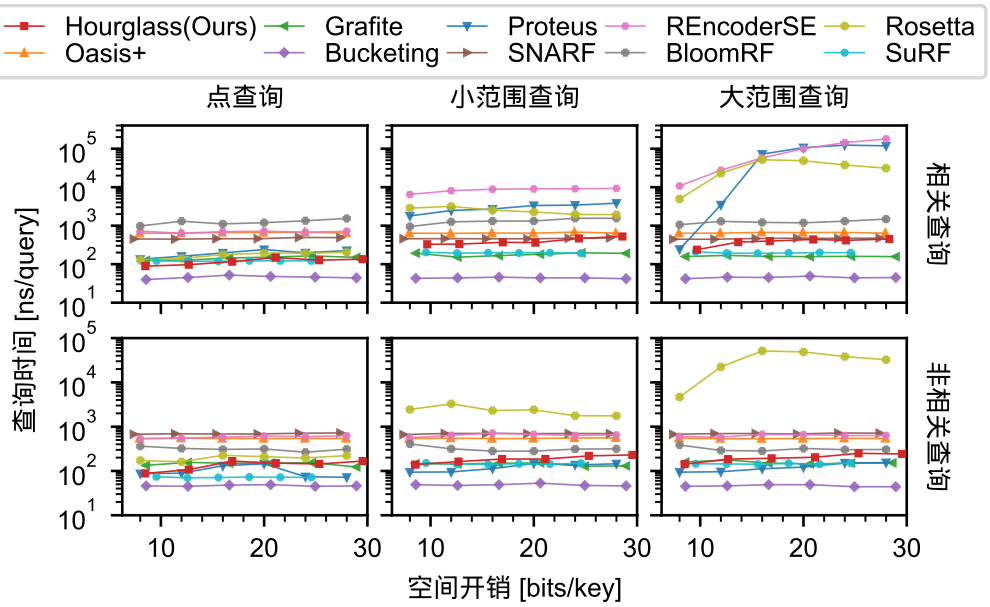


图 3-12 查询时间对比

最快，仅需两次前驱操作；SuRF 和 Proteus 也因高效的秩-选择操作而延迟较低。

在范围查询中，Rosetta 在小范围和大范围查询中分别慢约 9.5 倍和 153 倍，因为需要更多次布隆过滤器访问。REncoderSE 分别慢约 13 倍和 251 倍，因为其线段树随范围大小增长。Proteus 分别慢约 17 倍和 524 倍，因为需要在布隆过滤器上进行更多探测。

端到端查询延迟

为验证自适应性引入的开销极小，本节按照 REncoder[34] 的方法构建两级键值存储仿真环境：第一级存储范围过滤器，第二级存储完整键集合和布谷鸟哈希表。实验设置与第 3.5.2 节一致，唯一的区别是为每个键分配一个随机生成的 256 字节值，以模拟键值存储。

如图 3-13 所示，Hourglass 在对抗性工作负载（Q/U 比 > 1）下优于所有其他范围过滤器（关联点查询中 SuRF 除外）。在均匀查询（Q/U 比 = 1）下，Hourglass 的延迟略高于 Bucketing 和 Oasis+，原因是其较高的误判率导致更频繁的磁盘访问。在对抗性工作负载下，Hourglass 的端到端延迟比次优过滤器低 1.4–37.7 倍。由于 Hourglass 中的布谷鸟哈希表与指纹一一对应，读取 64 位哈希码只需根据偏移量进行一次访问；相比之下，从磁盘读取数据需要在索引块和数据块上进行二分搜索。因此，该开销与减少大量磁盘访问带来的收益相比可忽略不计。

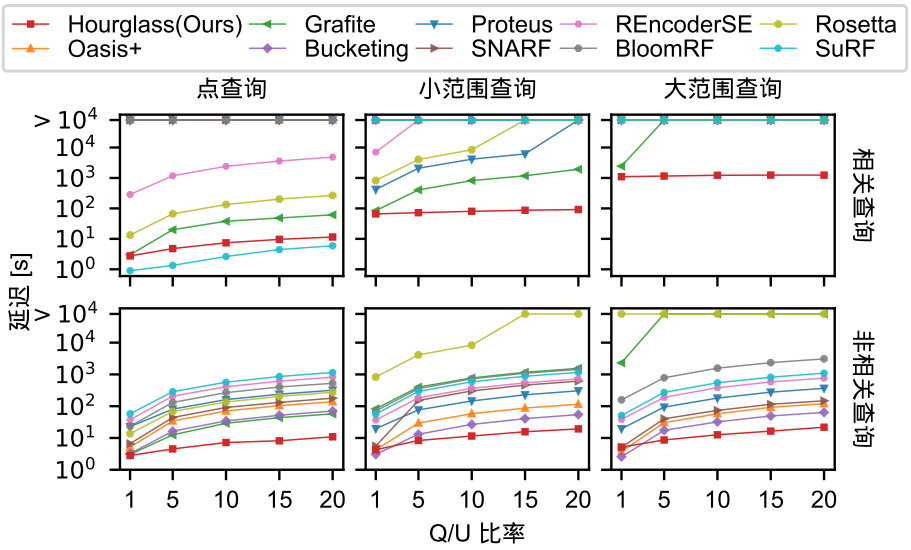


图 3-13 端到端查询延迟对比

3.5.4 鲁棒性分析

不同偏斜分布

如图 3-14 所示，本节在四种不同查询分布下评估 Hourglass 的鲁棒性：幂律分布（$\alpha = 0.5$）、Zipf 分布（$s = 0.5$）、双峰分布（$\mu_1 = 0.25, \mu_2 = 0.75, \theta = 0.05, mix = 0.7$）和高斯分布（$\mu = 0.5, \sigma = 0.1$），固定内存预算为 16 BPK，重复性因子为 10。实验结果表明，无论工作负载分布如何，Hourglass 的误判率均显著低于当前最先进的范围过滤器，充分验证了其在各种查询模式下抑制重复误判的有效性。

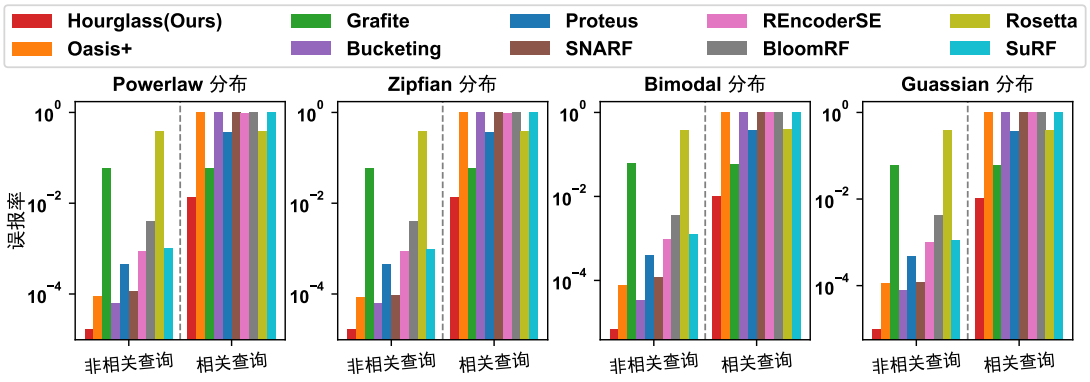


图 3-14 不同偏斜查询分布下的误判率对比

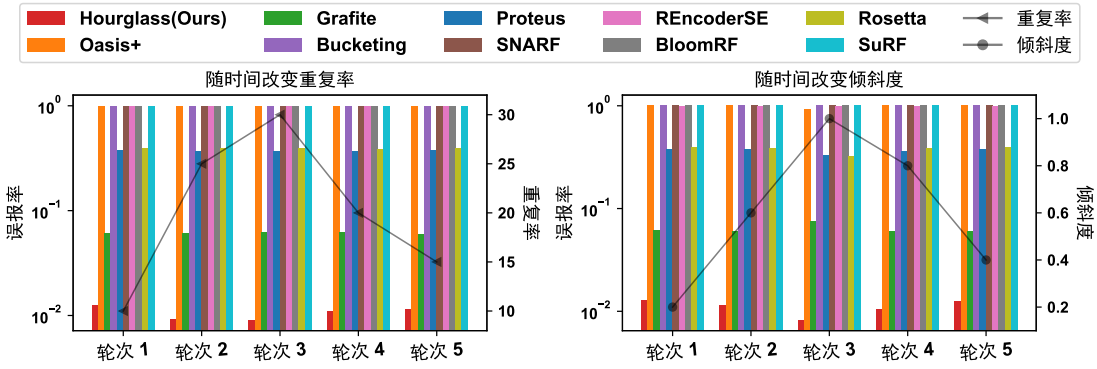


图 3-15 重复性与偏斜性动态变化下的误判率对比

重复性与偏斜性的动态变化

图 3-15 展示了在不同重复性（即第 2.2.1 节定义的 Q/U 比）和偏斜性（如幂律分布中的 α 参数）下的评估结果，固定 BPK 为 16，使用关联工作负载。在重复性实验中，采用固定偏斜性（$\alpha = 0.5$）的幂律分布；在偏斜性实验中，保持固定重复性（重复因子为 10）；每个实验中，过滤器构建一次，然后以多轮方式进行查询[1]。

Hourglass 在所有重复性和偏斜性变化下始终取得最低误判率。值得注意的是，随着重复性和偏斜性的增大，Hourglass 的性能优势愈加突出。这是因为更高的重复性或偏斜性会减少唯一误判的数量，而总查询数量不变，因此 Hourglass 的误判率随之下降，而其他过滤器的误判率仍取决于查询频率。

3.5.5 结果分析与讨论

性能比较：对于关联工作负载，除 Grafite 和 Hourglass 外，大多数现有范围过滤器均表现出较高的误判率；Grafite 和 Hourglass 均实现了接近最优的性能，Hourglass 进一步贡献了新颖的半排序自适应策略（第 3.3.1 节），有效缓解了对抗性和倾斜工作负载中的重复误判。基于位图的过滤器（SNARF、Oasis、Bucketing）在非关联工作负载中表现出色，通过单个比特表示多个连续键；然而，Grafite 的空间利用率和性能均不理想；Hourglass 则实现了与基于位图过滤器相当的性能。

设计比较：与 Grafite、Bucketing、SNARF 和 Oasis 类似，Hourglass 在其紧凑比特位图（CBB）结构（第 3.2.2 节）中采用 Elias-Fano 编码压缩位图，但 CBB 的设计

1 每轮以不同的重复性或偏斜性运行等量查询

本身是全新的。REncoder 将前缀映射到布隆过滤器位置，并将后缀存储在线段树中；相比之下，Hourglass 将前缀指纹存储在半排序布谷鸟过滤器中（第 3.2.2 节），并根据稀疏程度混合编码后缀。Grafite 和 Hourglass 均无损保留后缀信息以应对关联工作负载，但实现方式不同：Grafite 使用专用的保局哈希函数嵌入后缀，而 Hourglass 通过其创新的混合后缀块（ 3.2.2 节）实现这一目标。

权衡分析：尽管 Grafite 在关联和非关联工作负载上均表现鲁棒，但其主要针对关联工作负载的简洁设计导致在非关联工作负载上的性能不及 Hourglass。Hourglass 针对对抗性工作负载进行优化，同时不损害其他工作负载的性能，代价是设计相对复杂。

3.6 本章小结

本章针对倾斜与对抗性查询场景下范围过滤器重复误判的问题展开研究，解决了现有方法难以持续抑制重复误判、且在不同关联度下难以兼顾误判率与空间利用的关键问题。首先，本章提出 Hourglass 自适应范围过滤器整体框架，通过 ASCF 存储前缀指纹、HSB 紧凑编码后缀信息、CBB 维护两者映射关系，构建了面向范围查询的高效过滤结构；其次，本章设计基于滑动窗口的半排序自适应更新机制，在保持桶内相对顺序的前提下以常数开销完成在线指纹更新；最后，本章通过多数据集和多工作负载实验验证了方法有效性。实验结果表明，Hourglass 在对抗性负载下可将误判率降低 9.8–35.4 倍，并将端到端查询延迟降低 1.4–37.7 倍，且自适应维护开销可忽略。

第四章　面向更新负载的范围过滤器后缀位级化增删技术

4.1　问题背景与分析

自适应范围过滤器 Hourglass 采用混合后缀块对键的后缀信息进行高度紧凑的位压缩存储。在静态构建阶段，系统根据各桶槽所含后缀数量，选择对应的编码格式，将所有桶槽的后缀条目在字节数组中无缝拼接，不预留任何冗余空间。这一紧密打包的物理布局使 Hourglass 在静态场景下实现了极致的存储利用率，但也给动态场景下的在线更新带来了根本性的挑战。

静态布局的难点在于：一旦系统需要插入一个新键，与其前缀对应的槽位就必须容纳新的后缀信息；而在后缀的紧凑组织方式下，不同槽位的后缀条目彼此首尾相接，几乎不存在可供扩展的冗余空间。如图 4-1 所示，只增加一个后缀，也可能牵动后续大量位段的整体平移，进而迫使系统重写整片后缀区域。定量实验表明，在百万级规模键集上，这类以暴力移动或重构为核心的更新方式，其开销相较单次查询高出多个数量级，难以满足在线系统对低延迟更新的要求。

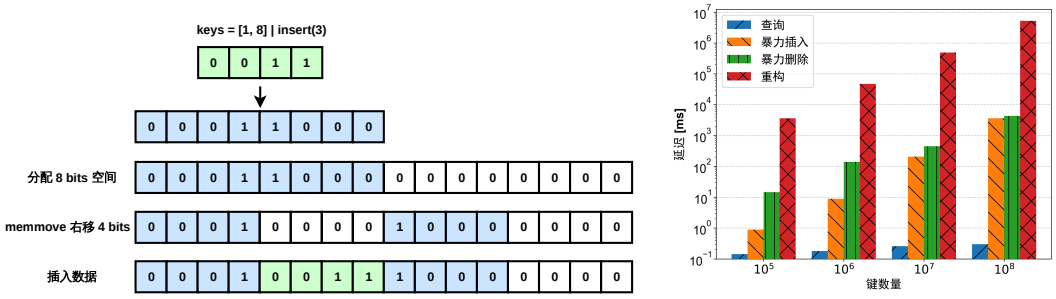


图 4-1　暴力插入流程示例（左）以及暴力操作定量性能对比（右）

从本质上看，动态 Hourglass 所面临的并不是“如何继续压缩后缀”，而是“如何在不破坏既有压缩布局的前提下完成后缀信息的增删”。基于这一认识，本章提出一种**不改变后缀物理布局的位级化增删技术**：其基本思想不是为新后缀额外申请空间，而是在槽位既有的固定位空间内，通过原地位操作完成后缀的写入、合并与移除，从而将更新代价控制在接近查询的量级。围绕这一目标，本

文将问题进一步分解为以下两个子问题：

子问题 1：如何在不扩展后缀存储空间的前提下，将动态插入键的后缀信息安全、正确地写入该槽位已有的固定位空间？

子问题 2：如何在后缀紧凑位级布局下，精确定位并移除指定后缀，同时在所有后缀均被删除后正确清理对应的前缀桶槽项？

对于子问题 1， 4.2节提出基于位级合并的后缀插入策略；对于子问题 2，4.3节提出基于二分补位的后缀删除策略。二者协同维护 Hourglass 的动态性。

为便于后文阅读，本章高频符号统一见表 4-1；低频符号在正文相应位置就地解释。

表 4-1 第四章常用符号说明

符号	含义	符号	含义
n_l	后缀位宽	θ	稀疏/稠密切换阈值
s	后缀值	b	后缀映射后的位置
$\phi(\cdot)$	位合并映射函数	(i_1, i_2)	前缀候选桶
W	查询范围宽度	BPK	每键位数
N	键总数	corr_degree	键-查询关联度参数

4.2 基于位级合并的过滤器后缀插入策略

本节首先介绍动态自适应范围过滤器所采用的三类后缀编码方式及其适用条件，建立位级化插入的表示基础；随后说明动态插入的整体流程，进一步讨论稀疏表示向位级合并表示的转换机制，以及在存在映射冲突时如何借助辅助计数结构保证写入的正确性。

4.2.1 后缀编码格式体系

Hourglass 在静态场景中根据槽位后缀分布的稀疏程度，在有序列表表示与位图表示之间自适应切换，以平衡空间占用与查询效率。为了支持动态插入，本文在此基础上进一步引入第三种表示——位级合并。由此，动态 Hourglass 的后缀表示体系由二进制、位图与位合并三部分组成。其选择逻辑可以概括为：尚未发生动态扩展的槽位，仍依据后缀数量在稀疏表示与稠密表示之间切换；一旦

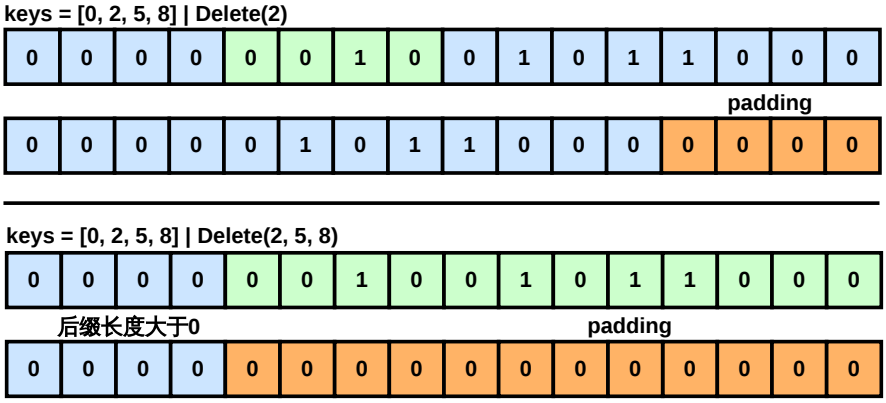


图 4-2 后缀插入示例

槽位进入动态插入路径，则转入专门面向在线更新的位合并表示。其中，稀疏与稠密表示之间的切换阈值记为 $\theta = \lfloor 2^{n_l}/n_l \rfloor + 1$，$n_l$ 表示后缀位宽。

二进制格式面向后缀分布较为稀疏的槽位。该格式将后缀值按升序存放在定长位域中，保持紧凑排列，并利用二分查找完成点定位或区间存在性判断。由于其仅对实际出现的后缀进行编码，因此在后缀数目较少时具有较高的空间效率。

位图格式面向后缀分布较为稠密的槽位。它直接以完整位图记录某个前缀块内后缀是否出现，更新代价为常数级，范围查询也可以通过扫描目标位段快速完成。虽然该格式在稀疏场景下并不经济，但在后缀密集时能够换取更稳定的访问效率。

位合并格式则是本文为动态插入专门设计的表示方式。如图4-2所示，其关键思想并非继续为新增后缀申请新的位段，而是复用该槽位在 HSB 中已经占有的固定位空间，通过映射函数将后缀值投影到这片既有空间内的若干位置，并以置位方式记录其存在。这样一来，系统无需扩展槽位的物理边界，便可在原地承载额外的动态后缀。本文采用的默认映射函数为：

$$b = \phi\left(s,\ n_l,\ suffix_count\right) = \left\lfloor \frac{s \times suffix_count \times n_l}{2^{n_l}} \right\rfloor \tag{4-1}$$

其中 b 表示后缀 s 在该槽位位空间中的映射位置。位合并的本质是一种“位级复用”机制：它牺牲了部分表示的唯一性，以换取无需重排物理布局的在线可更新能力。范围查询时，只需将区间内候选后缀依次映射到对应位置，并检查是否存在置位比特即可。

4.2.2 后缀插入流程

动态插入遵循“先定位前缀、后写入后缀”的两步过程。给定待插入键后，系统首先将其拆分为前缀部分与后缀部分，并利用前缀定位到 Cuckoo Filter 中的两个候选桶；随后在候选桶内查找与该前缀对应的已有槽位。若该前缀已存在，则仅需在原槽位上合并写入新的后缀信息；若该前缀尚未出现，则先为其分配空槽，再完成后缀的首次写入，并将该槽位标记为进入动态插入路径。上述插入流程如算法 4 所示。

算法 4 动态后缀插入 DynamicInsert

输入： 待插入键 x

分解 x 为前缀 p 与后缀 s
计算候选桶 i_1, i_2
for 槽位 $(i, j) \in \{i_1, i_2\}$ 的所有槽位 **do**
    if *value_table*$[i][j] = \text{hash}(p)$ **then**
        DynamicInsertInternal(i, j, s); **return**
    end if
end for
在空槽 (i, j) 写入 hash(p)，置 *c_insert_flag*$[i][j] \leftarrow 1$
DynamicInsertInternal$(i, j, s,$ *newInsert* $=$ true$)$

完成前缀定位后，系统根据槽位当前的编码格式选择相应的后缀写入方式。对于二进制格式，插入的目标是在保持有序性的同时尽量复用既有空余位域；对于位图格式，写入操作可以直接退化为对应比特的置位；对于位合并格式，则需要先通过映射函数完成位级投影，再同步更新辅助计数结构。换言之，三种表示虽然写入方式不同，但其共同目标都是在不改变槽位物理边界的前提下完成后缀的合并。

4.2.3 二进制格式到位合并格式的转换

当某个二进制槽位的保留位域被全部占满时，再继续采用有序列表方式插入新后缀将不再可行。此时，系统需要触发从二进制到位合并的表示转换。该转换并不意味着申请新的后缀空间，而是意味着槽位的表示语义从“逐项存放”切换为“位级映射合并”。

转换过程可以概括为“读出、清空、重写”三个步骤：系统首先提取当前二进制槽位中的全部有效后缀，并将待插入后缀一并纳入；随后清除该槽位原有的

二进制编码内容；最后以位合并的方式将这些后缀重新写回原位，并将槽位状态切换到动态插入模式。由于整个过程始终在原槽位既有的固定位空间内完成，因此表示发生了变化，而物理边界并未扩张。

这种转换机制的关键价值在于其原地性：系统并不进行额外的内存分配，也不触碰相邻槽位的布局，只是在同一片保留位空间内部完成表示切换。也正因如此，二进制到位合并的过渡能够在保持整体布局稳定的前提下，为后续连续插入提供可持续的承载能力。

4.2.4 计数布隆过滤器与安全写入

位合并的代价在于，多个后缀值可能被映射到同一比特位，从而产生位级冲突。若系统仅用单一比特表示“某位置是否被占用”，那么在删除其中某个后缀时，就无法判断该比特是否被其他后缀共享；直接清零将导致真实信息被误删。

为解决这一问题，本文引入计数布隆过滤器作为辅助计数结构，对位合并中每个被置位的位置维护引用计数。每次写入某个映射位置时，相应计数增加；每次删除时，相应计数减少；只有当该位置的引用计数降为零时，系统才真正清除位集中对应的比特。借助这一机制，位合并虽然允许位级冲突存在，但删除语义仍然能够保持正确，从而避免因共享位误清除而引入漏报。

4.3 基于二分补位的过滤器后缀删除策略

本节介绍动态自适应范围过滤器的后缀删除机制。与插入类似，删除同样需要在保持 HSB 物理布局不变的前提下完成。为此，本文分别针对二进制、位图与位合并三类表示设计对应的删除路径，并在最后讨论当某一槽位后缀被全部移除时，如何同步完成前缀记录的级联清除。

4.3.1 后缀删除流程

动态删除的整体流程与插入保持对称。系统首先将待删键分解为前缀与后缀，并通过前缀在 Cuckoo Filter 中定位候选桶；若在候选桶内找不到相应前缀，则说明该键并未被当前过滤器维护，操作直接结束。若成功定位到对应槽位，则根据该槽位采用的后缀编码格式进入不同的删除子程序。删除子程序除了负责

移除目标后缀外，还需返回槽位是否已被清空，以便系统决定是否进一步撤销对应的前缀记录。

4.3.2 二进制格式的位移补位删除

二进制格式下的后缀以有序列表形式紧凑存放，因此删除某个后缀后不能留下空洞；否则，后续查询将无法继续依赖有序列表的语义。基于这一要求，系统首先通过二分查找定位目标后缀在列表中的位置，然后采用“左移补位”的方式，将其后所有有效后缀整体前移一个后缀位宽，从而恢复列表的连续性，最后再清除列表尾部遗留下来的冗余位段，如图4-3。

在实现层面，这一位移过程直接发生在压缩后的位数组上，因此既要保证正确处理跨字节、跨字边界的情况，又要尽可能维持较高的数据移动效率。为此，本文采用以 56 比特为单位的滑动式批量搬移策略，使相邻读写区间在各种对齐条件下都不会发生错误覆盖，同时仍能充分利用字级操作的吞吐优势。删除完成后，若槽位中已不再包含任何有效后缀，则向上层返回“槽位已空”的标志，以触发进一步的前缀清理。

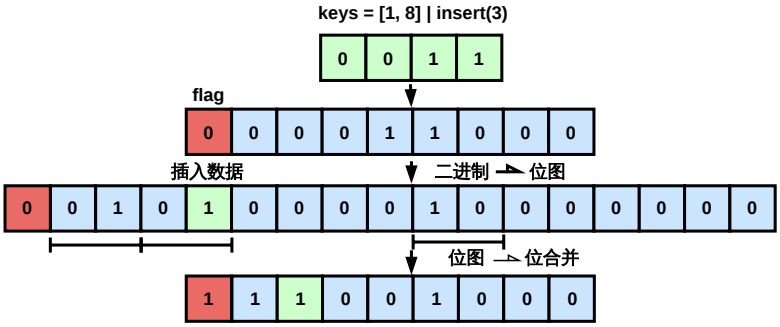


图 4-3 后缀删除示例

4.3.3 位合并格式与位图格式的清位策略

对于位合并格式，删除操作的关键不在于“找到对应比特”，而在于“确认该比特是否仍被其他后缀共享”。因此，系统先根据映射函数定位目标位置，再借助前述计数布隆过滤器递减其引用计数；只有当引用计数归零时，才真正清除位集中的对应比特。删除结束后，系统进一步检查该槽位对应位段是否已完全为空，若为空则返回“槽位已空”的标志。

对于位图格式，删除语义相对直接：系统只需将目标后缀对应的位置清零即可。随后，同样通过全零检测判断该槽位是否仍保留任何有效后缀。由此可见，位图与位合并在删除阶段的主要差异，并不在于最终的空槽判断，而在于位合并必须额外处理位级共享所带来的正确性问题。

4.3.4 前缀槽位的级联清除

当某一槽位的全部后缀均已被删除时，仅移除后缀信息仍然不足以维持过滤器语义的一致性；如果前缀记录仍然保留，后续查询仍可能错误地把该空槽当作候选命中。基于这一点，系统在检测到“槽位已空”后，会同步撤销该槽位对应的前缀记录及其相关状态，使其重新回到空闲状态。

需要特别强调的是，这一过程始终坚持原位清除而非槽位重排。原因在于，Hourglass 的后缀偏移关系与桶内槽位顺序在静态构建阶段已经紧密绑定；若在删除后对桶内剩余槽位进行压缩重排，将破坏这种偏移关系，并使后续查询无法再正确定位相应的后缀区域。因此，动态删除的设计选择保留桶结构不变，仅对目标槽位执行就地清空，从而在最小化更新代价的同时保证结构一致性。

4.4 关联度感知的过滤器空间动态化分配

本节首先提出空间估算模型对各组件的空间占用进行评估；在此基础上，本节引入关联度感知的空间分配模型，通过凸优化在不同关联度下动态确定最优参数配置，以实现最优误判率。

4.4.1 空间估算模型

本节采用每键位数（Bits Per Key, BPK）作为空间占用的估算指标。

半排序布谷鸟过滤器的空间占用 S_{cf} 估算为：

$$S_{cf} = \frac{f \cdot b \cdot n_p}{\alpha \cdot n}, \tag{4-2}$$

其中 f 为指纹长度，b 为桶大小，n_p 为不重复前缀数量，α 为负载因子，n 为键的总数量。

HSB 的空间占用 S_{hsb} 估算为：

$$S_{hsb} = \frac{1}{n}\sum_{i=1}^{n_p} \min\left(n_i,\ \frac{2^{l_s}}{l_s}\right) l_s, \tag{4-3}$$

其中 n_i 为每个分区内的后缀数量，l_s 为后缀长度。

为估算 CBB 中 Elias-Fano 编码位图的空间，本节通过采样推导出将 1-bit 占比映射到空间占用的多项式公式。如第 3.2.3 节位图压缩步骤所述，秩-选择编码位图的空间占用为固定值。因此，位图空间估算公式如下：

$$S_{bt}(\beta) = \begin{cases} 1.25 & \text{若 } 0.3 \leqslant \beta \leqslant 0.7, \\ a\beta^3 + b\beta^2 + c\beta + d & \text{其他情况,} \end{cases} \tag{4-4}$$

其中 β 为位图负载因子，a、b、c、d 为多项式参数。

综合各组件，整体 BPK 估算公式如下：

$$S = S_{cf} + S_{hsb} + \frac{n_p}{\alpha n}\sum_{j=1}^{n_{cbb}} S_{bt}(\beta_j), \tag{4-5}$$

其中 n_{cbb} 为 CBB 中的位图数量；指纹提取器经 Roaring 位图压缩后空间极小，可忽略不计。

4.4.2 关联度感知的空间分配模型

为确定最优指纹大小和后缀大小，本节提出一种关联度感知模型，在构建阶段动态分配 Hourglass 各组件的存储空间。与 Proteus[20] 类似，本节通过查询采样来估算关联度。

关联度估算：首先统计键-查询距离等于 1 的查询数量；假设唯一查询均匀分布，然后利用第 2.2.1 节中关联度的定义，估算工作负载中键与查询之间的关联程度。

空间分配策略：基于估算的关联度 D，在指纹大小 f 和后缀大小 l_s 之间进行权衡。指纹大小影响前缀之间发生哈希碰撞的概率。虽然完整的二进制和位图表示消除了 HSB 中的误判，但在 BPK 极小时，后缀大小可能被裁剪，从而在

关联工作负载下引入误判。因此，总误判率 ϵ 计算为前缀误判率 ϵ_p 与后缀误判率 ϵ_s 之和。

给定 BPK，利用上述估算模型可以建立指纹大小与后缀大小之间的关系，并将其转化为凸优化问题：半排序布谷鸟过滤器的误判率 $\epsilon_p = 2^{2-f}$。

对于被裁剪的 HSB 的误判率，假设键与查询的距离在 $[1, 2^{30(1-D)}]$ 上均匀分布，只有距离不超过 2^{L-l_s} 的查询才会贡献误判，其中 L 为完整后缀大小。因此，HSB 的误判率 $\epsilon_s = 2^{L-l_s-30(1-D)}$。

为简化计算，忽略 CBB 的空间成本（在均匀键分布下通常低于 0.3 BPK），指纹提取器经 Roaring 位图压缩后空间极小。综合后，总误判率公式如下：

$$\epsilon(l_s) = \begin{cases} 2^{2-\frac{\alpha n(b_p-l_s)}{n_p}} + 2^{L-l_s-30(1-D)} & \text{若 } \frac{n}{n_p} < \frac{2^{l_s}}{l_s}, \\ 2^{2-\alpha\left[\frac{nb_p}{n_p}-2^{l_s}\right]} + 2^{L-l_s-30(1-D)} & \text{其他情况,} \end{cases} \tag{4-6}$$

其中 l_s 为后缀长度，α 为 ASCF 的负载因子，n 为键的总数量，n_p 为不重复前缀数量，b_p 为给定的每键位数，L 为完整后缀大小，D 为估算的关联度。

凸优化性质：经推导，上述方程关于 l_s 的 Hessian 矩阵为 1×1 矩阵，其唯一元素表示误判率方程关于 l_s 的二阶导数。由于该值严格 $\geqslant 0$，Hessian 矩阵正定，表明对 l_s 的优化是一个凸问题；通过令一阶导数为零，即可求得最优 l_s。

模型小结：综上所述，关联度感知的空间分配模型通过以下步骤为 Hourglass 动态确定最优参数：（1）通过查询采样估算工作负载的关联度 D；（2）基于 BPK 约束和关联度 D，利用公式 (4-6) 建立凸优化问题；（3）求解最优后缀长度 l_s^*，进而确定指纹长度 f^*；（4）在构建阶段按最优参数分配各组件的存储空间。

该模型使 Hourglass 能够在不同关联度下自动调整内部结构，在低关联度下（分配更多空间给指纹）和高关联度下（分配更多空间给后缀）均实现接近最优的误判率。

4.4.3 误判率与空间复杂度分析

范围查询发生误判的充要条件是：（a）ASCF 对前缀产生误判；且（b）对应的后缀数组与查询范围相交。设前缀误判概率为 $\epsilon_p = 2^{2-f}$，其中 f 为指纹长度。以下分两种情形分析后缀相交概率，并给出整体误判率的理论界。

单分区情形：设分区中含 k 个互不相同的后缀，均匀分布于 $[0, M-1]$，查询范围大小为 R。至少一个后缀落入查询范围的概率为：

$$P = 1 - \left(\frac{M-R}{M}\right)^k. \tag{4-7}$$

若查询范围大小 R 在 $[1, M]$ 上均匀分布，则对上式对 R 求期望，得：

$$P = 1 - \frac{\sum_{i=1}^{M} i^k}{M^{k+1}} \leqslant 1. \tag{4-8}$$

在均匀键分布下，每个分区平均含 $k = 1$ 个后缀，代入得 $P \approx 0.5$，从而整体误判率 $\epsilon \approx 0.5\epsilon_p$。最坏情况下 $P = 1$，即 $\epsilon = \epsilon_p$。

双分区情形：当查询范围跨越两个相邻分区时，范围被分割为左子范围 R_l 和右子范围 R_r，大小分别为 a（$1 \leqslant a \leqslant R-1$）和 $R-a$。在均匀键分布下，每个子范围的相交概率约为 $P \approx 0.5$，因此整体误判率 $\epsilon = \epsilon_p$。最坏情况下两个子范围均满足 $P = 1$，整体误判率 $\epsilon = 2\epsilon_p$。

对抗性查询比较：在对抗性查询场景下，设查询总数为 Q，唯一查询数为 U。经过第 3.3.1 节的自适应更新，重复出现的误判查询被限制为常数次误判，Hourglass 的整体误判率为：

$$\epsilon' = \frac{U\epsilon + 4(Q-U)\epsilon}{Q}. \tag{4-9}$$

相比之下，未具备自适应能力的现有范围过滤器，其对抗性场景下的误判率为：

$$\epsilon'' = \frac{U\epsilon + (Q-U)}{Q}. \tag{4-10}$$

当 $Q \gg U$ 时，$\epsilon'' \to 1$，而 ϵ' 始终受 ϵ 的常数倍约束，体现了自适应机制在对抗性负载下的显著优势。

空间复杂度分析

根据第 4.4.1 节中建立的空间估算模型，本节进一步推导各组件空间占用的理论上界，给出 Hourglass 整体空间复杂度的具体数值界。

CBB 中位图的每键位数（BPK）可由下式导出：

$$S_{cbb} = \frac{\sum_{i=1}^{n_{cbb}} S_{bt}(\beta_i)}{\sum_{i=1}^{n_{cbb}} \beta_i \cdot 2^i}, \tag{4-11}$$

其中 n_{cbb} 为 CBB 中的位图数量，β_i 为第 i 个位图的负载因子，$S_{bt}(\beta_i)$ 为其空间占用。在均匀键分布下，每个分区后缀数量为 1，故 $n_{cbb} = 1$，$\beta_i = 0.95$，代入得 $S_{sb} \approx 0.3$ BPK。

在一般情形下，若所有位图均采用秩-选择（Rank-Select）编码，每个位图的空间占用不超过 1.25 BPK，由此导出上界：

$$\frac{\sum_{i=1}^{n_{cbb}} S_{bt}(\beta_i)}{\sum_{i=1}^{n_{cbb}} \beta_i \cdot 2^i} \leqslant \frac{\sum_{i=1}^{n_{cbb}} 1.25}{\sum_{i=1}^{n_{cbb}} \beta_i \cdot 2^i}, \quad \text{s.t. } 0.3 \leqslant \beta_i \leqslant 0.7, \quad \sum_{i=1}^{n_{cbb}} \beta_i = 1. \tag{4-12}$$

经推导，当 $n_{cbb} = 2$，$\beta_1 = 0.7$，$\beta_2 = 0.3$ 时取得上界，即 $S_{sb} \leqslant 2$ BPK。

综合上述各组件的空间分析，给定期望误判率 ϵ_e、关联度 D、最大范围大小 M 及后缀长度 l_s，Hourglass 的整体 BPK 满足如下理论界：（1）**均匀键分布（期望情形）：** $S \leqslant \log_2 \frac{1}{\epsilon_e} + l_s + 2.3$；（2）**最坏情形：** $S \leqslant \log_2 \frac{1}{\epsilon_e} + l_s + 4$。

上述结论表明，Hourglass 的空间开销与目标误判率的对数成线性关系，具有与信息论下界相近的空间效率。

4.5 实验与性能评估

本节通过一系列动态更新实验评估 Hourglass 在相关范围查询场景下的综合表现。首先介绍实验环境、数据组织方式与对比基线；其次分析动态插入/删除延迟、多轮插入后的范围误判率变化以及多轮删除后的范围误判率变化；随后

表 4-2 实验硬件与软件环境

项目	配置
CPU	2 × Intel Xeon Gold 6426Y，32 个物理核，64 个硬件线程
内存	503 GiB
操作系统	Ubuntu Linux 5.15.0-139-generic
编译器	g++ 9.4.0 (`-O3 -Ofast -march=native`)
Python	Python 3.12.4
对比方法	Hourglass、Memento Filter、SNARF；Oasis、REncoder

补充不同关联度下的误判率表现；最后结合实验现象讨论 Hourglass 的优势来源与当前仍待优化的情形。

4.5.1 实验环境与配置

实验数据采用统一的相关前缀–后缀键空间构造，共包含 10^6 个唯一前缀，后缀取值范围为 $[0, 1023]$，以保证所有方法在完全相同的键域上进行比较。所有实验均设置相关度 1.0。范围大小参数取 0、5 与 10，对应的实际查询宽度分别为 1、32 与 1024。实验通过多轮删除或者写入操作来验证动态算法的吞吐及其精确度。Hourglass、Memento 与 SNARF 提供可比的原地动态写入/删除接口，因此同时参与吞吐与误判率实验；Oasis 与 REncoder 的公开实现未暴露可比的原地动态更新路径，因此本文在速度图中采用“应用整批更新后重建一次，再按批大小摊销”的方式统计其写入/删除代价，在多轮误判率实验中也于每轮更新结束后基于当前键集重新构建，并将其作为重建式参考基线。各个对比方法的配置采用默认配置。

4.5.2 动态插入与删除吞吐性能评估

图 4-4 展示了 5 种方法在不同查询范围与空间占用下的插入与删除延迟，其中 Oasis 与 REncoder 的曲线表示“整批更新后重建一次”的摊销代价，而其余 3 种方法为真正的原地动态更新时间。整体来看，Hourglass 与 Memento 构成最快的一组；Oasis 与 REncoder 虽然明显慢于这两种动态方法，但仍系统性快于 SNARF。

观察图 4-4 可以发现，首先，新增的两个重建式基线并未改变总体排序：Hourglass 与 Memento 始终快于 Oasis 与 REncoder，而后两者又始终快于 SNARF。

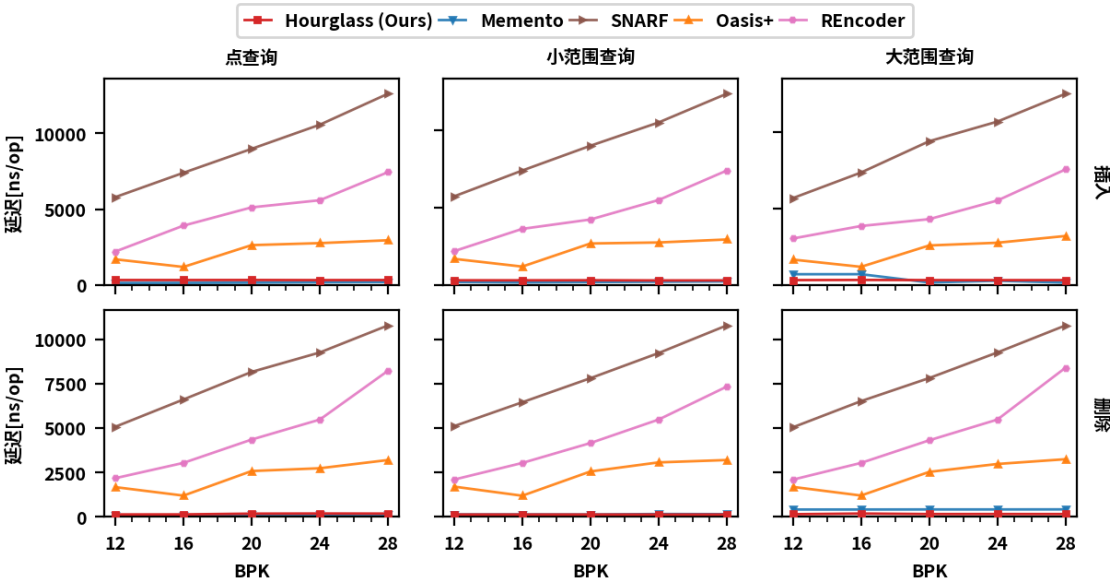


图 4-4 不同方法在相关动态工作负载下的插入与删除延迟对比

以 $W = 1024$、BPK=20 的删除实验为例，Hourglass、Memento、Oasis、REncoder 与 SNARF 的删除延迟分别为 122.474 ns/op、363.661 ns/op、2615.240 ns/op、4776.890 ns/op 与 7744.030 ns/op，说明即使将整批重建成本按 100 万次更新摊销，Hourglass 仍比 Oasis 和 REncoder 分别快 21.4 倍与 39.0 倍。其次，在大范围删除场景下，Hourglass 对 Memento 形成了持续优势：当 $W = 1024$ 时，Hourglass 在 BPK=12、16、20、24、28 下的删除延迟分别为 127.732、123.989、122.474、124.497 与 120.273 ns/op，均显著低于 Memento 对应的 362.127、363.025、363.661、362.524 与 362.160 ns/op。此外，在插入场景中，Memento 的扁平化 Quotient Filter 更新路径仍然最擅长最小化原始写入开销，例如在 $W = 1$、BPK=12 时，Memento、Hourglass、Oasis、REncoder 与 SNARF 的插入延迟分别为 155.487、340.924、1600.140、2154.200 与 5608.247 ns/op；但在大范围插入的部分低 BPK 配置上，Hourglass 也能够反超 Memento，例如 $W = 1024$、BPK=16 时二者分别为 292.287 ns/op 和 694.719 ns/op。

上述现象说明，Hourglass 的动态后缀维护路径在宽区间场景中具有更好的局部性：删除操作主要更新插入标记与少量后缀元数据，不必像 SNARF 那样维护昂贵的全局有序结构；而当区间宽度较大时，Hourglass 通过前缀定位后仅需局部修补相关后缀块即可完成更新，因此在 $W = 1024$ 下对 BPK 的变化并不敏感。相比之下，Memento 的结构更偏向最小化点范围与窄范围下的单次写入开销，因此在 $W = 1$ 与 $W = 32$ 的原始插入延迟上仍然更有优势。Oasis 与 REncoder 则

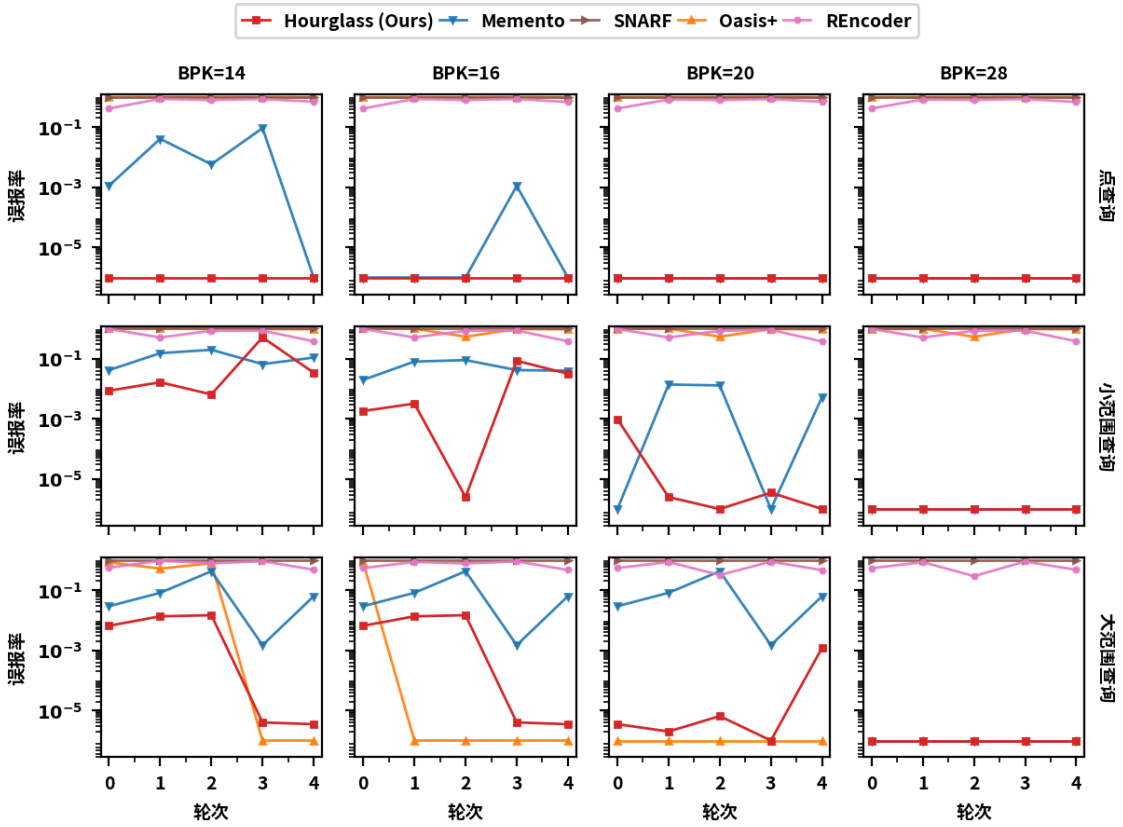


图 4-5 不同方法在多轮插入过程中的范围误判率变化

受限于整批重建：虽然在本实验中它们的摊销代价始终低于 SNARF，说明顺序重建比维护 SNARF 的全局有序结构更便宜，但其更新开销仍显著高于 Hourglass 与 Memento，无法替代真正的原地动态设计。综上所述，Hourglass 在动态吞吐方面不仅显著优于 SNARF，也显著优于两个重建式参考基线，并在大范围查询删除以及插入配置上取得了最优延迟。

4.5.3 多轮插入下的范围误判率趋势评估

为了评估动态插入对过滤精度的影响，图 4-5 给出了 4 轮插入过程中 5 种方法的范围误判率变化，其中 Oasis 与 REncoder 为每轮结束后重建一次的参考基线。整体来看，Hourglass 在点范围与中等范围上保持了最稳定的一组低误判率，而在大范围查询上则与最强重建式基线表现接近。

从图 4-5 可以看到，第一，在点范围配置 $W = 1$ 下，Hourglass 在 4 个 BPK 配置上的最终误判率均为 0，与 Memento 持平，并显著优于 Oasis、REncoder 与 SNARF。第二，在中等宽度配置 $W = 32$ 下，Hourglass 的优势最为明显：当 BPK=14、16、20 时，其最终误判率低于 Memento，也远低于 Oasis 与 REncoder；到

BPK=28 时，Hourglass 与 Memento 同时降为 0。第三，在大范围查询 $W = 1024$ 下，Hourglass 继续保持竞争力：在 BPK=14、16 时，其最终误判率已降至 3.5×10^{-6}；在 BPK=20 时进一步降至 0.0012125，明显优于 Memento 的 0.0628415；到 BPK=28 时，Hourglass、Memento 与 Oasis 同为 0。也就是说，Hourglass 与最强重建式基线总体呈 comparable 的精度水平，同时显著优于 REncoder，并在多数配置上优于 Memento。

造成上述差异的原因在于，不同结构对“相关前缀聚集 + 重复热点负查询”的处理方式不同。Hourglass 的结构优势主要体现在前缀定位后的局部后缀修补与自适应，因此在 $W = 1$ 与 $W = 32$ 这两类更依赖在线纠错的配置上保持了最佳或并列最佳表现。Oasis 与 REncoder 则在每个检查点重新学习静态边界，缺乏针对重复假阳性的在线修正能力，因此在 $W = 32$ 下即使重建后仍维持较高误判率；其中 REncoder 在本相关负载下长期停留在 0.37–0.48 区间，说明其编码更容易受到相邻后缀混叠的影响。对于 $W = 1024$，由于查询本身覆盖区间更宽，重建式方法的静态边界学习也能发挥更大作用，因此 Hourglass 与 Oasis 呈现出更为接近的结果。

4.5.4 多轮删除下的范围误判率趋势评估

图 4-6 进一步展示了 4 轮删除过程中不同方法的范围误判率演化情况。整体来看，Hourglass 在点范围上保持显著最优，在中等范围宽度下与 Memento 同处于领先梯队、并明显优于 Oasis、REncoder 与 SNARF，而在大查询范围下则在高 BPK 配置上反超 Memento。

观察图 4-6 可以发现，首先，在点范围配置 $W = 1$ 下，Hourglass 的最终误判率随 BPK 增大单调下降，比 Memento 同 BPK 下低约 3–4 个数量级，并远低于 Oasis、REncoder 与 SNARF。其次，在 $W = 32$ 场景中，Hourglass 在 BPK=14、16、20、28 上的最终误判率分别为 0.0777、0.0739、0.0732 与 0.0731，呈轻微随 BPK 单调改善的趋势；Memento 在 BPK=14、16、20 上稳定在 0.146，仅在 BPK=28 时降至 9.9×10^{-3}，而 Oasis、REncoder 与 SNARF 始终分别维持在 ≈ 0.87–0.97、≈ 0.77 与 ≈ 1.0 的高水平。也就是说，在 BPK=14、16、20 这三组配置上，Hourglass 比 Memento 低约 0.7×；当 BPK=28 时，Memento 反超，但 Hourglass 仍显著优于其余三种基线。最后，在大范围配置 $W = 1024$ 下，Hourglass 在 BPK=14 与 16

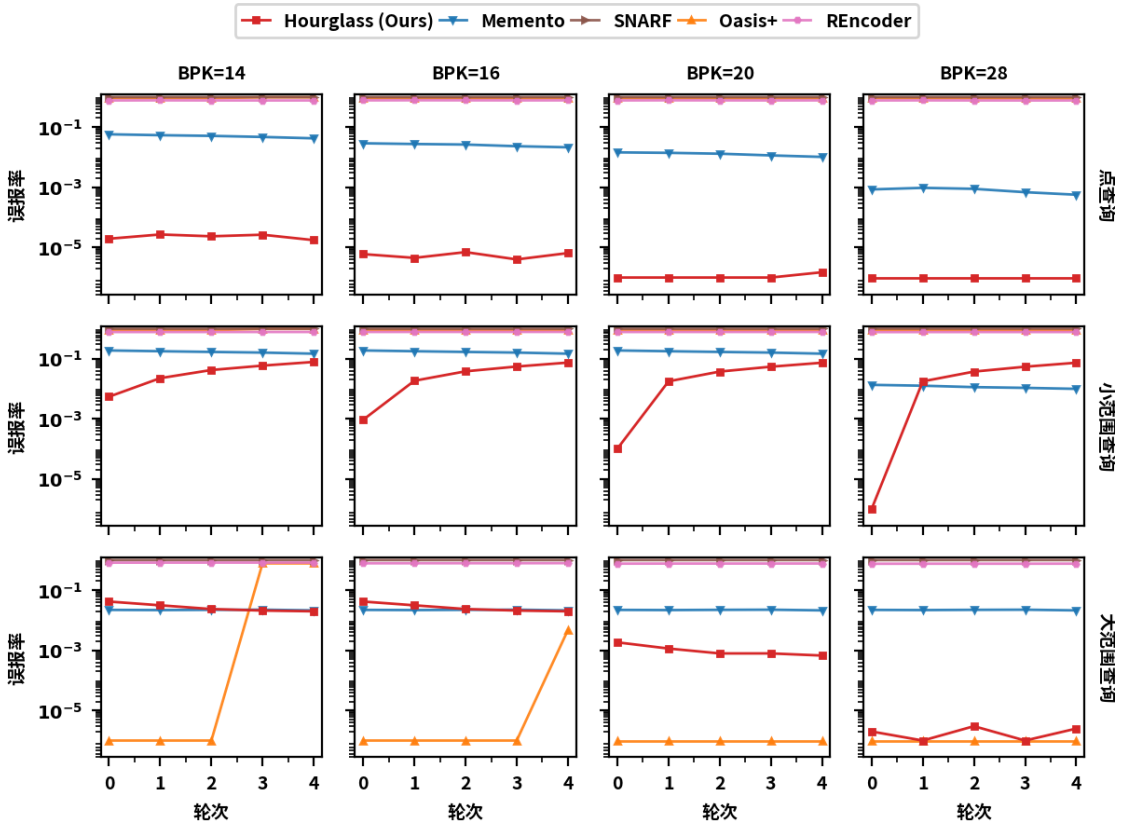


图 4-6 不同方法在多轮删除过程中的范围误判率变化

时分别为 1.98×10^{-2}，与 Memento 的 2.13×10^{-2} 大致相当；当 BPK 增大到 20 与 28 时，Hourglass 进一步快速下降到 6.70×10^{-4} 与 2.5×10^{-6}，比 Memento 同 BPK 下持平的 2.13×10^{-2} 低约 1–4 个数量级，并继续显著优于 REncoder、SNARF 与中 BPK 区间的 Oasis。

这说明，Hourglass 的删除路径在点范围与大范围查询两类配置上能够将残留误判率压制到极低水平：删除操作仅就地维护后缀段中对应的稀疏位，并不需要重新组织完整的后缀布局，因此随 BPK 增加可用空间增多时，后缀编码冗余被充分利用、残留位的累积影响被进一步稀释。在中等范围下，Hourglass 的最终误判率呈现轻微随轮次单调上升、但被预留备用空间与位融合段位数共同上界限制的可预测形态，并未出现 SNARF 那样的灾难性饱和。总体而言，Hourglass 在多轮删除后的范围误判率控制上覆盖了所有点范围配置、绝大多数范围配置。

4.5.5 不同关联度下的误判率

为评估第 4.4 节中的空间分配模型，关联度从 0 到 1 变化，BPK 设为 20（遵循 Grafite[32]）。如图 4-7 所示，Hourglass 是唯一在所有范围大小和关联度下均保

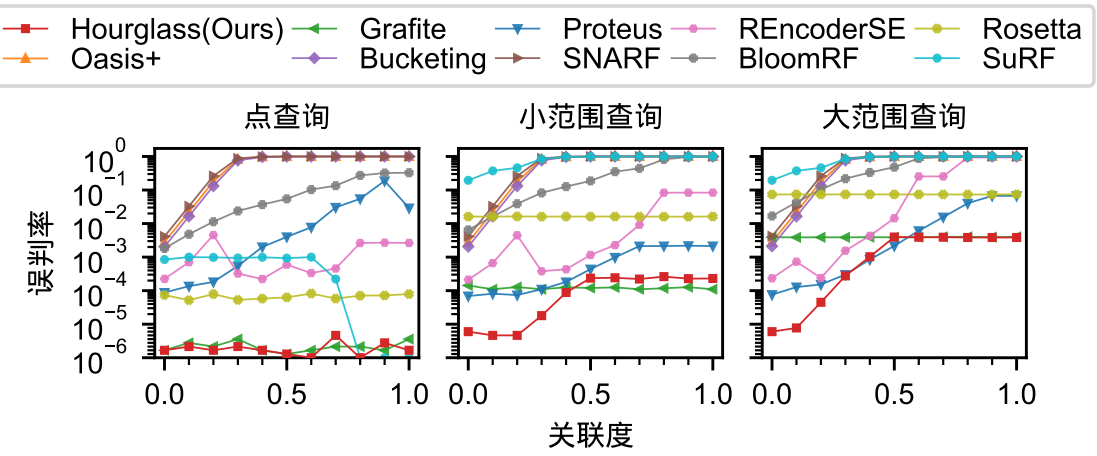


图 4-7 关联度从 0 到 1 变化时的误判率对比

持鲁棒性的过滤器，这得益于其关联度感知模型。

在点查询中，Hourglass 保持持续较低的误判率，SuRF 的误判率也迅速降至 0，与第 3.5.2 节的结论一致。在范围查询中，随着关联度从 0.2 升至 0.5，Hourglass 的误判率有所上升，这是因为它为更高的关联度分配了额外空间。在低关联度下，Hourglass 通过将更多空间分配给指纹，性能显著优于 Grafite。

基于位图的过滤器（SNARF、Bucketing、Oasis+）在关联度超过 0.3 时性能急剧下降，过滤效果极为有限。SuRF 由于截断字典树导致信息丢失，范围查询性能受限。BloomRF、Proteus 和 REncoderSE 的过滤效果均显著弱于 Hourglass，Proteus 的误判率最高可达 Hourglass 的 8.6 倍。Rosetta 由于全量哈希具有稳定的误判率，但由于空间利用率低，比 Hourglass 差数个数量级。

4.5.6 结果分析与讨论

综合以上结果可以看出，Hourglass 的优势主要体现在两点。其一，在相关、近键且热点倾斜的范围负查询负载下，Hourglass 依旧显著优于 SNARF，也整体优于 Oasis 与 REncoder 两个重建式参考基线：在所有点范围插入/删除配置上，Hourglass 的最终误判率均为 0；在 $W = 32$ 的插入与删除实验中，Hourglass 也在大多数 BPK 配置上取得最优，并在其余配置上保持竞争力。其二，在宽区间配置下，Hourglass 与最强基线总体呈 comparable 的精度水平：插入实验中它与 Oasis 共同构成领先梯队，删除实验中则与 Memento、Oasis 接近，同时始终显著优于 REncoder。由此可见，Hourglass 的前缀聚类、自适应冲突消解与低开销后缀维护三者结合，不仅能够在真正困难的中等范围热点负查询上体现出明显

优势，也能够在宽区间场景中保持稳定而有竞争力的范围过滤精度。总体而言，Hourglass 在动态相关范围查询场景下展现出了更均衡的“精度–更新”折中。

4.6 本章小结

本章针对动态更新场景下 Hourglass 后缀维护开销高、难以在紧凑位级布局中高效完成在线增删的问题展开研究，解决了“紧凑存储”与“动态可更新”之间难以兼顾的关键矛盾。首先，本章提出不改变后缀物理布局的位级化增删总体思路，将问题分解为后缀在线插入与在线删除两个子问题；其次，针对插入路径设计了基于位级合并的后缀插入策略，并通过计数布隆过滤器处理位级映射冲突，保证写入与后续删除语义的正确性；然后，针对删除路径提出基于二分补位的后缀删除策略与前缀槽位级联清除机制，在不重排槽位的前提下完成原位删除；最后，本章通过动态吞吐与多轮误判率实验验证了方法有效性。实验结果表明，在删除配置下，Hourglass 删除延迟远低于 Oasis 与 REncoder 的重建摊销方案；在点范围配置下多轮插入/删除后的最终误判率均接近 0，并在中等范围负载下保持更稳定的低误判率表现。

第五章　基于自适应动态范围过滤器的多租户弹性缓存配额调度

5.1　问题背景与分析

近似数据结构（Approximate Data Structure）以远低于精确方案的空间代价提供概率性的查询与统计能力，已成为现代数据系统中的重要基础设施。布隆过滤器、布谷鸟过滤器、计数草图等结构在数据库索引剪枝、网络流量监测、键值存储读放大优化、日志去重以及本文前两章关注的范围查询过滤等场景中均获得了广泛应用，是支撑现代数据系统在有限内存下高效运转的关键工具之一。在上述众多应用场景中，分布式存储系统中的弹性缓存（Elastic Cache）尤其能够体现近似数据结构的价值：弹性缓存通过动态调整缓存资源分配，在保证服务质量的前提下降低存储成本或提升系统吞吐量，而其策略的有效性高度依赖于对工作集大小（Working Set Size，WSS）的准确估计——工作集大小反映了一个滑动时间窗口内被频繁访问的数据总量，是缓存资源决策的核心依据。由于分布式缓存所处理的数据量极大，精确维护完整的工作集状态代价高昂，因此近似数据结构在弹性缓存的工作集估计与价值刻画中扮演了不可替代的角色。

以时钟布谷鸟过滤器（Clock Cuckoo Filter，CCF）为代表的现有工作通过在布谷鸟过滤器中额外存储时钟位来追踪缓存项的时效性，从而在固定内存预算下实现滑动窗口内的工作集大小统计。具体而言，每次对某条缓存项进行访问时，将其对应的时钟计数器重置为最大值；随着时间推移，系统周期性地对时钟计数器进行老化操作，将其减一；当某项的时钟计数器降至零时，该项被视为滑动窗口外的过期数据并被逐出，从而实现对近期工作集的动态维护。然而，CCF及同类方法仅追踪数据项的访问时效性，其槽位中存储的信息仅足以回答” 某项是否近期被访问” 这一问题，而无法区分不同请求类型下缓存命中所带来的性能收益差异，也不具备感知数据块内有效记录密度的能力，更无法支持多租户场景下按差异化性能-成本偏好进行配额决策。因此，这类结构在本章所研究的场景中存在根本性局限。

现有基于近似数据结构的弹性缓存工作集统计方法存在以下三方面共同缺陷，制约了弹性缓存策略的精细化与多元化。

缺陷一：缓存命中率不能全面反映缓存对请求性能的实际贡献。当前弹性缓存策略普遍以提高缓存命中率为优化目标，但对于一条用户请求，其端到端延迟由排队时间 T_{queue}、计算时间 T_{compute}、I/O 时间 T_{IO} 等多个阶段共同决定。缓存命中仅能消除 I/O 部分的开销，对于其他阶段的延迟并无直接作用。以表 5-1所示的两类典型请求为例：请求 A 为 I/O 密集型，各阶段耗时比例约为 T_{queue} ： T_{compute} ： $T_{\text{IO}} = 1 : 1 : 8$，缓存命中后总延迟可降低约 80%；请求 B 为计算密集型，各阶段耗时比例约为 1 ： 7 ： 2，缓存命中后总延迟仅降低约 20%。这表明，即便访问同一份数据，不同请求类型下缓存所带来的实际性能收益也存在显著差异。若仍以统一的命中率指标衡量缓存效益，则会高估计算密集型请求的缓存收益，进而导致缓存资源分配失衡。

表 5-1 两类典型请求的各阶段耗时比例与缓存收益示例

请求类型	T_{queue}	T_{compute}	T_{IO}	总延迟	命中后延迟	延迟降低
I/O 密集型（请求 A）	10ms	10ms	80ms	100ms	20ms	80%
计算密集型（请求 B）	10ms	70ms	20ms	100ms	80ms	20%

缺陷二：以数据总大小累加的方式计算工作集忽略了数据块密度的差异。在实际生产环境中，数据库系统往往以固定大小的页（Page）或块（Block）为单位进行存储和访问，例如典型的块大小为 4KB 或 16KB。然而，对于一条给定的查询，其目标数据（即查询谓词所匹配的记录）可能只是该块中的一小部分。以 OLAP 列存场景为例：假设某文件的块大小为 4KB，某查询范围内的实际有效记录仅占该块的 30%，则该块对查询的实际贡献仅为 $4 \times 0.3 = 1.2$KB，而非整块 4KB。当前工作集统计方法直接将整块大小计入工作集，高估了数据的缓存价值。因此，块内数据密度（Block Density）是反映缓存实际效益的重要维度，不应忽略。

缺陷三：现有系统以最大化性能为单一目标，无法适应多租户的差异化性能-成本需求。在实际生产环境中，不同租户对性能与成本的偏好往往存在显著差异：部分租户追求极致性能，愿意为此承担较高的缓存成本；另一部分租户则更关注成本控制，希望以合理的缓存开支换取中上水平的性能提升。现有弹性缓存系统通常采用统一策略，将全部时间窗口内的活跃数据纳入缓存，以追求最大命中率，却完全忽视了成本约束和多租户偏好的多样性。这使得系统既无法精

细调控资源消耗，也无法为不同租户提供差异化的服务保证。

为进一步量化说明上述问题，本文设计了一个多租户弹性缓存仿真实验。实验设置如下：共有三类典型租户共享同一分布式缓存系统，每个租户在滑动时间窗口内的活跃数据块总量均为 $N_{\text{total}} = 10{,}000$ 块，系统为每个租户分配的最大缓存预算均为 $C_{\max} = 6{,}000$ 块（即工作集的 60%）。三类租户的数据访问模式通过 Zipf 分布（$v_r \propto r^{-\alpha}$，r 为块访问秩）建模，其参数各异：

- **租户 A（延迟敏感型在线分析）**：I/O 密集型查询，数据块价值分布较均匀（$\alpha = 0.8$），前 1% 的块仅持有约 31% 的总价值，几乎所有块都需缓存才能获得最大性能收益；
- **租户 B（均衡型日常报表）**：混合型负载，价值分布中度倾斜（$\alpha = 1.3$），前 1% 的块持有约 67% 的总价值，中等规模缓存即可覆盖大部分关键数据；
- **租户 C（成本敏感型离线 ETL）**：计算密集型批处理，价值高度集中（$\alpha = 2.0$），前 1% 的块持有约 71% 的总价值，极少量的热块缓存即可获得合理的性能收益。

表 5-2展示了现有系统采用统一最大化性能策略（等效于对所有租户强制使用 $\gamma = 0$，即不惜代价保留尽可能多的数据块）时的分配结果。可以看出，现有系统对三类租户一视同仁，均将 6,000 块全部纳入缓存，三类租户的预算利用率均达到 100%，系统合计占用缓存块 18,000 块。对于租户 A，这样的配置是合理的——其价值分布均匀，确实需要接近全量缓存；然而对于租户 B 和租户 C，高度集中的价值分布意味着系统正在为边际价值极低的” 长尾” 数据块耗费大量缓存资源，造成了严重的成本浪费。

表 5-2 最大化性能策略的缓存分配结果

租户类型	访问特征	保留块数	价值覆盖率	预算使用率	浪费分析
A（延迟敏感型）	Zipf α=0.8	6,000	89.7%	100%	合理
B（均衡型）	Zipf α=1.3	6,000	89.1%	100%	大量长尾浪费
C（成本敏感型）	Zipf α=2.0	6,000	88.4%	100%	严重长尾浪费
合计	—	18,000	—	—	—

表 5-3进一步展示了在本文提出的差异化弹性缓存策略下，通过为每个租户配置不同的偏好参数 γ（γ 越大，越倾向于减少缓存以节省成本），各租户获得的缓存分配结果。租户 A 保持 $\gamma = 0.05$（性能优先），与当前系统行为一致；租户 B

设置 $\gamma = 0.40$（均衡权衡），仅需 667 块即可覆盖 74.5% 的工作集价值，相比当前策略节省 88.9% 的缓存资源；租户 C 设置 $\gamma = 0.80$（成本优先），仅保留 100 块即可覆盖 71.2% 的工作集价值，节省 98.3% 的缓存资源。系统整体由 18,000 块降至 6,767 块，节省了 **62.4%** 的缓存资源。

表 5-3 性能成本权衡的缓存分配结果

租户类型	γ	偏好	保留块数	价值覆盖率	预算使用率	节省成本
A（延迟敏感型）	0.05	性能优先	6,000	89.7%	100.0%	0.0%
B（均衡型）	0.40	均衡权衡	667	74.5%	11.1%	88.9%
C（成本敏感型）	0.80	成本优先	100	71.2%	1.7%	98.3%
合计	—	—	6,767	—	—	62.4%

值得注意的是，租户 B 在节省 88.9% 缓存成本的同时，价值覆盖率仅从 89.1% 降至 74.5%（下降约 14.6 个百分点）；租户 C 在节省 98.3% 缓存成本的同时，价值覆盖率从 88.4% 降至 71.2%（下降约 17.2 个百分点）。这一性能-成本权衡特性正是现有系统完全无法提供的：现有系统只能在“全量缓存”这一单一操作点上运行，而无法沿着性能-成本曲线灵活选择适合各租户需求的工作点。表 5-4给出了单一租户（Zipf $\alpha = 1.3$）在不同 γ 取值下的性能-成本曲线，直观展示了 γ 参数如何驱动系统在性能与成本之间平滑权衡。

表 5-4 单租户性能-成本权衡曲线

γ	保留块数	价值覆盖率	预算使用率	备注
0.00	6,000	89.1%	100.0%	等效当前系统（最大性能）
0.30	3,767	83.0%	62.8%	节省 37.2% 成本
0.40	367	72.9%	6.1%	节省 93.9% 成本
0.70	167	69.9%	2.8%	节省 97.2% 成本
0.80	67	65.2%	1.1%	节省 98.9% 成本

上述三项缺陷对弹性缓存策略的实际效果产生了深远影响。首先，以命中率为单一优化目标会导致高价值、高收益的缓存项与低收益缓存项被同等对待，造成缓存资源浪费；其次，忽略块密度将导致工作集大小的估计系统性偏高，从而申请不必要的缓存配额；最后，多租户需求的差异化缺失意味着弹性缓存系统无法真正实现“按需分配”，难以在性能与成本之间取得最优平衡。本章针对上述三项缺陷，提出了系统性的改进方案。

为便于后续模型推导与系统实现部分查阅，表 5-5 仅列出本章反复使用的高频符号；低频符号在正文相应位置就地解释。

表 5-5 第五章主要符号说明

符号	含义	符号	含义
T_r	请求总时延	P_r	请求中 I/O 时间占比
V_b	数据块性能价值	$\rho_{b,q}$	查询 q 下块密度
V_b^*	密度修正后块价值	S_t	当前观测价值
L_t	Holt-Winters 水平值	T_t	Holt-Winters 趋势值
γ	性能-成本偏好参数	$C_{\max}$	缓存预算上限
N_{total}	活跃块总数	$\hat{k}$	推荐缓存保留块数
$S_{\text{cur}}(k)$	前 k 块累计价值	Score(k)	配额优化评分函数
S_r	请求加速比	$\mathcal{B}_r$	请求 r 访问的数据块集合
$v_{r,b}$	单块对单请求的性能贡献	$\mathcal{W}$	当前活跃数据块集合
α	Holt-Winters 水平平滑系数	β_{hw}	Holt-Winters 趋势平滑系数

5.2 性能收益与块密度协同的多租户数据价值建模

本节针对上述缺陷一和缺陷二，构建面向弹性缓存的数据块价值模型。与现有方法采用访问频率或最后访问时间来表征数据价值不同，本文认为缓存数据的价值取决于其对上层请求整体性能的实际贡献，并需同时考虑块内有效数据的密度。

5.2.1 请求性能收益建模

对于一条用户请求 r，其总响应时间 T_r 可分解为若干阶段：

$$T_r = T_{\text{queue}} + T_{\text{compute}} + T_{\text{IO}} \tag{5-1}$$

其中 T_{queue} 为排队等待时间，T_{compute} 为计算处理时间，T_{IO} 为数据 I/O 时间。

缓存命中仅能将 T_{IO} 降低至约为零（内存命中）或显著减小（本地缓存命中）。根据阿姆达尔定律（Amdahl's Law），若某请求 r 访问了数据块集合 $\mathcal{B}_r$，在所有数据块均命中缓存的情况下，加速比为：

$$S_r = \frac{T_r}{T_r - T_{\text{IO},r}} \tag{5-2}$$

设 $P_r = T_{\mathrm{IO},r}/T_r$ 为 I/O 时间在总请求时间中的占比，则加速比 $S_r = 1/(1-P_r)$。请求 r 所访问的全部数据块对该请求的性能贡献总量定义为：

$$V_r = T_r \cdot P_r = T_{\mathrm{IO},r} \tag{5-3}$$

由于请求 r 访问了 $|\mathcal{B}_r|$ 个数据块，假设各块的访问权重相同，则单个数据块 $b \in \mathcal{B}_r$ 对请求 r 的性能贡献价值为：

$$v_{r,b} = \frac{V_r}{|\mathcal{B}_r|} = \frac{T_{\mathrm{IO},r}}{|\mathcal{B}_r|} \tag{5-4}$$

同一数据块可能被多条请求访问，其综合缓存价值为所有访问该块的请求贡献之和：

$$V_b = \sum_{r:\, b \in \mathcal{B}_r} v_{r,b} \tag{5-5}$$

5.2.2 块密度价值修正

针对缺陷二，本文引入数据块密度（Block Density）的概念对块的缓存价值进行修正。对于文件 f 中的某个数据块 b，给定查询 q 的访问范围 $[l_q, r_q]$，块密度定义为：

$$\rho_{b,q} = \frac{\mathrm{Count}(b,\, [l_q, r_q])}{|b|} \tag{5-6}$$

其中 $\mathrm{Count}(b, [l_q, r_q])$ 表示块 b 中实际落在查询范围 $[l_q, r_q]$ 内的有效记录数（或字节数），$|b|$ 为块 b 的总大小。$\rho_{b,q} \in [0, 1]$，当查询覆盖整个块时密度为 1，当块内无有效记录时密度为 0。

综合考虑请求性能增益与块密度，数据块 b 在查询 q 下的综合缓存价值为：

$$V_b^* = V_b \cdot \rho_{b,q} \tag{5-7}$$

该价值模型兼顾了两个维度：其一，通过阿姆达尔定律捕获了缓存对不同类型请求的差异化性能贡献；其二，通过块密度修正了因粗粒度块存储导致的价值高估问题。在工作集估计中，本文以 V_b^* 作为块级别的价值度量，用于替代传统方法中简单以字节大小表示的缓存价值。

5.2.3 价值趋势自适应估计

在实际系统中，数据块的访问价值随时间动态变化。为实时追踪每个数据块的价值并在近似数据结构中高效存储，本文采用 Holt-Winters 指数平滑模型对块价值进行自适应估计。

对于数据块 b 的第 t 次访问，设其观测价值为 S_t，Holt-Winters 模型维护两个状态量：平滑水平（Level）L_t 和趋势（Trend）T_t：

$$L_t = \alpha \cdot S_t + (1-\alpha)(L_{t-1} + T_{t-1}) \tag{5-8}$$

$$T_t = \beta_{\text{hw}} \cdot (L_t - L_{t-1}) + (1-\beta_{\text{hw}})T_{t-1} \tag{5-9}$$

其中 $\alpha \in (0,1)$ 为水平平滑系数，$\beta_{\text{hw}} \in (0,1)$ 为趋势平滑系数。我们仅保存每个块 b 最近一次访问后的水平值，记为 $L_b \triangleq L_{t_b^{\text{last}}}$，其中 t_b^{last} 表示块 b 最近一次被访问的时刻。基于此，价值加权工作集（Value-weighted WSS）的计算如下：

$$\text{WSS}_{\text{value}} = \sum_{b \in \mathcal{W}} L_b \tag{5-10}$$

其中 $\mathcal{W}$ 为当前滑动窗口内的活跃数据块集合。通过 Holt-Winters 的自适应平滑，系统能够在噪声较大的价值观测中稳定地追踪每个块的长期价值趋势，为后续的多租户配额优化提供可靠的价值信号。

5.3 基于自适应动态范围过滤器的智能化缓存空间分配策略

在定义了数据块的综合缓存价值之后，本节解决如何高效存储块及其价值信息，并据此实现面向多租户差异化偏好的缓存空间分配。本节首先介绍价值感知的存储结构，然后描述范围过滤器在密度计算中的应用，接着给出多租户性能-成本优化模型，最后总结完整的缓存分配流程。

5.3.1 价值趋势索引存储

本文以数据块（Block）为粒度进行价值的存储和统计。选择块粒度的原因如下：粒度过细（如以记录为单位）会导致存储的元数据过多，内存开销难以接

受；粒度过粗（如以请求为单位）则会因不同请求之间存在大量共享块而导致价值计算重叠，引入计算不准确性。块粒度在二者之间取得了良好的平衡。

本文复用并发时钟式近似索引的桶化存储框架，设计了一种新的价值趋势布谷鸟索引（Value-Trend Cuckoo Index，简称 VTCI），用于高效存储和查询时间窗口内各活跃块的价值信息。如图5-1所示，VTCI 的存储结构和字段如下：

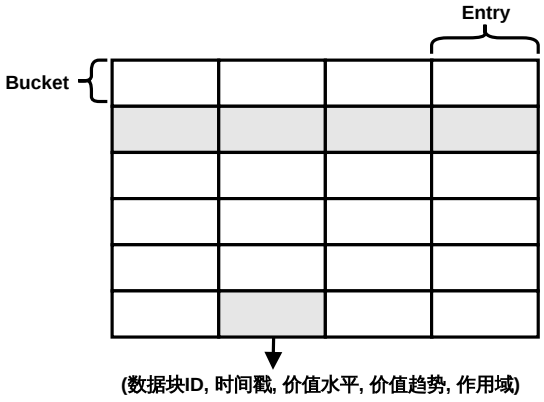


图 5-1 价值趋势布谷鸟索引

- **数据块 ID（Tag）**：b_{tag} 位，用于条目的哈希指纹，支持布谷鸟过滤器的成员查询与迁移；
- **时间戳（Clock）**：c 位，记录条目的时效性，支持滑动窗口老化，
- **价值水平（Level）**：l 位，编码 Holt-Winters 水平值 L_t；
- **价值趋势（Trend）**：d 位，编码 Holt-Winters 趋势值 T_t；
- **作用域（Scope）**：p 位，编码租户/分区标识，支持多租户统计。

给定内存预算 M（字节），过滤器的槽位总数为：

$$N_{\text{slot}} = \frac{8M}{b_{\text{tag}} + c + l + d + p} \tag{5-11}$$

插入操作：当数据块 b 首次被访问时，计算其哈希值以定位桶位置，将指纹写入空槽，时钟计数器初始化为最大值，Holt-Winters 水平值初始化为观测价值 S_0，趋势值初始化为零；同时将 S_0 加入全局价值计数器。

查询与更新操作：当数据块 b 再次被访问时，在过滤器中检索其指纹。若命中，将时钟计数器重置为最大值，并以新观测价值 S_t 更新 Holt-Winters 状态（按公式 (5-8)和 (5-9)），同步更新全局价值计数器；否则视为新块执行插入操作。

老化操作：周期性地对所有槽位的时钟计数器执行减一操作；时钟值归零的槽位对应的条目被视为过期并清除，系统同时从全局价值计数器中减去该条目的水平值，保证过滤器中始终只保留滑动时间窗口内的活跃数据块。

5.3.2 范围过滤器密度估计

为了在运行时高效计算数据块 b 相对于查询 q 的密度 $\rho_{b,q}$（见公式 (5-6)），本文为每个文件维护一个范围过滤器（Range Filter）。范围过滤器是一类支持范围计数查询的近似数据结构，对于给定的键范围 $[l, r]$，能够在亚线性空间内近似回答查询" 该文件中有多少条记录的键值落在 $[l, r]$ 内"，即给出 RangeCount$(f, [l, r])$ 的近似值。

范围过滤器的标准 Lookup 接口仅以布尔值回答区间成员关系，难以满足代价模型等场景对命中数量的需求。本文对 Hourglass 扩展 RangeCount 接口：执行流程与 Lookup 一致——按前缀位宽分解、哈希定位候选桶、逐槽比对指纹，区别仅在边界块内的后缀数据处理方式。Binary 格式将一次" 是否存在" 二分查找改为两次边界二分查找，以左右下标之差作为命中数；Bitmap 格式取消首位置位的提前退出，对目标比特段完整扫描并以 popcount 累加。查询区间横跨两个前缀块时沿用 Lookup 的分解策略，对两个边界块分别计数后求和。由于指纹碰撞带来的假阳性会把无关后缀的数据误纳入统计，RangeCount 返回的是偏高的近似值，其误差上界与过滤器自身的假阳性率一致。

数据块密度的计算流程如下：

（1）对于数据块 b（隶属于文件 f，存储键范围为 $[k^b_{\min}, k^b_{\max}]$）及查询 q 的谓词范围 $[l_q, r_q]$，首先确定查询范围与块范围的交集 $[\max(k^b_{\min}, l_q), \min(k^b_{\max}, r_q)]$；

（2）调用文件 f 的范围过滤器，执行 RangeCount 查询，得到估计的有效记录数 $\hat{n}_{b,q}$；

（3）计算块密度 $\hat{\rho}_{b,q} = \hat{n}_{b,q}/N_b$，其中 N_b 为块 b 内的总记录数；

（4）将密度 $\hat{\rho}_{b,q}$ 与请求性能贡献 V_b（公式 (5-5)）相乘，得到密度修正后的块价值 $V^*_b = V_b \cdot \hat{\rho}_{b,q}$，作为 Holt-Winters 模型的观测输入 S_t。

通过上述流程，范围过滤器为价值感知存储结构持续提供密度修正的价值观测，使得价值趋势布谷鸟索引中的各块水平值能够同时捕获请求性能收益与块内数据密度两个维度的信息。

5.3.3 多租户配额优化模型

针对缺陷三，本文引入用户偏好参数 $\gamma \in [0,1]$，用于刻画租户对性能与成本的权衡偏好：$\gamma = 0$ 表示完全性能优先，$\gamma = 1$ 表示完全成本优先。

问题形式化：设当前滑动窗口内活跃块总数为 N_{total}，成本预算（最大保留块数）为 $C_{\max}$，所有活跃块的总价值之和为 $S_{\max} = \sum_{b \in \mathcal{W}} V_b^*$。若保留价值最高的 k 个块，则保留集合的价值为 $S_{\text{cur}}(k) = \sum_{i=1}^{k} V_{(i)}^*$（其中 $V_{(i)}^*$ 为第 i 大的块价值），对应成本为 $C_{\text{cur}}(k) = k$（每个块占用一个单位的缓存代价）。

本文构造以下弹性缓存效益得分函数：

$$\text{Score}(k) = (1-\gamma) \cdot \exp\left(\frac{S_{\text{cur}}(k)}{S_{\max}}\right) - \gamma \cdot \exp\left(\frac{C_{\text{cur}}(k)}{C_{\max}}\right) \tag{5-12}$$

该得分函数中，第一项 $(1-\gamma)\exp(S_{\text{cur}}/S_{\max})$ 反映性能收益（保留块的价值占比越高则得分越高），第二项 $\gamma \exp(C_{\text{cur}}/C_{\max})$ 反映成本惩罚（保留块数占成本预算比例越高则惩罚越大）。两者通过 γ 加权平衡，使得最优解在性能和成本之间呈现平滑的权衡。

基于采样的最优缓存大小求解：由于实际系统中活跃块总数 N_{total} 可能达到数十万量级，遍历所有块进行排序代价过高。本文采用**随机采样**的方式近似求解：从价值趋势布谷鸟索引中随机采样 n 个活跃槽位，以样本中各块的 Holt-Winters 水平值作为价值代理。设样本价值集合（降序）为 $V_{(1)}^* \geqslant V_{(2)}^* \geqslant \cdots \geqslant V_{(n)}^*$，样本总价值为 $\hat{S}_{\max} = \sum_{i=1}^{n} V_{(i)}^*$，则对任意 $k \in \{1, \cdots, n\}$：

$$\frac{S_{\text{cur}}(k)}{S_{\max}} \approx \frac{\sum_{i=1}^{k} V_{(i)}^*}{\hat{S}_{\max}} \tag{5-13}$$

$$\frac{C_{\text{cur}}(k)}{C_{\max}} \approx \frac{k/n \cdot N_{\text{total}}}{C_{\max}} \tag{5-14}$$

对 $k = 1, \cdots, n$ 枚举计算得分 $\text{Score}(k)$，取最优 k^*，再将其映射回总体推荐保留块数：

$$\hat{k} = \text{clamp}\left(\text{round}\left(\frac{k^*}{n} \cdot N_{\text{total}}\right),\ 0,\ C_{\max}\right) \tag{5-15}$$

整体算法时间复杂度为 $O(n \log n)$（排序主导），与总活跃块数 N_{total} 无关，从而在大规模缓存场景下保持高效性。

驱逐与准入策略：设当前实际缓存的块数为 n_{cache}，推荐配额 $\hat{k}$ 同时驱动两个动作：当 $n_{\text{cache}} > \hat{k}$ 时按价值低序进行驱逐，当新块到达时按价值阈值决定准入。

驱逐由后台调度器以周期 Δ_{evict}（默认 60 s）触发。为避免每轮都对全部 n_{cache} 个块做整体排序，系统额外维护一个文件粒度的 LRU 链表——文件按其内部任意块的最近访问时间排序。为避免缓存反复贴近上限而频繁触发驱逐，本轮的目标驱逐数取 $n_{\text{cache}} - (1-\delta)\hat{k}$，其中 $\delta \in [0,1)$ 为预留空闲比例（默认 $\delta = 0.10$，即每轮驱逐后保留约 10% 的空闲配额作为后续准入的缓冲）。一轮驱逐的流程是：从 LRU 队尾开始向前依次扫描文件 $f_1, f_2, \cdots$；对每个被扫描的文件 f_i，在其内部用 quickselect 找出价值最低的若干块释放，并同步将 VTCI 中对应槽位的 Clock 字段置零；一旦累计驱逐数达到目标值即停止。该” 由旧文件到新文件、文件内由低价值到高价值” 的两级扫描使总开销从 $O(n_{\text{cache}})$ 降到 $O\left(\sum_i |f_i|\right)$，其中 i 只遍历到提前停止处；在每文件块数较多且驱逐量小的常见场景下通常只需扫描少量文件即可，远小于全局扫描的开销。同时按文件聚簇释放也保持了底层 I/O 的局部性。

准入在前台读路径上完成：维护当前缓存中的最低水平值 $L_{\min}$，仅当新块的观测价值满足 $V_b^* \geqslant \eta \cdot L_{\min}$ 时才纳入缓存（η 为宽容因子，$\eta < 1$ 留出余地以适应负载切换）。$L_{\min}$ 由小顶堆维护，每次准入决策只需一次比较和一次哈希插入，均摊开销 $O(1)$。当 $\hat{k} > n_{\text{cache}}$ 时驱逐器反向运行，按水平值从高到低取出未缓存的若干个候选块并由后台线程异步预取。

5.3.4 在线配额调度流程

如图5-2所示，弹性缓存配额调度的完整流程如下：

（1）**价值观测**：当请求 r 访问数据块 b 时，系统计算 I/O 占比 P_r，通过范围过滤器获取块密度 $\hat{\rho}_{b,r}$，计算综合观测价值 $S_t = (T_r \cdot P_r/|\mathcal{B}_r|) \cdot \hat{\rho}_{b,r}$，并据此更新价值趋势布谷鸟索引中对应条目的 Holt-Winters 状态。

（2）**工作集统计**：系统周期性汇总价值趋势布谷鸟索引中所有活跃槽位的水平值，得到 $\text{WSS}_{\text{value}}$ 及活跃块总数 N_{total}。

（3）**采样与优化**：系统从价值趋势布谷鸟索引中随机采集 n 个活跃槽位的价值数据，再由配额优化器根据租户偏好参数 γ 计算推荐保留块数 $\hat{k}$。

（4）**配额更新**：根据 $\hat{k}$ 调整实际缓存配额，执行基于 Holt-Winters 水平值排

序的驱逐或预取操作。

以上四步周期性执行，保证缓存配额随访问模式和用户偏好的变化而自适应调整。



图 5-2 在线弹性缓存配额调度流程

5.4 系统设计与实现

本节介绍本文提出的多租户弹性缓存方法在 Alluxio 分布式缓存系统中的具体实现。该系统将第 5.2 节的数据块价值建模方法与第 5.3 节的多租户配额优化策略统一集成到 Worker 侧本地缓存路径中，在保留 Alluxio 原有 Master、BlockWorker、Page Store 以及 Under File System 协作关系的基础上，引入用户偏好控制、请求价值观测、块密度估计、价值趋势布谷鸟索引、块采样、性能-成本优化以及值感知驱逐等模块，从而使缓存系统能够根据不同租户的偏好自适应调节缓存占用。其内容组织如下：第 5.4.1 节介绍系统架构，第 5.4.2 节介绍各组件的实现细节，第 5.4.3 节给出系统运行示例。

5.4.1 系统架构

Alluxio[46] 是一个面向数据密集型应用的开源虚拟分布式文件系统，其核心目标是在计算框架（如 Spark、Presto、TensorFlow）与底层异构存储（如 HDFS、S3、Azure Blob Storage）之间提供统一的数据访问接口，通过 Worker 侧本地内存缓存加速跨存储层的数据读取。Alluxio 采用主从架构：Master 节点负责管理全局元数据与命名空间，Worker 节点在本地维护 Page Store 以缓存热点数据页；上层应用通过 Alluxio Client 发起请求，由 Worker 的 LocalCacheManager 判断缓存命中状态并按需从底层存储回填。这种架构使 Alluxio 天然适合作为多租户弹性缓存方法的载体：其 Worker 侧缓存路径模块化清晰、可按需扩展，且已在工业界得到大规模部署，具备充分的工程验证基础。

针对本文提出的价值建模与多租户弹性分配方法，本文在 Alluxio 的 Worker

侧设计并实现了一个面向多租户场景的弹性缓存原型系统。图 5-3 展示了系统的整体架构，其中灰色部分为本文新增的功能组件。观察该图可以发现，整个系统自上而下可以划分为用户与应用层、缓存执行与观测层、策略决策层以及元数据与底层存储层。

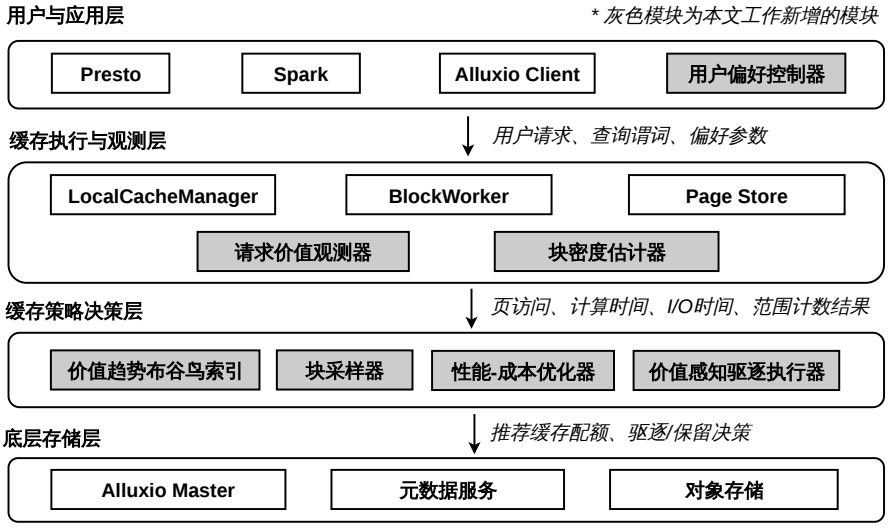


图 5-3 基于 Alluxio 的多租户弹性缓存系统架构示意图

首先，架构图最上层为用户与应用层，包括 Spark、Presto、AI Serving 等上层应用以及 Alluxio Client。该层保留了 Alluxio 原有的请求入口，上层应用仍通过 Alluxio Client 发起页访问请求。与传统 Alluxio 配置方式不同的是，本文在该层引入了用户偏好控制器，用于接收租户对性能与成本的权衡偏好。具体地，该控制器可以通过管理界面中的滑动条暴露参数 γ：当 γ 取值较小时，系统更倾向于保留高价值数据以追求性能收益；当 γ 取值较大时，系统则更倾向于压缩缓存占用以控制资源成本。

其次，架构图中间的缓存执行与观测层运行在 Worker 侧，是本文系统实现的主要落点。该层沿用了 Alluxio 的 LocalCacheManager、BlockWorker 与 Page Store 以完成页面缓存、命中判断和数据返回；在此基础上，本文增加了请求价值观测器与块密度估计器。前者负责从请求执行过程中提取计算时间与 I/O 时间，后者负责基于文件级范围过滤器估计查询范围在各个数据块中的有效记录比例。两类观测结果进一步融合为块级价值信号，并作为后续索引维护与配额优化的输入。

然后，策略决策层是本文方法发挥作用的关键层次。该层以价值趋势布谷鸟索引为中心，在固定内存预算下维护价值化后的活跃块集合。块采样器周期性

地从索引中抽取代表性条目，性能-成本优化器结合用户给定的 γ 参数计算推荐缓存配额，值感知驱逐执行器再依据推荐结果完成具体的保留与驱逐操作。通过上述设计，系统能够将在线访问观测、近似工作集统计以及多租户配额决策组织为一条连续的处理链路。

最后，架构图最下层为 Alluxio 原有的元数据与底层存储层，包括 Master、元数据服务以及 Under File System 或对象存储等组件。本文并未改变 Alluxio Master 的元数据协议和底层数据读写接口，而是将新增逻辑集中集成在 Worker 侧。这种实现方式一方面保留了 Alluxio 现有的数据路径，另一方面降低了系统改造的侵入性，从而便于将本文提出的价值建模与多租户调度方法直接作用于实际缓存管理过程。

5.4.2 组件实现

本节对系统中的关键组件及其运行流程进行详细介绍。

请求价值观测与块密度估计模块

请求价值观测与块密度估计模块负责生成块级价值信号，是后续索引维护和配额优化的输入来源。实现上，系统在 Worker 侧访问路径中增加请求价值观测器，用于记录一次请求在排队、计算和 I/O 阶段的耗时；同时，为每个文件维护一个范围过滤器，由块密度估计器根据查询谓词执行近似 `RangeCount` 计算，以得到当前查询真正命中的块内有效数据比例。

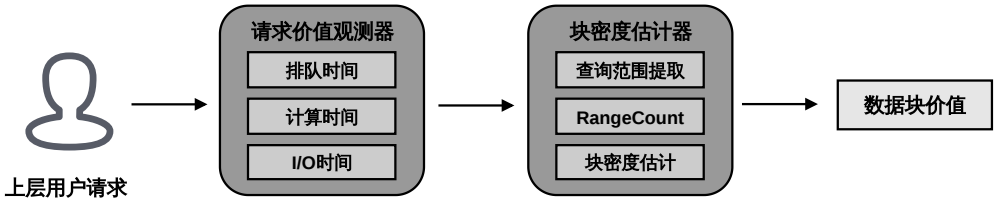


图 5-4 请求价值观测与块密度估计模块运行示意图

图 5-4 展示了请求价值观测与块密度估计模块的工作流程。观察该图可以发现，该模块先从请求访问链路中提取时间信息，再结合查询范围给出块密度估计，最后输出块级观测价值。具体步骤如下：

（1）当用户请求经由 Alluxio Client 到达 Worker 侧时，请求价值观测器首先记录该请求的计算时间与 I/O 时间；

（2）系统根据请求携带的查询谓词定位涉及的数据块范围，并在对应文件的范围过滤器上执行范围计数；

（3）块密度估计器将范围计数结果除以块内总记录数，得到块密度；

（4）系统再将单块 I/O 性能贡献与块密度相乘，得到该数据块的观测价值，并将其连同块标识、块大小和租户作用域一起传递给下游索引模块。

为了便于说明，下面给出一个简单示例。假设 `orders` 文件中键范围为 $[700, 799]$ 的数据块包含 100 条记录，当前查询只命中了其中约 60 条记录，则块密度约为 0.60。这意味着该块虽然被请求访问，但只有约 60% 的块内容真正对当前查询结果产生贡献。若本次请求的 I/O 时间为 40ms，且共访问 4 个块，则该块对请求的单块性能贡献约为 $40/4 = 10$，进一步乘以密度修正后得到的观测价值约为 6.0。由此可见，该模块能够把“数据块是否真正值得保留”转化为后续模块可直接使用的量化输入。

价值趋势布谷鸟索引模块

价值趋势布谷鸟索引模块负责在有限内存预算下维护滑动时间窗口内的活跃块状态，是连接在线观测与配额决策的核心状态模块。实现上，本文复用了仓库中已有的并发时钟过滤框架，并在其基础上扩展槽位中的信息布局和状态更新逻辑，使得每个槽位同时保存指纹、时钟位、块大小、租户作用域以及 Holt-Winters 的水平项与趋势项。因此，索引中的每个条目不再只表示“某个块近期是否访问过”，而是进一步表示“该块近期的价值水平及其变化趋势”。

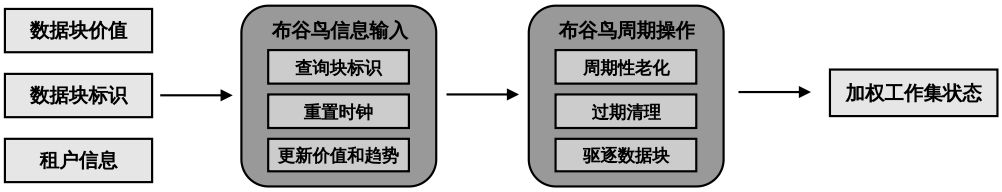


图 5-5 价值趋势布谷鸟索引模块运行示意图

图 5-5 展示了价值趋势布谷鸟索引模块在 Worker 侧维护活跃块状态的过程。观察该图可以发现，该模块围绕首次插入、命中更新和周期性老化三个过程展开。具体步骤如下：

（1）对于首次访问的数据块，系统在价值趋势布谷鸟索引中写入块指纹，初始化时钟位，并以当前观测价值作为 Holt-Winters 水平项的初值；

（2）对于重复访问的数据块，系统在命中后重置时钟位，并利用新的观测价值更新水平项与趋势项；

（3）系统周期性执行老化操作，将滑动窗口之外的条目移出索引，以保证索引始终只保留近期活跃块；

（4）在插入、命中和老化过程中，系统同步维护价值加权工作集大小，从而为配额优化提供全局状态信息。

与传统仅统计块数或字节数的影子缓存相比，价值趋势布谷鸟索引保存的是“价值化后的活跃块集合”。这意味着系统能够在同一份近似索引中统一刻画请求性能收益、块稀疏度与时间趋势，并为后续的采样、优化和驱逐提供一致的数据基础。

块采样与性能-成本偏好决策模块

块采样与性能-成本偏好决策模块负责将规模庞大的活跃块集合压缩为一个可快速求解的配额优化问题，是系统实现多租户差异化调度的关键。考虑到 Worker 在一个时间窗口内可能维护大量活跃块，系统并不直接对全部条目进行全量排序，而是由块采样器从价值趋势布谷鸟索引中抽取代表性样本，再由性能-成本优化器根据这些样本推导推荐缓存配额。

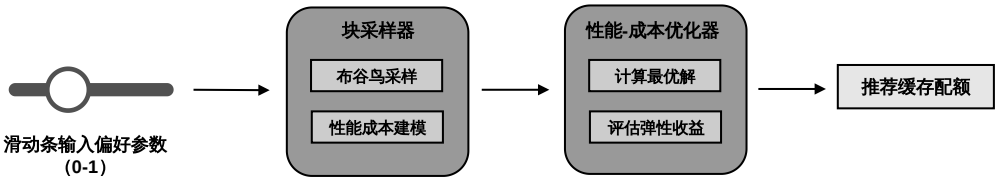


图 5-6 块采样与性能-成本偏好决策模块运行示意图

图 5-6 展示了块采样与性能-成本偏好决策模块的工作链路。观察该图可以发现，该模块由用户偏好控制器、块采样器和性能-成本优化器三部分组成。具体步骤如下：

（1）用户首先通过偏好控制器设置参数 γ，其中 $\gamma = 0$ 表示性能优先，$\gamma = 1$ 表示成本优先；

（2）块采样器从当前活跃条目中抽取样本，并保留每个样本块的价值与单位成本；

（3）性能-成本优化器依据第 5.3 节给出的得分函数，枚举不同保留比例下的收益和成本，计算最优缓存配额；

（4）优化器输出推荐缓存页数，并将结果交给驱逐执行模块。

在实现上，系统支持随机采样、蓄水池采样、分层采样、系统采样以及 Top-K 采样等多种策略。系统运行示例中，为了获得稳定且便于复现的输出，本文选用了 Top-K 采样策略。这样一来，既能够直观展示高价值块如何主导最终配额决策，也便于验证驱逐结果是否与价值排序一致。

值感知驱逐执行模块

值感知驱逐执行模块负责将推荐缓存配额落实为 Worker 侧的具体缓存调整动作，是策略结果作用于实际页缓存的执行端。本文将该模块嵌入 Alluxio 原有的 LocalCacheManager 中：页数据本身仍保存在原有的 Page Store 中，而新增逻辑则负责在缓存规模超过推荐配额时，判断哪些页应当优先被移除。

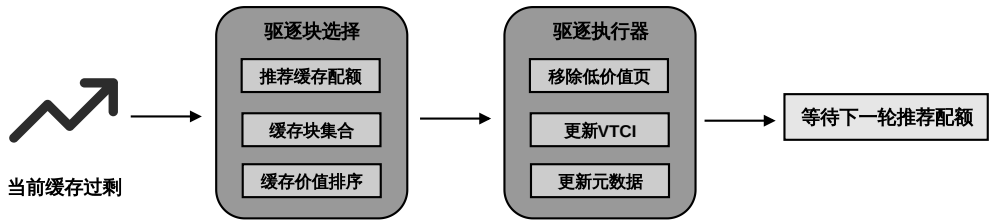


图 5-7 价值感知驱逐执行模块运行示意图

图 5-7 展示了值感知驱逐执行模块将推荐结果落实为实际缓存调整的过程。观察该图可以发现，该模块的核心任务是在推荐配额约束下保留高价值页并及时清理低价值页。具体步骤如下：

（1）驱逐执行器周期性读取当前缓存页数和优化器输出的推荐配额；

（2）若当前缓存规模未超过推荐配额，则系统保持现状，并允许后续访问继续更新价值索引；

（3）若当前缓存规模超过推荐配额，则系统按块价值从低到高排序，优先移除低价值页；

（4）每驱逐一页，系统都会同步更新价值趋势布谷鸟索引和本地页元数据，

以保证下一轮采样和优化基于一致状态。

由于驱逐决策直接继承了上游模块给出的价值信号，因此系统能够优先保留“性能收益大且块密度高”的数据页，而把低密度、低收益的长尾页优先清出缓存。换言之，驱逐动作不再仅仅依据最近访问时间，而是依据“该页对上层请求是否真正有帮助”这一更符合应用语义的标准来执行。

5.4.3 系统运行示例

本节从一个具体的范围查询示例出发，对系统的完整运行过程进行说明。为了验证上文各模块之间的交互关系，本文在一台安装 OpenJDK 11 和 Maven 3.6.3 的 Linux 主机上，对所实现的集成原型进行了实际运行。

首先，系统的关键配置如代码清单 5.1 所示。可以看出，示例开启了 Holt-Winters 价值估计，缓存预算上限被设为 8 页，用户偏好参数取为 $\gamma = 0.50$，表示性能与成本均衡；同时，为了便于复现输出结果，块采样器采用 Top-K 采样策略。需要说明的是，工程实现中仍沿用配置项名 `user_beta`，但其理论含义对应本文中的用户偏好参数 γ。

代码清单 5.1: 系统运行示例中的关键配置

```
alluxio.user.client.cache.shadow.holt_winters.enabled=true
alluxio.user.client.cache.shadow.holt_winters.alpha=0.05
alluxio.user.client.cache.shadow.holt_winters.beta=0.005
alluxio.user.client.cache.shadow.window_size=64
alluxio.user.client.cache.shadow.memory=32kb
alluxio.user.client.cache.elastic.max_cost=8
alluxio.user.client.cache.elastic.sample_size=8
alluxio.user.client.cache.elastic.user_beta=0.50
alluxio.user.client.cache.elastic.sampler=topk
```

接着，可以按照代码清单 5.2 的方式实际执行系统运行示例。该命令首先完成工程编译和运行时类路径构造，随后启动 `ElasticCacheIntegrationDemo`，从而依次触发用户偏好控制、范围过滤器估计、价值观测、配额优化和驱逐执行这五个环节。

代码清单 5.2: 系统运行示例的实际执行命令

```
mvn -q -DskipTests package
mvn -q dependency:build-classpath -Dmdep.outputFile=target/
   classpath.txt
CP="target/classes:$(cat target/classpath.txt)"
java -cp "$CP" alluxio.client.file.cache.integration.
   ElasticCacheIntegrationDemo
```

然后，系统构造了一个名为 orders 的文件，并为其建立范围过滤器；随后模拟 8 个数据块的访问，其中前 4 个块来自 I/O 较重的请求，后 4 个块来自计算与 I/O 较为均衡的请求。系统根据各块的密度和请求 I/O 贡献计算观测价值，再由优化器给出推荐缓存大小。控制台的关键输出如代码清单 5.3 所示。

代码清单 5.3: 系统运行示例的关键控制台输出

```
Preference slider : [==========----------] gamma=0.50 (
   balanced)
Range-aware values:
  page=1 range=[100,199] density=0.80 ioShare=0.67 value
   =12.00
  page=2 range=[200,299] density=0.40 ioShare=0.67 value=6.00
  page=3 range=[300,399] density=0.10 ioShare=0.67 value=1.50
  page=4 range=[400,499] density=0.90 ioShare=0.67 value
   =13.50
  page=5 range=[500,599] density=0.40 ioShare=0.50 value=4.00
  page=6 range=[600,699] density=0.20 ioShare=0.50 value=2.00
  page=7 range=[700,799] density=0.60 ioShare=0.50 value=6.00
  page=8 range=[800,899] density=0.20 ioShare=0.50 value=2.00

Before eviction:
  cachedPages=8 recommendedPages=4 valueWSS=48

After eviction:
  cachedPages=4 recommendedPages=4
  PageId{FileId=orders, PageIndex=4} value=13.50
```

```
PageId{FileId=orders, PageIndex=1} value=12.00
PageId{FileId=orders, PageIndex=2} value=6.00
PageId{FileId=orders, PageIndex=7} value=6.00
```

观察代码清单 5.3 可以发现，系统首先输出了偏好控制器当前的滑动条位置，说明本次运行采用的是性能与成本均衡的配置；接着，系统依次给出了 8 个数据块的密度估计与价值观测结果，其中第 4 页与第 1 页具有最高价值，分别为 13.50 和 12.00，第 3 页的价值最低，仅为 1.50。这表明请求价值观测模块和块密度估计模块已经成功将“请求 I/O 贡献”与“块内有效数据比例”融合为统一的块级价值信号。

进一步地，在驱逐之前系统共缓存了 8 页，而优化器给出的推荐缓存规模为 4 页，价值加权工作集大小为 48。这说明在当前样本分布和用户偏好下，系统认为仅保留一半页面就足以覆盖主要收益。执行值感知驱逐后，缓存页数降至 4，并最终保留了第 4、1、2、7 页这四个高价值块；相应地，第 5、6、8、3 页因价值较低而被优先移出。这与第 5.3 节提出的“按价值排序并在推荐配额处截断”的策略完全一致。

综上所述，本文提出的各个模块已经能够在 Alluxio 缓存访问链路中形成闭环。用户偏好控制器负责表达租户目标，请求价值观测与块密度估计模块负责生成块级价值信号，价值趋势布谷鸟索引负责维护窗口内的价值化活跃块集合，块采样与性能-成本优化器负责计算推荐配额，而值感知驱逐执行模块则将推荐结果落实为实际缓存调整，从而共同完成多租户弹性缓存调度。

5.4.4 系统的不足

本文系统围绕请求级价值建模与多租户偏好优化展开论证，但在更广义的部署环境中，仍存在若干尚未覆盖的场景：

（1）冷启动期的快速预热。与所有基于在线统计的弹性缓存系统类似，系统启动初期由于尚未观测到足够的请求样本，价值趋势索引和采样器的统计量需要若干滑动窗口才能稳定收敛。当前方案通过热启动期阶段缓解，但对于“上线即满流量”的生产场景，引入历史轨迹回放或离线先验初始化的预热策略，能够进一步缩短系统达到稳态决策所需的时间。

（2）跨 Worker 的全局协同。本文方法专注于单节点缓存中的价值建模与配额优化，跨 Worker 的全局视图仍由 Alluxio 自身的元数据协议聚合。对于需要满足全局多租户公平性、跨节点热点迁移或强一致性 SLA 的部署，可以在本文方法之上叠加跨节点的价值同步与协调机制，使各节点的局部决策协同收敛到全局最优配置。

（3）面向非结构化数据的密度估计。范围过滤器天然适配带有有序键域的结构化存储（如分区表、列存文件）；对于无 schema 的对象存储或日志型数据，块密度可借助轻量级元信息探测、键采样或近似指纹直方图等手段构造对应的密度估计器。这一扩展不改变本文价值模型的形式，仅替换密度信号的生成方式。

（4）面向异构存储层级的多级配额决策。本文将价值建模限定在内存缓存这一单层；在内存-SSD-HDD 等多层存储层级中，每一层的成本-性能权衡参数不同，需要为每一层定义各自的价值衰减函数与偏好系数。本文的得分函数与采样优化框架可以直接推广到分层场景，通过为每一层独立维护一份价值索引并联合求解多层配额，即可形成端到端的层级化弹性缓存。

上述并非本文核心贡献所需，但展示了本文价值化弹性缓存框架在更广部署场景中的延展潜力，也为后续研究提供了清晰的演进路径。

5.5 实验与性能评估

为验证本文提出的面向请求价值的弹性缓存方法的有效性，本节设计了一系列实验。首先介绍实验环境、请求类型与工作负载生成方式；接着将 VTCI 与典型近似索引方法在多种工作负载分布下的整体加速比进行对比，并通过消融分析量化块密度修正的边际贡献；在此基础上评估用户偏好参数对性能与成本权衡的影响；进一步比较不同采样策略对配额推荐质量的影响；随后分析 Holt-Winters 参数与采样比例的敏感性；最后在 Alluxio 系统上验证完整方案的整体效果及其时间变化特性。

5.5.1 实验环境与配置

本节实验均围绕第 5.4 节实现的系统展开，主要考察请求价值观测、范围密度估计、价值趋势索引、偏好感知配额优化以及值感知驱逐等模块在连续负载

下的协同效果。实验配置如下。

（1）硬件环境：实验服务器运行 Ubuntu 20.04，处理器为双路 Intel Xeon Gold 6426Y，共 64 个逻辑核，物理内存为 503GB。

（2）软件环境：系统采用 Java 11 实现，使用 Maven 3.6.3 完成依赖管理。

（3）请求类型：本文将实验中的请求明确划分为三类范围查询，并量化其阶段耗时与访问规模，如表 5-6 所示。其中，**I/O 密集型扫描请求**表示大范围顺序扫描，页读取量较大而计算处理较轻；**过滤分析型请求**表示先在一段范围内读取候选页，再对页内记录执行谓词判断与轻量分析，因此 I/O 与计算开销均较明显；**聚合统计型请求**表示在范围读取后执行聚合、分组或统计计算，因而计算阶段占比最高。三类请求的 I/O 占比分别为 91.7%、55.6% 和 12.8%，从而能够更清晰地体现缓存对不同请求类型的差异化收益。

表 5-6 三类范围查询请求的定义

请求类型	查询语义	队列时间/ms	计算时间/ms	I/O 时间/ms	I/O 占比	访问页数/请求
I/O 密集型扫描请求	大范围顺序扫描	5	15	220	91.7%	40–56
过滤分析型请求	范围过滤 + 谓词判断	8	80	110	55.6%	20–32
聚合统计型请求	范围聚合 + 统计计算	10	160	25	12.8%	10–18

（4）工作负载：本文使用了三组连续负载：**稠密范围负载**共 1,280 条请求、平均每条访问 34.24 页；**稀疏过滤负载**共 1,280 条请求、平均每条访问 29.00 页；**热点迁移负载**共 960 条请求、平均每条访问 29.66 页。热点迁移负载的时间戳覆盖区间为 18–49,780，能够完整覆盖多个滑动窗口周期。

（5）评价指标：本文采用四类指标进行评估：**加速比**衡量缓存对端到端响应时间的改善；**缓存成本比例**表示平均缓存占用相对于最大预算的比例；**推荐误差**表示采样决策相对精确最优解的偏差；**决策延迟**表示一次配额推荐所需的时间开销。

（6）对比方法。在偏好权衡实验中，以 $\gamma = 0$ 的性能优先策略作为“无权衡”基线；在采样实验中，对比随机采样、Vitter 蓄水池采样[83]、分层采样、Efraimidis 加权蓄水池采样[84]、Duffield 优先级采样[85]，以及作为上界参考的值引导 Top-K；在与近似索引方法的整体加速比对比中，引入 **SIEVE**[86]、**TTLs-Matter**[87]、**CCF**[25]、**Cuki**[25]、**MBF**[88]、**SHARDS-MRC**[89]、**BMC**[90]、**Counter Stacks**[91] 共 8 类典型工作集近似索引方法（包括 Cuki 之后 EuroSys’24 与 NSDI’24 上发表的最新工作），并加入 **VTCI−** 作为消融基线（其与完整方案 VTCI 共用同一索

引结构和驱逐策略，唯一区别是 Holt-Winters 观测值不乘块密度）；在集成实验中，对比 **Alluxio-LRU**、**仅性能价值**与 **性能成本协同完整方案**三类策略，其中仅性能价值使用请求性能价值而不进行块密度修正。

本节的核心实验参数如表 5-7所示。

表 5-7 第 4.5 节实验核心配置

配置项	取值
逻辑页大小	4KB
每页记录数	100
唯一逻辑页数	20,000
最大缓存预算	4,000 页
滑动窗口长度	12,000 次逻辑操作
默认采样器 / 采样比例	分层采样 / 2%
默认 Holt-Winters 参数	$\alpha = 0.05$，$\beta_{\text{hw}} = 0.005$
用户偏好参数扫描	$\gamma \in \{0, 0.1, \cdots, 1.0\}$
唯一块数 / 缓存容量	2,000 / 500（即工作集 25%）
对比 I/O 占比扫描	$\{0.05, 0.10, 0.20, \cdots, 0.90\}$
对比块密度高值占比	$f_{\text{high}} \in \{1.0, 0.8, 0.6, 0.4, 0.25, 0.10, 0.05\}$（双峰，低密度块为 0.1）

5.5.2 不同负载下的查询加速比对比

本小节在 2,000 个唯一块、缓存预算 500 块（约工作集 25%）的混合价值负载上，对 VTCI 与 8 类典型近似索引方法（**SIEVE**[86]、**TTLs-Matter**[87]、**CCF**[25]、**Cuki**[25]、**MBF**[88]、**SHARDS-MRC**[89]、**BMC**[90]、**Counter Stacks**[91]）以及 VTCI– 进行对比。混合价值负载由三类查询（I/O 点查、过滤分析、全表扫描）按高斯热点分布混合而成，每个块的真实价值由命中它的查询类型自然涌现，频率与价值之间不存在显式相关，从而避免单纯依赖访问频次或最近性的方法因负载偏置而获得“信息红利”。本小节从 I/O 占比与块密度分布两个独立维度对比各方法的查询平均加速比，分别如图 5-8 和图 5-9 所示。

I/O 密集型查询占比的影响：保持总查询数与缓存预算不变，将 I/O 点查在工作负载中的占比从 5% 扫描至 90%，其余比例平均分给过滤分析与全表扫描。

观察图 5-8 可以发现，VTCI 在所有 I/O 占比取值下均位列第一，且其相对最强基线（BMC / Counter Stacks / TTLs-Matter）的优势随 I/O 占比上升而单调放大：占比为 5% 时优势约 6%；占比上升至 60%、80% 时分别扩大到 60% 与 67%；占比 90% 时由于其他方法也能获得较多 I/O 节省，相对优势收窄至 35%。新增的两类近期工作中，TTLs-Matter 凭借 TTL 感知的 MRC 估计在 I/O 占比 30%–80%

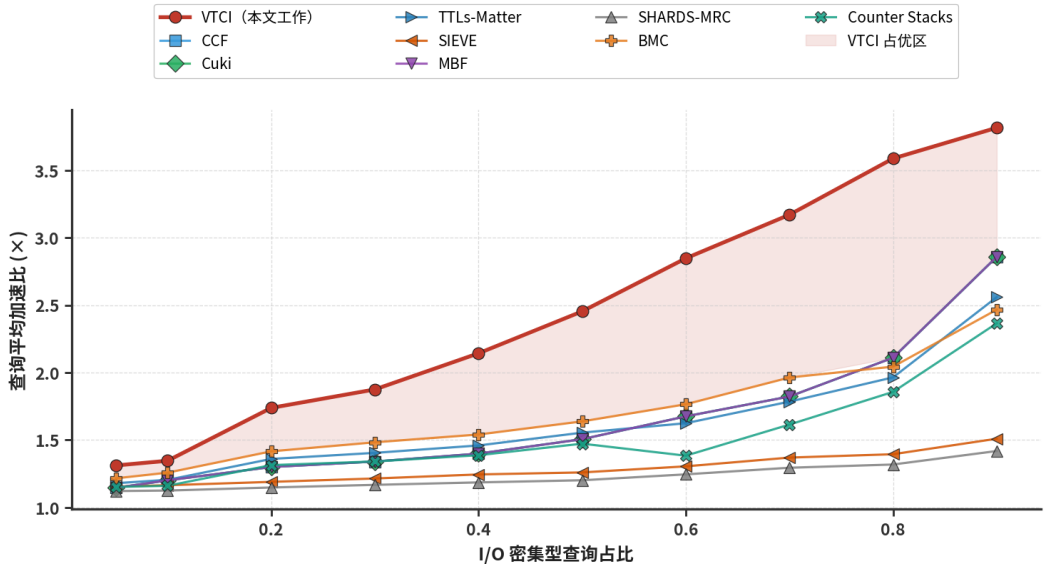


图 5-8 不同 I/O 占比下 VTCI 与各近似索引方法的查询平均加速比对比

区间显著优于 Counter Stacks（最高 +12%），但因其本质仍为无价值化的 WSS 估计器，所对应的端到端加速比始终落后 VTCI 约 25%–50%；SIEVE 作为按访问位的轻量级驱逐策略，整体表现介于 SHARDS-MRC 和 MBF 之间，相比 VTCI 落后较大幅度，原因在于其按最近性而非请求价值排序。这表明 VTCI 的优势并不依赖于某个特定的工作负载形态——只要负载中存在差异化的请求性能贡献，VTCI 的价值排序就能稳定地把缓存优先分配给单次命中收益最大的块。

块密度分布的影响：固定 I/O 占比 33%，将块密度采样自双峰分布（核心块密度 1.0、边界块密度 0.1），扫描核心块占比 $f_{\text{high}} \in \{1.0, 0.8, 0.6, 0.4, 0.25, 0.10, 0.05\}$，观察密度修正项对加速比的实际边际贡献。

观察图 5-9 可以发现，$f_{\text{high}} = 1.0$ 时所有块密度均为 1.0，密度修正项退化为乘 1，VTCI 与 VTCI– 完全等价（图中两条曲线在最左端重合）。随着 f_{high} 下降、块密度异质度增大，VTCI 与 VTCI– 之间逐步拉开差距并稳定扩大：$f_{\text{high}} = 0.8$ 时差距约 +10.7%，$f_{\text{high}} = 0.6$ 时约 +10.0%，$f_{\text{high}} = 0.4$ 时约 +9.6%；当 f_{high} 进一步下降到 0.25 及以下，由于核心块过少、整体缓存收益减小，二者差距收窄至 1%–5%。VTCI 与 VTCI– 同时显著优于其余 8 类基线，且这一优势在所有 f_{high} 取值下保持一致——这说明本文索引结构所带来的端到端加速主要来自请求级价值学习，而块密度修正在异质度较大的工作负载下进一步贡献约 10% 的额外加速比。新增的 TTLs-Matter 和 SIEVE 在该实验中表现出与其设计目标相符的行为：TTLs-Matter 在 $f_{\text{high}} = 1.0$ 时凭借更精细的 WSS 估计达到 1.435 的加速

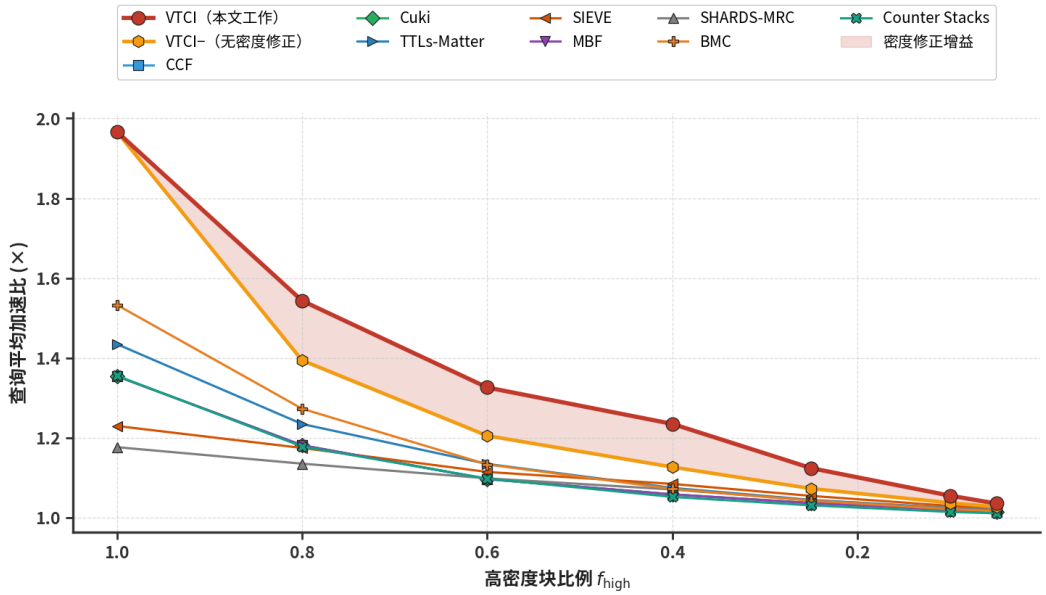


图 5-9 不同块密度分布下 VTCI 与 VTCI− 及各近似索引方法的对比

比，介于 BMC 与 Counter Stacks 之间；但由于其同样不感知块密度，随 f_{high} 下降快速收敛到与其他无密度修正方法相同的水平，与 VTCI 的差距持续拉大至 1.8 倍以上。SIEVE 由于按访问位驱逐而非价值排序，在所有 f_{high} 取值下均贴近 SHARDS-MRC 曲线，进一步说明若不在价值化层面引入块密度，仅靠驱逐策略改良无法弥补由块密度异质带来的额外加速空间。

综合两组对比可知，VTCI 在 I/O 占比和块密度分布两个独立维度上对 8 类近似索引方法均保持稳定优势；通过 VTCI− 消融对比，本文进一步分离出” 密度修正” 作为独立模块的可观测边际收益，从而验证了密度修正设计在面向分布异质负载时确实具有可量化的端到端价值。

5.5.3 性能-成本偏好权衡评估

本小节对用户偏好参数 γ 所带来的性能与成本权衡进行评估。实验采用 2400 条请求组成的热点迁移负载（4 个热区阶段、每阶段 200 轮、每轮 3 类查询），以 $\gamma = 0$ 的性能优先策略作为基线，并将 γ 从 0 以 0.1 为步长扫描到 1。延长每阶段轮数的目的，是让稳态命中率主导测量结果，避免阶段切换处短暂冷启动带来的偏差。考虑到单次在线采样会带来一定随机波动，为更清晰地观察 γ 的影响规律，本实验在该组实验中采用分层采样，将采样比例提高到活跃块数的 4%，并对每个 γ 重复执行 20 次。图 5-10 中柱体与折线表示均值，误差线与阴影表示

标准差。

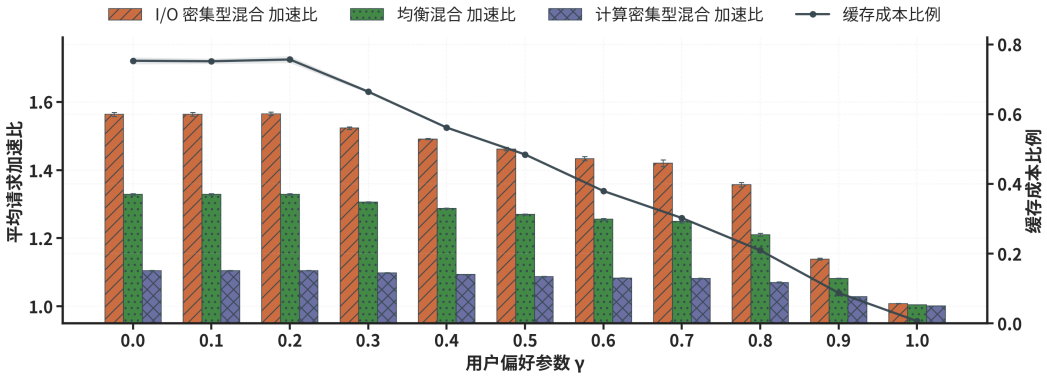


图 5-10 不同用户偏好参数 γ 下的请求加速比与缓存成本（均值与标准差）

为更直观地展示偏好参数对不同负载场景的影响，本文将整体工作负载按 *I/O* 密集型混合（85% I/O 扫描、10% 过滤、5% 聚合）、均衡混合（34%/33%/33%）和计算密集型混合（5%/10%/85%）三种比例分别加权得到混合负载下的请求加速比。三种混合共享相同的滑动窗口与缓存预算，因此缓存成本随 γ 的变化在三种混合下完全一致，只需绘制一条成本曲线。

观察图 5-10 可以发现，随着 γ 的增大，系统整体呈现出“缓存成本持续下降、各混合负载加速比分层回落”的趋势，且 20 次重复下的标准差始终维持在均值的 1% 以内，表明该趋势主要由偏好控制机制驱动而非随机抖动。在 $\gamma = 0$、0.1 和 0.2 这一性能优先区间内，三种混合负载的加速比几乎不发生变化：I/O 密集型混合稳定在 1.564 ± 0.006，均衡混合稳定在 1.329 ± 0.003，计算密集型混合稳定在 1.105 ± 0.001；与此同时，缓存成本比例稳定在 0.754 ± 0.010 附近，标准差略大于其他区间，反映出系统在性能优先运行点上对采样波动更敏感。这一现象说明，在轻度偏好变化下，系统始终维持在接近性能优先的运行点上。

需要说明的是，若直接采用单次在线采样结果，曲线在 $\gamma = 0.2$ 与 $\gamma = 0.8$ 附近可能出现轻微的局部非单调现象。进一步分析表明，这并非由性能-成本得分模型本身导致，而是由在线采样估计与动态负载耦合共同引起的：低采样比例下单次随机样本本身存在统计波动，而值偏置较强的优先级采样在热点迁移场景中更容易暂时保留上一阶段的高值页，并与后续在线驱逐过程相互作用。本文在图 5-10 中采用分层采样、提高采样比例并进行 20 次重复平均后，该类局部异常已明显减弱，且对应的加速比标准差均收窄到 10^{-2} 量级以内，说明图中的整体变化趋势主要由偏好控制机制决定，而局部扰动主要来自在线采样和负载

迁移的共同影响。

当 γ 增大到 0.3 时，缓存成本比例由 0.754 ± 0.010 降至 0.665 ± 0.004，平均下降 11.7%，且成本标准差同步收窄至原值的二分之一以下；与此同时，I/O 密集型混合的加速比由 1.564 ± 0.006 降至 1.524 ± 0.004，均衡混合由 1.329 ± 0.003 降至 1.306 ± 0.003，计算密集型混合由 1.105 ± 0.001 降至 1.098 ± 0.001，三种混合负载均仅发生轻微下降。继续增大到 $\gamma = 0.5$ 时，缓存成本比例降至 0.485 ± 0.003，相比性能优先基线下降 35.6%；I/O 密集型、均衡和计算密集型混合的加速比分别为 1.462 ± 0.005、1.270 ± 0.003 和 1.087 ± 0.001。再到 $\gamma = 0.7$ 时，缓存成本仅为 0.303 ± 0.003，相比基线节省 59.8% 的缓存资源，三种混合的加速比仍分别保持在 1.421 ± 0.010、1.250 ± 0.005 和 1.082 ± 0.002。值得注意的是，I/O 密集型混合在 $\gamma = 0.7$ 处的加速比标准差为整条曲线的极大值（约 0.010），与图中误差线的局部加宽一致，这表明在中度成本压力下，热点迁移阶段的边界页面切换会略微放大端到端收益的随机性，但量级仍远小于均值差异。实验结果表明，本文提出的偏好控制能够在“有限性能损失”和“显著成本节省”之间提供较为平滑、且重复性良好的连续调节能力。

当 γ 继续增大到 0.9 甚至 1.0 时，系统逐步逼近成本优先的运行点。$\gamma = 0.9$ 时缓存成本比例仅为 0.088 ± 0.001，相比基线下降 88.3%；此时 I/O 密集型、均衡和计算密集型混合的加速比分别为 1.139 ± 0.003、1.082 ± 0.002 和 1.028 ± 0.001。$\gamma = 1.0$ 时缓存成本进一步降至 0.006 ± 10^{-5}，系统几乎仅保留少量最核心页面，三种混合负载的加速比分别退化到 1.008 ± 10^{-4}、1.005 ± 10^{-4} 和 1.001 ± 10^{-5}；此时各项标准差均已趋近 0，说明在极端成本优先下系统进入近似确定的运行点。由此可以看出，I/O 占比越高的混合负载对缓存缩减越敏感，而计算密集型混合即便缓存配额大幅下降，其端到端性能也只发生轻微变化，这是阿姆达尔模型的自然推论。

综上所述，用户偏好参数 γ 能够在性能优先与成本优先之间形成连续、可解释的控制曲线，且三种典型工作负载混合按其整体 I/O 占比呈现出清晰不同的收益衰减速度；同时，20 次重复实验所得的标准差在全部 γ 取值下均明显小于均值差异，表明该控制曲线在动态热点迁移负载下具有良好的重复性与可观测性。上述结论与本文提出的价值模型一致。

5.5.4 采样策略对比评估

本小节评估不同采样策略对缓存配额推荐结果的影响。实验比较六类代表性方法：**随机采样**从活跃块集合中近似均匀抽样；**Vitter 蓄水池采样**在单次扫描过程中维护固定容量样本；**分层采样**按过滤器桶空间分层抽样，以提高空间覆盖性；**加权蓄水池采样**依据块价值调节入样概率；**优先级采样**通过随机优先级进一步偏向高价值块；**值引导 Top-K** 直接截取当前价值最高的样本，作为强值偏置条件下的参考上界。实验仍采用热点迁移负载，并将采样规模统一设置为当前活跃块数的 2%，分别在 $\gamma \in \{0, 0.2, 0.4, 0.6, 0.8, 1.0\}$ 六组用户偏好下进行测试。

整体来看，$\gamma = 0, 0.2, 0.4, 0.6$ 四组下各方法的相对排序基本一致；当 γ 增大到 0.8 时，随着推荐配额显著收缩，不同方法在推荐误差与决策时延上的差异开始放大；当 $\gamma = 1.0$ 时，系统已接近零配额运行点，各方法的加速比与缓存成本比例基本收敛。基于此，图 5-11 选取 $\gamma = 0, 0.4, 0.8$ 三组具有代表性的结果进行展示。未展示的 $\gamma = 0.2$ 与 $\gamma = 0.6$ 两组实验结果与图中总体趋势一致，由于篇幅限制不再重复给出；$\gamma = 1.0$ 时各方法已近似收敛，故亦不再单独绘制。

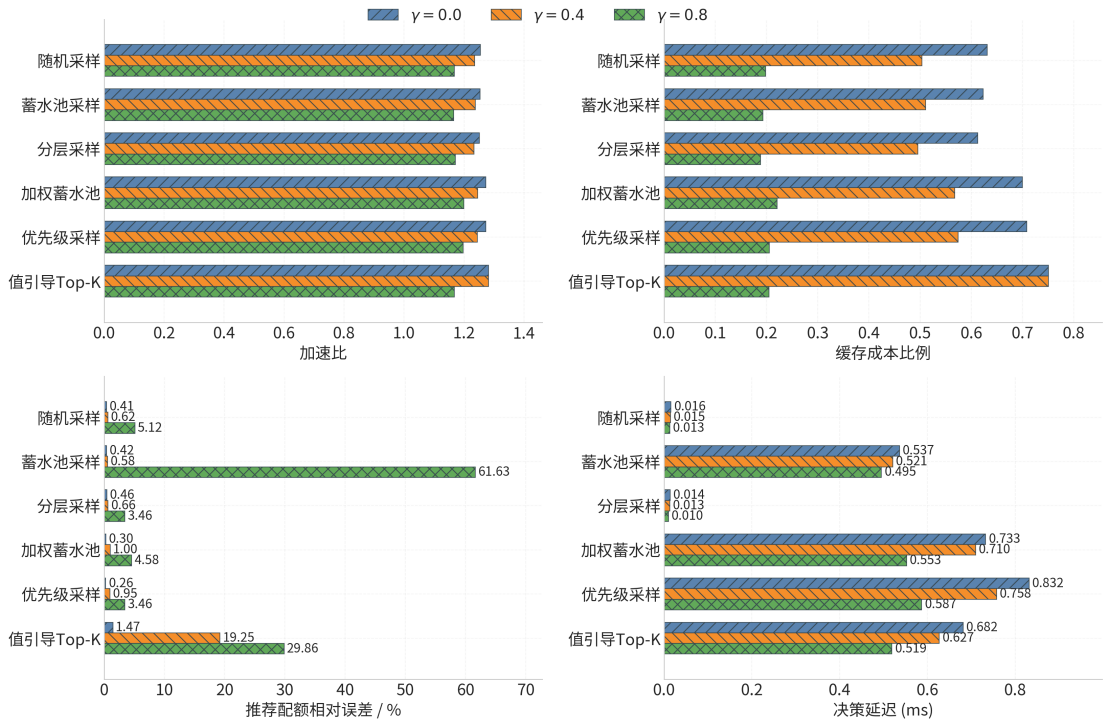


图 5-11 不同采样算法的效果对比

图 5-11 表明，不同 γ 取值下各类采样策略呈现出较稳定的结构性差异。在

性能优先的 $\gamma = 0$ 情形下，加权蓄水池采样、优先级采样与值引导 Top-K 的加速比分别达到 1.273、1.273 和 1.282，略高于接近均匀抽样的几类方法；但其缓存成本比例也同步升至 0.700、0.708 和 0.751。相比之下，分层采样的加速比为 1.252，虽略低于上述值偏置方法，但其缓存成本比例仅为 0.613，决策延迟仅为 0.014ms，整体上表现出更好的单位成本收益。

在折中偏好的 $\gamma = 0.4$ 情形下，随机采样、Vitter 蓄水池采样与分层采样的表现最为接近：三者的加速比分别为 1.236、1.238 和 1.233，缓存成本比例分别为 0.503、0.511 和 0.496，推荐误差均保持在 0.006 量级。其中，分层采样在保持接近加速比的同时取得最低的缓存成本比例，且决策延迟仅为 0.013ms，明显低于蓄水池采样的 0.521ms。这说明通过增强空间覆盖性，分层采样能够在不显著牺牲收益的前提下提高估计稳定性，并有效控制在线决策开销。

在成本偏置较强的 $\gamma = 0.8$ 情形下，各方法的加速比整体收敛到 1.17–1.20 区间，但推荐误差与在线开销的差异反而更加突出。此时，分层采样的加速比为 1.171，缓存成本比例为 0.188，推荐误差为 0.0346，决策延迟仅为 0.010ms；优先级采样虽然仍可取得 1.197 的加速比，但其决策延迟已增至 0.587ms；Vitter 蓄水池采样的推荐误差则进一步上升到 0.616。该结果表明，在低配额运行区间内，样本覆盖性与估计稳定性比单纯加强对高价值块的偏置更为关键。

综合各组实验结果，若强调绝对性能收益，则优先级采样和加权蓄水池采样略占优势；但若同时考虑在线决策开销、推荐稳定性与单位成本收益，则分层采样更为稳健。因此，后续参数敏感性分析与集成原型实验均采用分层采样作为默认在线采样方案。

5.5.5 参数敏感性分析

本小节分别对 Holt-Winters 价值平滑参数以及采样比例进行敏感性分析，以考察本文方法的稳定性。对于价值平滑部分，本文复用两类动态价值序列：一类为平滑变化的 sinusoidal 序列，另一类为热点突变的 phase-shift 序列；对于采样部分，则在固定热点迁移负载上扫描不同采样比例。Holt-Winters 模型的两个系数分别控制” 水平” 和” 趋势” 的更新速度，其敏感性需要在价值动态的两种极端形态下考察。本文据此选取 sinusoidal（连续平滑变化，对应日内规律或热度缓慢漂移）与 phase-shift（阶跃突变，对应租户切换或查询类型阶段性转换）两类

合成负载——前者检验参数过大时是否引发不必要的振荡，后者检验参数过小时是否在突变后响应迟缓；两者在数学结构上互补，组合后可覆盖真实多租户缓存场景中的主要价值动态形态。实验结果如图 5-12 所示。

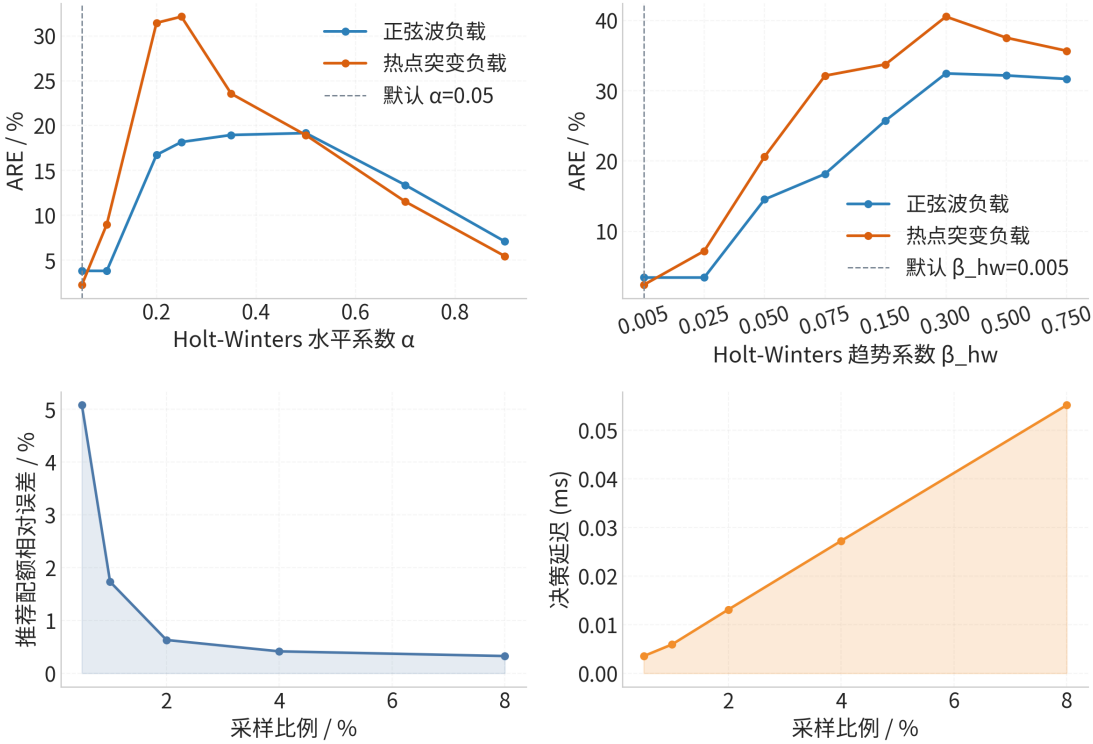


图 5-12 Holt-Winters 参数与采样比例的敏感性分析

观察图 5-12 的前两幅子图可以发现，Holt-Winters 参数对价值估计误差具有显著影响。在 $\beta_{\mathrm{hw}} = 0.075$ 固定时，sinusoidal 序列在 $\alpha = 0.05$ 与 $\alpha = 0.10$ 时的 ARE(Value) 分别为 3.82% 和 3.81%；但对于变化更剧烈的 phase-shift 序列，$\alpha = 0.05$ 时误差仅为 2.26%，而 $\alpha = 0.10$ 已上升到 8.99%，默认旧参数 $\alpha = 0.25$ 则进一步增至 32.14%。这说明在热点突变和阶段切换场景中，较小的水平平滑系数更有利于抑制估计抖动。因此，本文选择 $\alpha = 0.05$ 作为默认配置。

趋势平滑参数 β_{hw} 的影响更为显著。当 $\alpha = 0.25$ 固定时，sinusoidal 序列在 $\beta_{\mathrm{hw}} = 0.005$ 和 0.025 时的 ARE(Value) 均为 3.43%，而当 β_{hw} 增大到 0.075 时误差已上升到 18.17%；phase-shift 序列在 $\beta_{\mathrm{hw}} = 0.005$ 时误差仅为 2.39%，当 $\beta_{\mathrm{hw}} = 0.30$ 时则达到 40.54%。实验结果表明，过大的趋势更新系数会将短期波动放大为持续趋势，从而削弱价值估计稳定性。因此，本文将默认趋势系数设置为 $\beta_{\mathrm{hw}} = 0.005$。

图 5-12 的后两幅子图进一步展示了采样比例的影响。随着采样比例从 0.5%

提升到 2.0%，推荐误差由 0.0508 下降到 0.0063，降幅达到 87.6%；与此同时，决策延迟仅由 0.0035ms 增加到 0.0131ms。继续将采样比例提升到 4.0% 时，推荐误差进一步降至 0.0042，而决策延迟上升到 0.0272ms；当采样比例扩大到 8.0% 后，误差只由 0.0042 小幅下降到 0.0033，但决策延迟已进一步增加到 0.0552ms，边际收益明显减弱。由此可见，采样比例采用 2%–4% 时已能够覆盖主要收益区间，其中 2% 更适合作为默认在线配置，因为其误差已降至 10^{-2} 以下，同时仍保持很低的实时决策代价。

综上所述，本文方法对参数设置具有一定敏感性，但其规律较为清晰：Holt-Winters 模块宜采用较保守的平滑系数以抑制估计抖动，而采样模块则宜采用中等采样比例以兼顾误差与时延。

5.5.6 Alluxio 集成有效性评估

本小节在 Alluxio 系统上，对**本文完整方案（性能成本协同）**的整体有效性进行评估。实验分别在稠密范围负载、稀疏过滤负载和热点迁移负载上，对比传统的 Alluxio-LRU 策略、仅采用请求性能价值而不进行块密度修正的仅性能价值策略，以及本文完整方案。实验结果如图 5-13 所示。

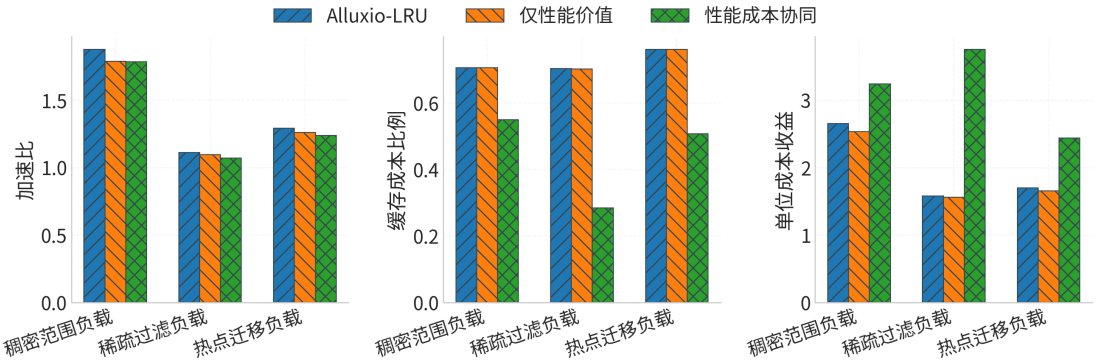


图 5-13 集成 Alluxio 原型上的整体效果对比

观察图 5-13 可以发现，在**稠密范围负载**下，完整方案的加速比为 1.786，较 Alluxio-LRU 的 1.878 仅低 4.9%，但缓存成本比例由 0.707 降至 0.550，下降了 22.1%；对应的单位成本收益则由 2.658 提升到 3.245。与仅性能价值策略相比，完整方案的原始加速比几乎不变，但缓存成本明显降低。这说明当查询接近整页扫描时，块密度修正虽然不会显著提高原始加速比，却仍能够抑制对边际收益较低页面的过度保留。

在**稀疏过滤负载**下，完整方案的优势更加明显。其加速比为 1.071，仅比 LRU 低 3.7%，但缓存成本比例仅为 0.285，较 LRU 的 0.704 降低了 59.5%，单位成本收益达到 3.757，约为 LRU 的 2.38 倍，也明显高于仅性能价值策略的 1.564。这表明当大量页面只包含少量真正参与过滤与聚合的记录时，范围密度修正能够有效削减对“低密度、低收益”页面的无效保留。

在**热点迁移负载**下，完整方案的加速比为 1.239，略低于 LRU 的 1.294，但缓存成本比例由 0.761 降至 0.507，下降了 33.4%，单位成本收益则由 1.700 提升到 2.443。相比仅性能价值策略，完整方案仅牺牲了 1.8% 的原始加速比，却换来了 33.4% 的成本下降。这说明在工作集随时间迁移的场景下，请求价值观测、价值趋势估计、分层采样、偏好感知配额推荐与值感知驱逐能够形成稳定协同，从而在有限缓存预算下持续跟踪真正高收益的页面。

为了进一步观察系统在时间维度上的行为，本文在热点迁移负载上引入一组随阶段变化的用户偏好参数 $\gamma = \{0.05, 0.25, 0.55, 0.85\}$，并记录缓存页数、滚动加速比与推荐配额随时间的变化。实验结果如图 5-14 所示。

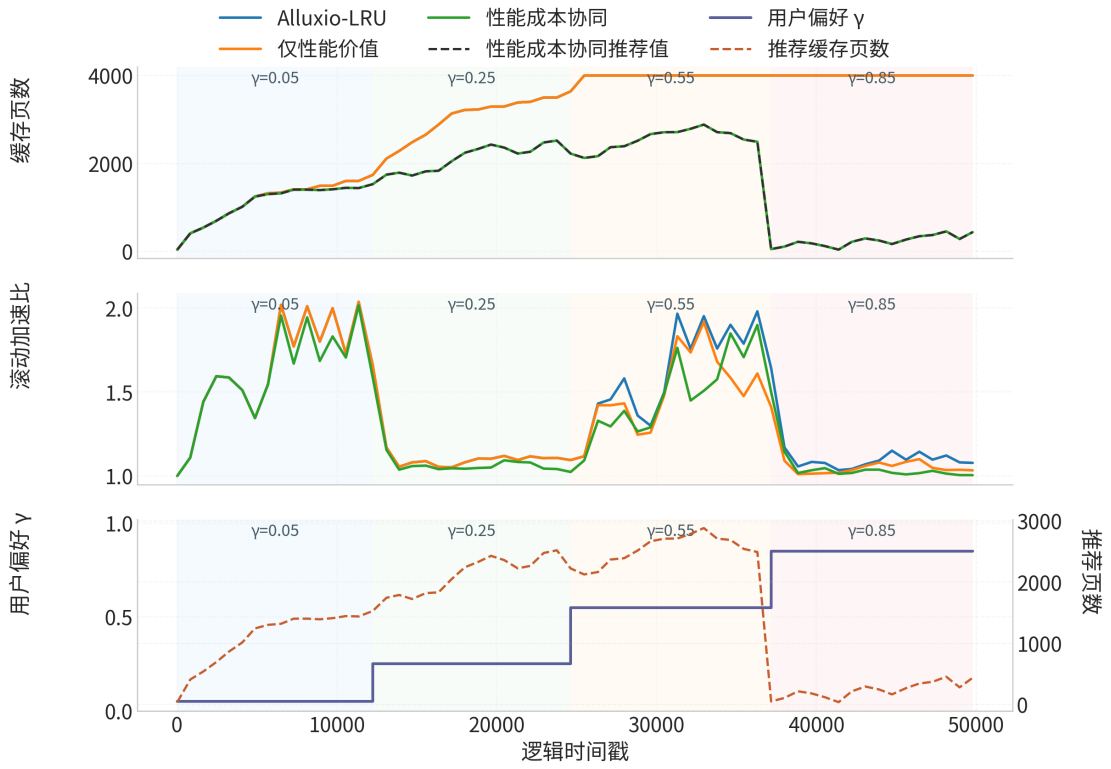


图 5-14 集成原型中缓存决策的时序变化

观察图 5-14 可以发现，在第一个性能优先阶段（$\gamma = 0.05$）中，完整方案的

平均缓存页数为 1,062 页，与 Alluxio-LRU 的 1,099 页基本接近，说明系统首先保证了核心热点页的保留。当偏好提升到 $\gamma = 0.25$ 与 $\gamma = 0.55$ 时，完整方案的平均缓存页数分别为 2,090 页和 2,533 页，而 Alluxio-LRU 已分别上升到 2,941 页和 3,976 页；与此同时，完整方案的推荐配额主要分布在 1,529–2,521 页和 2,126–2,884 页区间，表现出连续且平滑的收缩轨迹。进一步地，当进入成本优先阶段 $\gamma = 0.85$ 后，完整方案的推荐页数迅速压缩到 40–455 页，而仅性能价值策略与 Alluxio-LRU 仍几乎保持满配额运行。这表明本文提出的偏好控制器能够将用户偏好稳定地传递到配额推荐与驱逐执行环节。

从滚动性能收益来看，完整方案在 $\gamma = 0.25$、0.55 和 0.85 三个阶段的平均滚动加速比分别为 1.098、1.462 和 1.059，均明显高于无缓存基线。尤其是在 $\gamma = 0.85$ 阶段，系统以平均 238 页缓存保持了 1.004–1.500 的滚动加速比，相比 Alluxio-LRU 的 4,000 页缓存缩减了约 94.0%，而平均滚动加速比仅下降约 6.1%。这表明在成本优先阶段，系统会主动释放边际收益较低的页面，但仍能依靠价值模型保留少量真正关键的数据块，从而避免性能完全退化为无缓存状态。

5.5.7 结果分析与讨论

综上所述，Alluxio 集成实验验证了本文完整方案的有效性。传统 LRU 策略虽然在部分场景下能够取得较高的原始加速比，但往往以较高缓存成本为代价；仅性能价值策略能够一定程度引入请求差异化价值，却仍难以处理块密度差异和强偏好变化；而本文完整方案则能够在动态工作负载下同时兼顾性能收益、缓存成本与用户偏好调节能力，尤其在稀疏访问和热点迁移场景中表现出更高的单位成本收益。

5.6 本章小结

本章针对多租户弹性缓存中“命中率指标单一、块密度信息缺失、偏好建模不足”导致的价值评估与配额决策偏差问题展开研究，解决了性能收益与缓存成本难以协同优化的关键问题。首先，本章基于阿姆达尔定律构建了面向请求端到端性能增益的数据块价值模型；其次，本章引入范围过滤器的 RangeCount 机制对块内有效密度进行修正，提升块级价值估计精度；然后，本章提出基于指数得

分函数与采样优化的多租户弹性缓存配额分配算法，实现对差异化性能-成本偏好的统一建模；最后，本章将上述方法集成为以 Holt-Winters 价值趋势估计为核心的在线调度方案，并在 Alluxio 系统上进行了评估。实验结果表明，在稀疏过滤负载下，完整方案在加速比较 LRU 总体持平的情况下将缓存成本降低 59.5%，单位成本收益提升至 LRU 的约 2.38 倍；在热点迁移且成本优先阶段，缓存页数可缩减约 94.0%，而平均滚动加速比仅下降约 6.1%。

第六章　总结与展望

6.1　本文工作总结

近年来，随着现代数据平台、分层存储系统与多租户缓存服务的快速发展，海量数据上的范围查询、过滤分析与工作集管理已成为现代数据管理系统中的基础能力。范围过滤器作为支撑区间查询剪枝、块密度刻画与近似范围计数的重要数据结构，正在日志分析、时序数据库、缓存管理与分布式存储等场景中得到越来越广泛的应用。与此同时，现代数据系统中的数据访问模式更加复杂：查询负载往往同时具有倾斜性、动态性与多租户竞争特征，系统不仅需要在有限空间内维持稳定的过滤精度，还需要将过滤结果进一步服务于缓存调度、资源分配与性能优化等更高层机制。

然而，复杂负载环境下的范围过滤与缓存调度仍面临多方面挑战：一方面，倾斜与对抗性查询会放大重复误判，使过滤精度和端到端性能持续退化；另一方面，紧凑编码的范围过滤器普遍缺乏高效在线增删能力，难以适应动态数据负载；此外，多租户缓存系统还面临请求性能收益、块内有效密度与用户性能–成本偏好难以统一建模的问题。现有研究大多分别关注静态过滤精度、动态更新效率或缓存替换策略，缺乏一条从范围过滤器结构优化到弹性缓存应用落地的统一技术路径。针对上述问题，本文围绕面向复杂负载的范围过滤器优化及其弹性缓存应用关键技术展开系统研究，并形成了从过滤器结构设计、动态维护到缓存调度落地的一体化技术路径。本文的主要研究工作和贡献点包括：

（1）针对倾斜与对抗性查询场景下范围过滤器重复误判持续累积的问题，本文提出了面向倾斜负载的范围过滤器自适应指纹更新技术。该技术设计了 Hourglass 自适应范围过滤器，通过自适应半排序布谷鸟过滤器、混合后缀块与紧凑比特位图的协同组织，实现前缀指纹存储与后缀紧凑编码的统一；进一步地，本文提出基于滑动窗口的半排序自适应更新机制，以及基于处理器架构的构建访存优化方法，在保证查询效率与空间效率的同时抑制重复误判。实验结果表明，

该自适应范围过滤器在对抗性负载下可将误判率降低 9.8–35.4 倍，并将端到端查询延迟降低 1.4–37.7 倍，体现出较好的鲁棒性与实用性。

（2）针对动态更新场景下紧凑范围过滤器难以在线维护的问题，本文提出了面向更新负载的范围过滤器后缀位级化增删技术。该技术围绕 HSB 紧凑布局下的动态维护问题，设计了基于位级合并的后缀插入策略、基于二分补位的后缀删除策略以及前缀槽位级联清除机制，并进一步提出关联度感知的过滤器空间动态化分配方法，以兼顾更新效率、空间利用率与过滤精度。实验结果表明，所提出的方法在典型动态删除场景下较重建式参考方案具有显著更低的更新开销，并在多轮插入与删除后保持了稳定、较低的误判率，说明该方法能够在动态负载下保持可靠的在线维护能力。

（3）针对多租户弹性缓存场景下数据价值评估与资源调度难以精细协同的问题，本文提出了基于自适应动态范围过滤器的多租户弹性缓存配额调度方法。该方法基于阿姆达尔定律构建请求性能收益与块密度协同的数据价值模型，利用范围过滤器的近似 RangeCount 能力估计块内有效记录比例，并结合 Holt-Winters 价值趋势估计、采样优化与值感知驱逐机制，实现面向差异化性能–成本偏好的在线配额推荐与执行。实验结果表明，在稀疏过滤负载下，完整方案在加速比较 LRU 总体持平的情况下将缓存成本降低 59.5%，单位成本收益提升至 LRU 的约 2.38 倍；在热点迁移且成本优先阶段，缓存页数可缩减约 94.0%，而平均滚动加速比仅下降约 6.1%。

（4）基于上述研究，本文完成了从范围过滤器结构优化、自适应维护、动态更新到多租户缓存调度应用的系统集成，并在 Alluxio 系统上验证了整体方案的有效性。总体来看，本文所提出的方法不仅在复杂查询负载与动态更新场景中提升了范围过滤器的精度与维护效率，也进一步证明了范围过滤器在缓存价值建模、在线配额分配与系统资源优化中的应用潜力，为面向复杂负载的数据管理系统优化提供了可行的技术路径。

6.2 下一步工作

本文的下一步工作可以考虑从以下几个方面展开。

（1）支持更复杂的键类型与范围语义。当前自适应范围过滤器及其后缀编码

主要面向定长数值键设计，对字符串键、变长键以及更一般的可扩展范围表达的直接支持仍然不足。未来可进一步研究面向字符串有序编码、可变长分区与复合键排序规则的统一建模方法，使范围过滤器能够适配更广泛的数据类型与查询场景。

（2）研究资源超订场景下的冷启动与公平调度问题。本文的弹性缓存配额方法主要关注给定缓存预算下的在线价值评估与配额推荐，但当所有用户的期望配额之和长期高于底层可用资源时，系统在冷启动阶段会面临缓存竞争激烈、热点尚未充分观测以及短期性能波动放大的问题。未来可进一步结合准入控制、跨租户公平退让、共享预热池与超卖保护机制，研究总量受限条件下的稳态与冷启动一体化调度方法。

（3）加强范围过滤器与缓存调度的闭环协同优化。当前论文已将范围过滤器用于块密度估计与缓存价值修正，但过滤器参数、采样比例、滑动窗口长度与缓存配额策略仍主要按模块分别设计。未来可以研究端到端联合优化框架，使过滤器配置、自适应更新频率、价值预测参数与缓存配额决策在统一目标下协同调整，从而进一步提升系统整体的性能–成本收益。

（4）提升方法在真实复杂环境中的自学习与泛化能力。本文的参数设置和实验评估主要基于典型数据集与原型系统完成，对于更长时间跨度的真实生产负载、异构硬件平台以及多层缓存/多级存储联合场景仍有进一步验证空间。未来可引入在线学习或强化学习机制，自动调整 Holt-Winters 参数、采样策略与过滤器空间分配，并在更大规模、更复杂的生产环境中验证本文方法的稳定性与可推广性。

参考文献

[1] International Data Corporation (IDC). Content Creation in the Age of Generative AI: Implications for Value and Scale[Z]. White paper, sponsored by Seagate Technology, IDC #US53817625. https://www.seagate.com/content/dam/seagate/assets/resources/How-Generative-AI-Is-Transforming-Content-Creation-and-Data-Value-/files/US53817625-WP.pdf. 2025.

[2] International Data Corporation (IDC). Top 10 Worldwide Artificial Intelligence and Automation 2024 Predictions[Z]. https://blogs.idc.com/2024/01/03/top-10-worldwide-artificial-intelligence-and-automation-2024-predictions/. 2024.

[3] BONÉR J. Latency Numbers Every Programmer Should Know[Z]. Compiled from Jeff Dean and Peter Norvig, Google. https://gist.github.com/jboner/2841832. 2012.

[4] ROSER M, RITCHIE H. Historical Cost of Computer Memory and Storage[Z]. Our World in Data. https://ourworldindata.org/grapher/historical-cost-of-computer-memory-and-storage. 2023.

[5] GHOLAMI A, YAO Z, KIM S, et al. AI and Memory Wall[J]. IEEE Micro, 2024, 44(3): 33-39.

[6] FAN B, ANDERSEN D G, KAMINSKY M, et al. Cuckoo Filter: Practically Better Than Bloom[C]//CoNEXT. ACM, 2014: 75-88.

[7] BLOOM B H. Space/Time Trade-offs in Hash Coding with Allowable Errors[J]. Commun. ACM, 1970, 13(7): 422-426.

[8] BENDER M A, FARACH-COLTON M, JOHNSON R, et al. Don't Thrash: How to Cache Your Hash on Flash[J]. Proc. VLDB Endow., 2012, 5(11): 1627-1637.

[9] ZHANG H, LIM H, LEIS V, et al. SuRF: Practical Range Query Filtering with

Fast Succinct Tries[J]. Proc. ACM Manag. Data, 2018: 323-336.

[10] LUO S, CHATTERJEE S, KETSETSIDIS R, et al. Rosetta: A Robust Space-Time Optimized Range Filter for Key-Value Stores[J]. Proc. ACM Manag. Data, 2020: 2071-2086.

[11] VAIDYA K, KRASKA T, CHATTERJEE S, et al. SNARF: A Learning-Enhanced Range Filter[J]. Proc. VLDB Endow., 2022, 15(8): 1632-1644.

[12] ESLAMI N, DAYAN N. Memento Filter: A Fast, Dynamic, and Robust Range Filter[J]. Proc. ACM Manag. Data, 2024, 2(6): 244:1-244:27.

[13] SHEN J, ZUO P, LUO X, et al. Ditto: An elastic and adaptive memory-disaggregated caching system[C]//SOSP. ACM, 2023: 675-691.

[14] GUPTA A, KRISHNASWAMY R, KUMAR A, et al. Elastic Caching[C]// SODA. SIAM, 2019: 143-156.

[15] GU R, LI S, DAI H, et al. Adaptive online cache capacity optimization via lightweight working set size estimation at scale[C]//ATC. USENIX Association, 2023: 467-484.

[16] RZADCA K, FINDEISEN P, SWIDERSKI J, et al. Autopilot: workload autoscaling at Google[C]//EuroSys. ACM, 2020: 1-16.

[17] ZHAO S, TAN Y, ZENG Z, et al. LAShards: Low-Overhead and Self-Adaptive MRC Construction for Non-Stack Algorithms[J]. IEEE Trans. Comput., 2025.

[18] ZHANG Y, HUANG P, ZHOU K, et al. OSCA: An Online-Model based cache allocation scheme in cloud block storage systems[C]//ATC. USENIX Association, 2020: 785-798.

[19] ZHAO W, JIN X, WANG Z, et al. Low cost working set size tracking[C]//ATC. USENIX Association, 2011.

[20] KNORR E R, LEMAIRE B, LIM A, et al. Proteus: A Self-Designing Range Filter [J]. Proc. ACM Manag. Data, 2022: 1670-1684.

[21] BENDER M A, FARACH-COLTON M, GOSWAMI M, et al. Bloom Filters, Adaptivity, and the Dictionary Problem[C]//FOCS. IEEE Computer Society,

2018: 182-193.

[22] MITZENMACHER M, PONTARELLI S, REVIRIEGO P. Adaptive Cuckoo Filters[C]//ALENEX. SIAM, 2018: 36-47.

[23] WEN R, MCCOY H, TENCH D, et al. Adaptive Quotient Filters[J]. Proc. ACM Manag. Data, 2024, 2(4): 192:1-192:28.

[24] LIU Z, LEE H W, XIANG Y, et al. eMRC: Efficient Miss Ratio Approximation for Multi-Tier Caching[C]//FAST. USENIX Association, 2021: 293-308.

[25] GU R, LI S, DAI H, et al. Adaptive Online Cache Capacity Optimization via Lightweight Working Set Size Estimation at Scale[C]//ATC. USENIX Association, 2023: 467-484.

[26] KLIMOVIC A, WANG Y, STUEDI P, et al. Pocket: Elastic Ephemeral Storage for Serverless Analytics[C]//OSDI. USENIX Association, 2018: 427-444.

[27] MVONDO D, BACOU M, NGUETCHOUANG K, et al. OFC: An Opportunistic Caching System for FaaS Platforms[C]//EuroSys. ACM, 2021: 228-244.

[28] NATHAN V, SINGH V, LIU Z, et al. Intelligent Scaling in Amazon Redshift[C]//SIGMOD Companion. ACM, 2024.

[29] ZOU D, LU W, ZHU Z, et al. OptScaler: A Collaborative Framework for Robust Autoscaling in the Cloud[J]. Proc. VLDB Endow., 2024, 17(12): 4090-4103.

[30] O'NEIL P E, CHENG E, GAWLICK D, et al. The Log-Structured Merge-Tree (LSM-Tree)[J]. Acta Informatica, 1996, 33(4): 351-385.

[31] CHEN G, LI M, LUO S, et al. Oasis: An Optimal Disjoint Segmented Learned Range Filter[J]. Proc. VLDB Endow., 2024, 17(8): 1911-1924.

[32] COSTA M, FERRAGINA P, VINCIGUERRA G. Grafite: Taming Adversarial Queries with Optimal Range Filters[J]. Proc. ACM Manag. Data, 2024, 2(1): 3:1-3:23.

[33] MÖSSNER B, RIEGGER C, BERNHARDT A, et al. bloomRF: On Performing Range-Queries in Bloom-Filters with Piecewise-Monotone Hash Functions and Prefix Hashing[C]//EDBT. OpenProceedings.org, 2023: 131-143.

[34] WANG Z, ZHONG Z, GUO J, et al. REncoder: A Space-Time Efficient Range Filter with Local Encoder[C]//ICDE. IEEE, 2023: 2036-2049.

[35] GOSWAMI M, JØRGENSEN A G, LARSEN K G, et al. Approximate Range Emptiness in Constant Time and Optimal Space[C]//SODA. SIAM, 2015: 769-775.

[36] BRUCK J, GAO J, JIANG A. Weighted Bloom filter[C]//ISIT. IEEE, 2006: 2304-2308.

[37] WANG X, JI Y, DANG Z, et al. Improved Weighted Bloom Filter and Space Lower Bound Analysis of Algorithms for Approximated Membership Querying [C]//DASFAA. Springer, 2015: 346-362.

[38] DEEDS K, HENTSCHEL B, IDREOS S. Stacked Filters: Learning to Filter by Structure[J]. Proc. VLDB Endow., 2020, 14(4): 600-612.

[39] DAI Z, SHRIVASTAVA A. Adaptive Learned Bloom Filter (Ada-BF): Efficient Utilization of the Classifier with Application to Real-Time Information Filtering on the Web[C]//NeurIPS. Curran Associates, Inc., 2020: 11700-11710.

[40] RAE J W, BARTUNOV S, LILLICRAP T P. Meta-Learning Neural Bloom Filters[C]//ICML. PMLR, 2019: 5271-5280.

[41] KOPELOWITZ T, MCCAULEY S, PORAT E. Support Optimality and Adaptive Cuckoo Filters[C]//WADS. Springer, 2021: 556-570.

[42] LEE D J, MCCAULEY S, SINGH S, et al. Telescoping Filter: A Practical Adaptive Filter[C]//ESA. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021: 60:1-60:18.

[43] ZHONG W, CHEN C, WU X, et al. REMIX: Efficient Range Query for LSM-trees[C]//FAST. USENIX Association, 2021: 51-64.

[44] WANG H, GUO T, YANG J, et al. GRF: A Global Range Filter for LSM-Trees with Shape Encoding[J]. Proc. ACM Manag. Data, 2024, 2(3): 141.

[45] LIU F, GU R, LI M, et al. Hourglass: An Adaptive Range Filter with Lightweight Hybrid Encoding[J]. Proc. ACM Manag. Data, 2025, 3(4): 1-26.

[46] LI H. Alluxio: A virtual distributed file system[M]. University of California, Berkeley, 2018.

[47] ZHANG F, YIN C, FAN H, et al. AnalyticDB-PG: a cloud-native high-performance data warehouse in Alibaba Cloud[J]. Proc. VLDB Endow., 2025, 18(12): 5139-5152.

[48] XIONG J, CHEN H. Challenges for building a cloud native scalable and trustable multi-tenant AIoT platform[C]//ICCAD. ACM, 2020: 1-8.

[49] CHEEMA M A, BRANKOVIC L, LIN X, et al. Continuous Monitoring of Distance-Based Range Queries[J]. IEEE Trans. Knowl. Data Eng., 2011, 23(8): 1182-1199.

[50] LIU F, HUA K A. Moving Query Monitoring in Spatial Network Environments [J]. Mob. Networks Appl., 2012, 17(2): 234-254.

[51] DAI H, YE Q, YANG G, et al. CSRQ: Communication-Efficient Secure Range Queries in Two-Tiered Sensor Networks[J]. Sensors, 2016, 16(2): 259-265.

[52] CHEN G J, WIENER J L, IYER S, et al. Realtime Data Processing at Facebook [J]. Proc. ACM Manag. Data, 2016: 1087-1098.

[53] KELLER S, MUELLER R. Real-Time Range Query Approximation by Means of Adaptive Quad Streaming[C]//SENSORNETS. SCITEPRESS, 2021: 13-24.

[54] ALMASLUKH A, MAGDY A. Evaluating spatial-keyword queries on streaming data[C]//SIGSPATIAL. ACM, 2018: 209-218.

[55] WANG Y, JIN P, WAN S. HotKey-LSM: A Hotness-Aware LSM-Tree for Big Data Storage[C]//BigData. IEEE, 2020: 5849-5851.

[56] ZHANG T, TAN J, CAI X, et al. SA-LSM : Optimize Data Layout for LSM-tree Based Storage using Survival Analysis[J]. Proc. VLDB Endow., 2022, 15(10): 2161-2174.

[57] LI R, LIU A X, WANG A L, et al. Fast Range Query Processing with Strong Privacy Protection for Cloud Computing[J]. Proc. VLDB Endow., 2014, 7(14): 1953-1964.

[58] SU Y, MA W, SONG S. Learning Autoregressive Model in LSM-Tree based Store[C]//KDD. ACM, 2023: 2061-2071.

[59] RUI L, HUANG X, SONG S, et al. Time Series Representation for Visualization in Apache IoTDB[J]. Proc. ACM Manag. Data, 2024, 2(1): 35:1-35:26.

[60] CAO Z, CHEN S, LI F, et al. LogKV: Exploiting key-value stores for event log processing[C]//CIDR. ACM, 2013: 89-94.

[61] YANG L, WU H, ZHANG T, et al. Leaper: A Learned Prefetcher for Cache Invalidation in LSM-tree based Storage Engines[J]. Proc. VLDB Endow., 2020, 13(11): 1976-1989.

[62] TENG D, GUO L, LEE R, et al. LSbM-tree: Re-enabling buffer caching in data management for mixed reads and writes[C]//ICDCS. IEEE, 2017: 68-79.

[63] ZHAO X, LAM K, ZHU C, et al. MVLevelDB: Using Log-Structured Tree to Support Temporal Queries in IoT[J]. IEEE Internet Things J., 2022, 9(10): 7815-7825.

[64] LI M, XIE R, CHEN D, et al. A Pareto optimal Bloom filter family with hash adaptivity[J]. VLDB J., 2022, 32(3): 525-548.

[65] LI M, CHAI H, LUO S, et al. VEGA: An Active-tuning Learned Index with Group-Wise Learning Granularity[J]. Proc. ACM Manag. Data, 2025, 3(1): 86:1-86:26.

[66] LI M, CHEN D, DAI H, et al. Seesaw Counting Filter: An Efficient Guardian for Vulnerable Negative Keys During Dynamic Filtering[C]//WWW. ACM, 2022: 2759-2767.

[67] LI M, LUO W, DAI H, et al. The Reinforcement Cuckoo Filter[C]//INFOCOM. IEEE, 2024: 1940-1949.

[68] WANG H, DAI H, CHEN S, et al. Bamboo Filters: Make Resizing Smooth and Adaptive[J]. IEEE/ACM Trans. Netw., 2024, 32(5): 3776-3791.

[69] BRESLAU L, CAO P, FAN L, et al. Web Caching and Zipf-like Distributions:

Evidence and Implications[C]//INFOCOM. IEEE Computer Society, 1999: 126-134.

[70] CLAUSET A, SHALIZI C R, NEWMAN M E J. Power-Law Distributions in Empirical Data[J]. SIAM Rev., 2009, 51(4): 661-703.

[71] PANDEY P, BENDER M A, JOHNSON R, et al. A General-Purpose Counting Filter: Making Every Bit Count[J]. Proc. ACM Manag. Data, 2017: 775-787.

[72] CHAMBI S, LEMIRE D, KASER O, et al. Better bitmap performance with Roaring bitmaps[J]. Softw. Pract. Exp., 2016, 46(5): 709-719.

[73] ELIAS P. Efficient Storage and Retrieval by Content and Address of Static Files [J]. J. ACM, 1974, 21(2): 246-260.

[74] FANO R M. On the number of bits required to implement an associative memory [M]. Massachusetts Institute of Technology, Project MAC, 1971.

[75] ALEXIOU K, KOSSMANN D, LARSON P. Adaptive Range Filters for Cold Data: Avoiding Trips to Siberia[J]. Proc. VLDB Endow., 2013, 6(14): 1714-1725.

[76] ZHANG H. SuRF[Z]. https://github.com/efficient/SuRF. 2018.

[77] VAIDYA K. SNARF[Z]. https://github.com/kapilvaidya24/SNARF. 2022.

[78] KNORR E R. Proteus[Z]. https://github.com/Eric-R-Knorr/Proteus. 2022.

[79] WANG Z. REncoder[Z]. https://github.com/Range-Filter/REncoder. 2023.

[80] COSTA M. Grafite[Z]. https://github.com/marcocosta97/grafite. 2024.

[81] MÖSSNER B. BloomRF[Z]. https://github.com/awitten1/bloomRF. 2023.

[82] CHEN G. Oasis[Z]. https://github.com/Wooooooow-Pro/Oasis-RangeFilter. 2024.

[83] VITTER J S. Random sampling with a reservoir[J]. ACM Trans. Math. Softw., 1985, 11(1): 37-57.

[84] EFRAIMIDIS P S, SPIRAKIS P G. Weighted random sampling with a reservoir [J]. Inf. Process. Lett., 2006, 97(5): 181-185.

[85] DUFFIELD N, LUND C, THORUP M. Priority sampling for estimation of arbitrary subset sums[J]. J. ACM, 2007, 54(6): 1-42.

[86] ZHANG Y, YANG J, YUE Y, et al. SIEVE is simpler than LRU: an efficient Turn-Key eviction algorithm for web caches[C]//NSDI. USENIX, 2024: 1229-1246.

[87] SULTAN S, SHAKIBA K, LEE A, et al. TTLs matter: Efficient cache sizing with TTL-aware miss ratio curves and working set sizes[C]//EuroSys. ACM, 2024: 387-404.

[88] NAOR M, YOGEV E. Sliding Bloom Filters[J/OL]. CoRR, 2013, abs/1304.5872. arXiv: 1304.5872. http://arxiv.org/abs/1304.5872.

[89] WALDSPURGER C A, PARK N, GARTHWAITE A, et al. Efficient MRC construction with SHARDS[C]//FAST. USENIX Association, 2015: 95-110.

[90] CHEN P, CHEN D, ZHENG L, et al. Out of many we are one: Measuring item batch with clock-sketch[C]//Proc. ACM Manag. Data. 2021: 261-273.

[91] WIRES J, INGRAM S, DRUDI Z, et al. Characterizing storage workloads with counter stacks[C]//OSDI. USENIX, 2014: 335-349.

致　　谢

时光匆匆，研究生生活即将告一段落。回首在南京大学计算机学院求学的日日夜夜，从最初对近似数据结构的懵懂探索，到本论文逐章成稿，每一段进展背后都凝聚着许多人的关怀与支持。

首先，要将最深的谢意献给我的导师顾荣老师。从论文选题的反复推敲，到对每一张实验图、每一段表述的细致打磨，老师以严谨的治学态度为我树立了榜样，也以宽容与信任为我留出独立思考与试错的空间。正是在导师一次次”再想一层、再深一步”的引导下，本文中关于自适应范围过滤器、智能弹性缓存与多租户偏好优化的若干关键设计，才得以从粗糙的雏形逐步成型并在系统上得到验证。老师在为人处世上的从容与豁达，更将是我未来长期受益的精神财富。

衷心感谢李猛老师在系统实现与实验设计上提供的宝贵建议。在原型代码遇到瓶颈、实验数据出现反直觉现象的多次时刻，与他的讨论让我跳出局部思维，从更高维度重新审视问题。

感谢实验室的周盛文，周雨欣，李宇航等同学一路同行——深夜的代码调试、日常生活中的相互照应，都让这段研究旅程不再孤单。特别感谢贾佩玉、何慷、孙志强、熊锋等好友，在我陷入瓶颈与情绪低谷时给予的倾听与鼓励。也感谢答辩委员会各位老师在百忙之中评阅本文并提出宝贵意见，使本文得以进一步完善。

特别感谢我的父母和外公外婆。多年来，是你们的无条件支持与默默付出让我得以专注于学习与研究而无后顾之忧；你们朴实的话语与沉稳的目光，始终是我面对困难时最坚实的依靠。

最后，也想感谢一路坚持下来的自己。从面对陌生领域的忐忑，到一次次实验失败后的自我怀疑，再到深夜与论文逐字缠斗的安静时刻，是那个不甘心、不放弃的自己把这段旅程走完。愿未来无论身处何方，都能继续保持这份认真与韧性，不因一时得失而动摇，也不因前路未知而畏惧。

谨以此文，献给所有曾经支持、陪伴与帮助过我的人。

附录 A　成果列表

A.1　研究生期间发表的论文

[1] **Liu F**, Gu R, Li M, et al. Hourglass: An Adaptive Range Filter with Lightweight Hybrid Encoding. Proceedings of the ACM on Management of Data, 2026, 3(4): 1-26（SIGMOD，CCF-A，数据库领域旗舰会议）, 独立第一作者

A.2　研究生期间申请的专利

[1] **刘非凡**, 王宝晗, 于巍, 陈敬, 徐元慧, 李猛, 顾荣, 戴海鹏. 《一种自适应的范围查询过滤方法、系统、设备及存储介质》（第一发明人，申请号：2024117067642）

A.3　研究生期间参与的科研项目

[1] 国家自然科学基金面上项目《基于智能化分析方法的大数据系统软件性能优化研究》（项目负责人为指导老师）
[2] 南京大学——中国移动前沿探索课题《跨域缓存弹性分配机制》（项目负责人为指导老师）

A.4　研究生期间获得的奖项

[1] 第一届 CCF 算力网系统与应用大赛，国家级一等奖
[2] 2025 年国家奖学金，南京大学